Introduction to Security & Cryptography

Prepared By

Dr. Gehad Taher

Contents

Security and Cryptography Foundations

1. Introduction to Information Security

Learning Objectives

By the end of this chapter, the reader should be able to:

- Define information security and explain its importance.
- Identify the main goals of information security.
- Explain confidentiality, integrity, and availability.
- Differentiate between authentication, authorization, and accountability.
- Distinguish between threats, vulnerabilities, and attacks.
- Describe common types of security controls.
- Understand the basic idea of risk management.
- Recognize examples of real-world security incidents.
- Explain the purpose of the OSI Security Architecture.
- Classify security attacks as passive or active.
- Describe release of message contents, traffic analysis, masquerade, replay, message modification, and denial-of-service attacks.
- Distinguish security services from security mechanisms.
- Identify specific and pervasive security mechanisms.
- Explain the basic network-security model.

1.1 Introduction

Information has become one of the most valuable assets in modern society. Individuals, organizations, universities, hospitals, banks, companies, and governments all depend on information to make decisions, provide services, and manage daily operations. This information may include personal data, financial records, medical files, academic results, passwords, business plans, research data, and confidential communication.

As the use of digital systems increases, the need to protect information becomes more important.

Computers, networks, mobile phones, cloud platforms, and online services have made information easier to store, process, and share. However, they have also introduced new security risks. Attackers may try to steal data, modify records, interrupt services, or gain unauthorized access to systems.

Information security is the field concerned with protecting information and information systems from unauthorized access, misuse, disclosure, modification, destruction, or disruption. It is not limited to technical tools only. It also includes people, processes, policies, physical protection, software, hardware, and networks.

This chapter introduces the basic concepts of information security. It explains the main goals of security, known as the CIA triad, and discusses important concepts such as authentication, authorization, accountability, threats, vulnerabilities, attacks, security controls, and risk management.

1.2 What Is Information Security?

Information security, often abbreviated as InfoSec, is the practice of protecting information from unauthorized access, use, disclosure, modification, destruction, or disruption.

A simple definition is:

Information Security

Information security is the protection of information and information systems from unauthorized access, misuse, modification, or destruction.

Information security applies to both digital and non-digital information. For example, a printed exam paper stored in a locked cabinet also requires security. However, in this book, the main focus is on digital information and computer-based systems.

Information security aims to protect many types of information, such as:

- Personal information, such as names, addresses, phone numbers, and national identification numbers.
- Financial information, such as bank account details and credit card numbers.
- Medical records and health information.
- Academic records, exam results, and research data.
- Business secrets and intellectual property.
- Government and military information.
- Usernames, passwords, and cryptographic keys.

Without proper security, attackers may steal, modify, delete, or misuse this information. For example, if an attacker obtains a user's password, they may access the account and perform unauthorized actions. If a company loses customer data, it may face legal problems, financial loss, and damage to its reputation.

1.3 Why Information Security Matters

Information security is important because modern life depends heavily on digital systems. People use online banking, email, social media, e-commerce websites, mobile applications, cloud storage, and online learning platforms every day. Organizations also rely on databases, servers, networks, and cloud services to store and process sensitive information.

A failure in information security can lead to serious consequences, including:

- Loss of privacy.
- Financial loss.
- Identity theft.
- Damage to reputation.

- Disruption of services.
- Legal consequences.
- Loss of customer trust.
- Exposure of confidential information.

For example, if an attacker gains access to a university system, they may view student records, modify grades, or steal personal information. If a hospital system is attacked by ransomware, patient data may become unavailable, which may delay medical services. If an online banking system is compromised, customers may lose money or sensitive financial information.

Therefore, information security is not only a technical issue. It is also a legal, ethical, social, and business issue. Good security protects individuals, organizations, and society.

1.4 The CIA Triad

The three main goals of information security are commonly known as the CIA triad. CIA stands for:

- Confidentiality
- Integrity
- Availability

These three principles form the foundation of most security systems. A secure system should protect information from unauthorized access, prevent unauthorized modification, and ensure that information is available when needed.

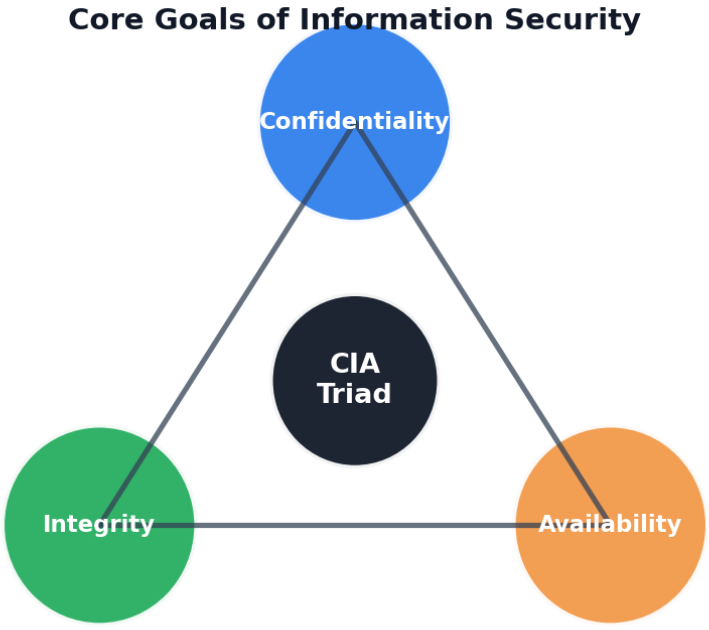


Figure 1.1: The CIA triad: confidentiality, integrity, and availability.

Confidentiality

Confidentiality means that information should be accessible only to authorized users. Unauthorized individuals should not be able to read, copy, or obtain sensitive data.

Examples of confidentiality include:

- A password should not be visible to other users.
- A bank account balance should be seen only by the account owner.
- A medical record should be accessed only by authorized healthcare workers.
- A private message should not be readable by attackers.
- A company's secret business plan should not be available to competitors.

Common methods used to protect confidentiality include:

- Encryption.
- Access control.
- Strong passwords.
- Multi-factor authentication.
- Secure communication protocols.
- Data classification.

In cryptography, confidentiality is mainly achieved through encryption. Encryption transforms readable data, called plaintext, into unreadable data, called ciphertext. Only someone with the correct key can decrypt the ciphertext and recover the original message.

Example

When a user sends a private message through a secure messaging application, encryption helps ensure that only the intended receiver can read the message.

Integrity

Integrity means that information should remain accurate, complete, and unchanged unless modified by an authorized user. If data is changed without permission, integrity is violated.

Examples of integrity include:

- A student's grade should not be changed by an unauthorized person.
- A bank transaction amount should not be modified during transmission.
- A software update should not be altered by an attacker.
- A legal document should remain unchanged after being signed.
- A medical prescription should not be modified without permission.

Methods used to protect integrity include:

- Hash functions.
- Digital signatures.
- Checksums.
- Access permissions.
- Version control.
- Audit logs.

Example

If a file is downloaded from the internet, a hash value can be used to check whether the file has been modified. If the hash value of the downloaded file is different from the original hash value, this may indicate that the file has been changed or corrupted.

Integrity is very important in financial systems, healthcare systems, academic systems, and legal systems because incorrect or modified data can lead to serious consequences.

Availability

Availability means that information and systems should be accessible to authorized users when needed. Even if information is confidential and accurate, it is not useful if users cannot access it.

Examples of availability include:

- A banking website should be available when customers need it.
- A university learning platform should be accessible during exams.
- A hospital system should be operational during emergencies.
- A cloud storage service should allow users to access their files.
- An email service should be available for communication.

Threats to availability include:

- Denial-of-service attacks.
- Hardware failures.
- Power outages.
- Network failures.
- Malware infections.
- Natural disasters.
- Poor system maintenance.

Availability can be improved by:

- Regular backups.
- Redundant systems.
- Disaster recovery plans.
- Load balancing.
- Regular maintenance.
- Protection against denial-of-service attacks.
- Reliable power and network infrastructure.

Example

If a university online learning system becomes unavailable during final exams, students may not be able to submit their work. This would be a failure of availability.

1.5 Authentication, Authorization, and Accountability

In addition to the CIA triad, information security also depends on three important concepts: authentication, authorization, and accountability.

Authentication

Authentication is the process of verifying the identity of a user, device, or system.

In simple words, authentication answers the question:

Authentication Question

Who are you?

Examples of authentication include:

- Logging in with a username and password.
- Using a fingerprint to unlock a phone.
- Entering a one-time password sent by SMS or email.
- Using a smart card to access a secure building.

- Using face recognition to access a mobile device.

Authentication factors are usually classified into three categories:

- Something you know, such as a password or PIN.
- Something you have, such as a smart card, phone, or security token.
- Something you are, such as a fingerprint, face, or iris pattern.

Using more than one authentication factor is called multi-factor authentication. Multi-factor authentication provides stronger protection than using a password alone because an attacker would need to compromise more than one factor.

Authorization

Authorization is the process of determining what an authenticated user is allowed to do.

Authorization answers the question:

> **Authorization Question**
>
> What are you allowed to access?

For example, a student may be allowed to view their own grades, but not modify them. A teacher may be allowed to enter grades for students in their course. An administrator may be allowed to create accounts, reset passwords, and manage system settings.

Authorization is usually managed using:

- Permissions.
- Roles.
- Access control lists.
- Security policies.

Authentication and authorization are related, but they are not the same. Authentication verifies identity, while authorization determines permissions.

Accountability

Accountability means that actions performed by users can be traced and linked to them.

Accountability answers the question:

> **Accountability Question**
>
> Who did what, and when?

Accountability is important because it helps organizations detect misuse, investigate incidents, and enforce policies.

Accountability can be achieved using:

- Log files.
- Audit trails.
- User activity monitoring.
- Digital signatures.
- Unique user accounts.

For example, if a database record is modified, the system should record which user made the change and at what time. This information can be useful during a security investigation.

1.6 Threats, Vulnerabilities, and Attacks

To understand information security, it is important to distinguish between three related terms: threat, vulnerability, and attack.

Threat

A threat is anything that has the potential to cause harm to a system, organization, or user. A threat may or may not happen, but it represents a possible danger.

Examples of threats include:

- Cybercriminals.
- Malware.
- Insider misuse.
- Natural disasters.
- System failures.
- Human mistakes.
- Power failure.

For example, a hacker who wants to steal passwords is considered a threat.

Vulnerability

A vulnerability is a weakness in a system that can be exploited by a threat.

Examples of vulnerabilities include:

- Weak passwords.
- Unpatched software.
- Poor network configuration.
- Lack of encryption.
- Insecure web forms.
- Missing access controls.
- Poor physical security.

For example, if users choose simple passwords such as `123456`, the system has a password-related vulnerability.

Attack

An attack is an intentional attempt to exploit a vulnerability and cause harm.

Examples of attacks include:

- Phishing attacks.
- Password guessing attacks.
- Malware attacks.
- Denial-of-service attacks.
- Man-in-the-middle attacks.
- SQL injection attacks.

The relationship between these concepts can be explained as follows:

Threat, Vulnerability, and Attack Relationship

A threat uses an attack to exploit a vulnerability and cause damage.

For example:

- Threat: A cybercriminal.
- Vulnerability: A weak password.
- Attack: Password guessing.

- Impact: Unauthorized access to an account.

Understanding the difference between threats, vulnerabilities, and attacks helps security professionals choose suitable protection methods.

1.7 The OSI Security Architecture

The Open Systems Interconnection Security Architecture provides a systematic framework for discussing communication security. It does not define one security product or one encryption algorithm. Instead, it organizes security concepts into three related areas:

- Security attacks.
- Security services.
- Security mechanisms.

OSI Security Architecture

The OSI Security Architecture is a framework that organizes communication security requirements by identifying security attacks, security services, and the mechanisms used to provide those services.

The framework helps students answer three questions:

1. What type of attack threatens the system?
2. What security service is needed to respond to that threat?
3. What security mechanism can provide the required service?

For example, an attacker may try to read a private message. This is a confidentiality threat. The required security service is data confidentiality, and encryption is one mechanism that can help provide that service.

Connecting Attacks, Services, and Mechanisms

Consider a student sending an assignment through a university portal.

- **Possible attack:** An attacker reads or changes the assignment during transmission.
- **Required services:** Confidentiality, integrity, and origin authentication.
- **Possible mechanisms:** Encryption, message authentication, digital signatures, and secure communication protocols.

R **Important Point.** A security service describes the protection that is required. A security mechanism describes how that protection is provided.

1.8 Classification of Security Attacks

A security attack is an action that attempts to compromise the confidentiality, integrity, availability, authenticity, or proper operation of information and systems.

Security attacks can be classified into two broad categories:

- Passive attacks.
- Active attacks.

Passive Attacks

A passive attack attempts to learn information without modifying data or disrupting system operation. The attacker observes communications but tries to remain undetected.

Passive Attack

A passive attack attempts to obtain information from a system or communication without changing the data or affecting normal operation.

Passive attacks are often difficult to detect because the communication may continue normally. Protection therefore focuses strongly on prevention, especially encryption and traffic-protection techniques.

The two main passive attacks are release of message contents and traffic analysis.

Release of Message Contents

Release of message contents occurs when an unauthorized party reads information being transmitted or stored.

Examples include:

- Reading an unencrypted email.
- Capturing passwords sent through an insecure network.
- Listening to an unprotected voice call.
- Viewing confidential files without authorization.

Release of Message Contents

A student connects to an untrusted wireless network and submits login information through an insecure connection. An attacker monitoring the network may read the username and password.

Traffic Analysis

Traffic analysis occurs when an attacker studies communication patterns even when the message contents are encrypted.

The attacker may observe:

- Which systems communicate.
- When communication occurs.
- How frequently messages are sent.
- The size of transmitted messages.
- The duration of communication sessions.

This information may reveal relationships, operational patterns, or important events without revealing the plaintext itself.

Traffic Analysis

An attacker cannot read encrypted messages between a company and an incident-response provider. However, a sudden increase in communication may suggest that the company is handling a security incident.

Active Attacks

An active attack attempts to modify data, create false data, interrupt services, impersonate another entity, or otherwise affect system operation.

Active Attack

An active attack attempts to alter system resources, modify communication, create unauthorized data, or disrupt normal operation.

Active attacks are often difficult to prevent completely. Security therefore focuses on prevention where possible, rapid detection, containment, and recovery.

The main active attacks are masquerade, replay, message modification, and denial of service.

Masquerade

A masquerade attack occurs when an attacker pretends to be another user, device, or system.

Examples include:

- Logging in with stolen credentials.
- Sending messages from a compromised account.
- Creating a fake website that imitates a legitimate service.
- Spoofing a trusted device identity.

Strong authentication, certificate validation, and secure credential management help reduce masquerade attacks.

Replay

A replay attack occurs when an attacker captures a valid message or transaction and sends it again later to produce an unauthorized result.

Replay Attack

An attacker records a valid electronic payment request and sends the same request again. If the system does not use unique transaction identifiers, timestamps, counters, or nonces, the payment may be processed twice.

Common replay protections include:

- Nonces.
- Timestamps.
- Sequence numbers.
- Short validity periods.
- Unique transaction identifiers.

Message Modification

Message modification occurs when an attacker changes, inserts, deletes, reorders, or delays information.

Examples include:

- Changing a bank-transfer amount.
- Modifying a student's submitted assignment.
- Altering software during download.
- Changing a destination account number.

Hash-based message authentication, authenticated encryption, digital signatures, and secure protocols can help detect unauthorized modifications.

Denial of Service

A denial-of-service attack attempts to make a service or resource unavailable to legitimate users. A distributed denial-of-service attack uses many systems to generate attack traffic.

Examples include:

- Flooding a website with excessive requests.
- Exhausting server memory or connection capacity.
- Disrupting a network link.
- Preventing users from accessing an online examination system.

Possible defenses include capacity planning, filtering, rate limiting, redundancy, traffic monitoring, and specialized denial-of-service protection services.

Passive and Active Attack Comparison

Feature	Passive Attack	Active Attack
Main goal	Observe or learn information	Modify, impersonate, replay, or disrupt
Effect on data	Normally no direct change	Data or operation may be changed
Detection	Often difficult	May produce visible effects
Primary response	Prevention	Prevention, detection, and recovery
Examples	Eavesdropping and traffic analysis	Masquerade, replay, modification, and denial of service

1.9 Security Services

A security service provides a particular type of protection for systems, data, or communication. A service describes the security result that is needed, not the technical implementation used to achieve it.

The main security services are:

- Authentication.
- Access control.
- Data confidentiality.
- Data integrity.
- Non-repudiation.
- Availability.

Authentication Service

The authentication service provides assurance about identity or data origin.

Two important forms are:

- **Peer-entity authentication:** verifies the identity of a communicating party during a connection.
- **Data-origin authentication:** provides assurance about the source of a message or data unit.

Access Control Service

Access control prevents unauthorized use of information, systems, applications, and other resources. It determines who may access a resource and which operations are permitted.

Data Confidentiality Service

Data confidentiality protects information against unauthorized disclosure.

It can include:

- Connection confidentiality.
- Connectionless confidentiality.

- Selective-field confidentiality.
- Traffic-flow confidentiality.

Data Integrity Service

Data integrity provides assurance that information has not been modified, inserted, deleted, reordered, or replayed without authorization.

Integrity services may detect modification only, or they may support recovery after a violation is detected.

Non-Repudiation Service

Non-repudiation provides evidence intended to reduce the ability of a party to falsely deny sending, receiving, or approving data.

Examples include:

- Non-repudiation of origin.
- Non-repudiation of receipt.

Digital signatures can support non-repudiation evidence, but effective non-repudiation also depends on identity verification, private-key protection, timestamps, logging, policy, and legal context.

Availability Service

Availability ensures that authorized users can access systems and information when required. Availability depends on resilience, redundancy, maintenance, backups, monitoring, recovery planning, and protection against disruption.

1.10 Security Mechanisms

A security mechanism is a process, technique, device, or control designed to detect, prevent, or recover from a security attack.

Security Mechanism

A security mechanism is a method used to implement one or more security services and respond to security attacks.

The OSI Security Architecture distinguishes between specific security mechanisms and pervasive security mechanisms.

Specific Security Mechanisms

Specific mechanisms can be associated with particular communication services or protocol operations.

- **Encipherment:** transforms data using an encryption algorithm and key.
- **Digital signature:** provides integrity and origin-authentication evidence.
- **Access control:** enforces authorized use of resources.
- **Data-integrity mechanisms:** detect unauthorized changes.
- **Authentication exchange:** allows communicating entities to verify identities.
- **Traffic padding:** adds extra traffic to make communication patterns less informative.
- **Routing control:** selects or restricts communication paths according to security requirements.
- **Notarization:** uses a trusted third party to provide evidence about communication or transactions.

Pervasive Security Mechanisms

Pervasive mechanisms support security across multiple layers, systems, or services.

- **Trusted functionality:** system components relied upon to enforce security correctly.
- **Security labels:** classifications or attributes attached to information and resources.
- **Event detection:** identifies security-relevant events.
- **Security audit trail:** records actions and events for accountability and investigation.
- **Security recovery:** restores a secure state after failure or attack.

One Service May Use Several Mechanisms

A secure website may provide server authentication, confidentiality, and integrity by combining certificates, digital signatures, key exchange, encryption, authenticated encryption, logging, and certificate validation.

1.11 Basic Network-Security Model

A network-security model describes how two parties communicate securely across a channel that may be observed or attacked.

The model normally contains:

- A sender.
- A receiver.
- A message or data stream.
- A security transformation.
- Secret or public keying information.
- A communication channel.
- A possible attacker.
- In some systems, a trusted third party.

The sender applies a security transformation before transmission. The receiver applies the corresponding operation to recover or verify the data.

A simplified confidentiality model is:

$$\text{Plaintext} + \text{Key} \xrightarrow{\text{Encryption}} \text{Ciphertext}$$

$$\text{Ciphertext} + \text{Key} \xrightarrow{\text{Decryption}} \text{Plaintext}$$

A secure communication design must also answer:

1. Which algorithm or protocol will be used?
2. How will keys be generated?
3. How will keys be distributed or established?
4. How will public keys and identities be authenticated?
5. How will replay, modification, and impersonation be prevented?
6. How will compromised keys be replaced?

R **Important Point.** A strong algorithm is not enough. Secure communication also requires key management, authentication, correct implementation, and suitable operational procedures.

1.12 Security Controls

Security controls are measures used to reduce risk and protect systems. They can be technical, physical, or administrative.

Security controls are often divided into three main categories:

- Preventive controls.
- Detective controls.
- Corrective controls.

Preventive Controls

Preventive controls are designed to stop security incidents before they happen.

Examples include:

- Password policies.
- Firewalls.
- Access control systems.
- Encryption.
- Security awareness training.
- Multi-factor authentication.
- Physical locks.

For example, a firewall can prevent unauthorized network traffic from reaching an internal system.

Detective Controls

Detective controls are designed to discover security incidents after they occur or while they are happening.

Examples include:

- Intrusion detection systems.
- Log monitoring.
- Security cameras.
- Antivirus alerts.
- File integrity monitoring.
- Audit reports.

For example, an intrusion detection system may generate an alert when suspicious network activity is detected.

Corrective Controls

Corrective controls are designed to reduce damage and restore normal operations after a security incident.

Examples include:

- Backups.
- Disaster recovery plans.
- Incident response procedures.
- System patches.
- Password resets.
- Malware removal tools.

For example, if ransomware encrypts files, a backup can help restore the affected data.

1.13 Security Policies and Risk Management

A security policy is a formal document that defines rules for protecting information and systems. It tells users what is allowed, what is not allowed, and what responsibilities they have.

Examples of security policies include:

- Password policy.
- Acceptable use policy.
- Data classification policy.
- Remote access policy.
- Backup policy.
- Incident response policy.

Policies are important because technology alone cannot guarantee security. Users must understand how to behave securely and follow clear rules.

Risk management is the process of identifying, analyzing, and reducing security risks. A risk exists when a threat can exploit a vulnerability and cause harm.

A simple risk management process includes:

1. Identify assets.
2. Identify threats.
3. Identify vulnerabilities.
4. Estimate the likelihood of an attack.
5. Estimate the possible impact.
6. Choose suitable security controls.
7. Monitor and review the risk regularly.

Risk Formula

A common way to describe risk is:

$$\text{Risk} = \text{Likelihood} \times \text{Impact}$$

For example, if a university database contains sensitive student records and has weak access control, the risk may be high because the impact of a data breach could be serious.

Risk can be managed in different ways:

- Avoid the risk by stopping the risky activity.
- Reduce the risk by applying security controls.
- Transfer the risk, for example by using insurance or outsourcing.
- Accept the risk if it is low or unavoidable.

Effective risk management helps organizations use their resources wisely. Instead of trying to protect everything equally, organizations can focus on the most important assets and the most serious risks.

1.14 Real-World Security Incidents

Real-world security incidents show that information security is necessary in practice. Security failures can affect individuals, companies, universities, hospitals, and governments.

Examples of common security incidents include:

- A company database is leaked because of weak access controls.
- Users lose money because of phishing emails.
- A hospital system is affected by ransomware.

- A website becomes unavailable because of a denial-of-service attack.
- A mobile application exposes user data because of poor encryption.
- An employee accidentally sends confidential files to the wrong person.

These incidents demonstrate the importance of applying security principles from the beginning of system design. Security should not be added only after a system is attacked. Instead, it should be considered during planning, development, deployment, and maintenance.

For example, a web application should be designed with secure authentication, proper input validation, encrypted communication, strong access control, logging, and regular updates. These measures reduce the chance of successful attacks.

1.15 Practical Activity

Self-Study Activity

Choose an online system that you use regularly, such as an email account, online banking application, university portal, or cloud storage service. Analyze it using the following questions:

1. What type of information does the system store?
2. What confidentiality risks may exist?
3. What integrity risks may exist?
4. What availability risks may exist?
5. What authentication method does the system use?
6. What security controls can you identify?
7. What improvements would you suggest?

This activity helps connect the concepts of this chapter with real systems used in daily life.

Group Activity: Attack, Service, and Mechanism Mapping

Group Size: 3 to 5 students
Total Marks: 10 marks

For each scenario below, identify the attack category, affected security property, required security service, and suitable security mechanisms.

1. An attacker reads unencrypted student records.
2. An attacker records and resends a payment request.
3. A fake university portal collects user passwords.
4. An attacker changes the destination account in a transfer message.
5. A flood of requests makes the learning platform unavailable.
6. An attacker cannot read encrypted traffic but studies its timing and size.

Marking Scheme

Assessment Item	Marks
Correct passive or active classification	2
Correct identification of the attack type	2
Correct affected security property	2
Appropriate security service	2
Appropriate mechanisms and clear explanation	2

1.16 Chapter Summary

In this chapter, we introduced the basic concepts of information security. Information security is the practice of protecting information from unauthorized access, modification, destruction, and disruption.

The main goals of information security are confidentiality, integrity, and availability. Confidentiality ensures that only authorized users can access information. Integrity ensures that information remains accurate and unchanged unless modified by authorized users. Availability ensures that systems and data are accessible when needed.

We also discussed authentication, authorization, and accountability. Authentication verifies identity, authorization determines permissions, and accountability makes user actions traceable.

The chapter explained the difference between threats, vulnerabilities, and attacks. A threat is a possible danger, a vulnerability is a weakness, and an attack is an attempt to exploit a vulnerability.

Finally, we introduced security controls, policies, and risk management. Security controls help prevent, detect, and correct security incidents. Risk management helps organizations identify and reduce security risks in a systematic way.

Information security provides the general principles needed to protect digital systems. However, many of these principles depend on cryptography. Encryption, hashing, digital signatures, and secure key exchange are essential tools for achieving confidentiality, integrity, authentication, and non-repudiation. In the next chapter, we introduce the fundamentals of cryptography and explain how cryptographic techniques help protect digital information.

1.17 Review Questions

1. What is information security?
2. Why is information security important in modern systems?
3. What are the three main goals of the CIA triad?
4. Explain confidentiality and give one example.
5. Explain integrity and give one example.
6. Explain availability and give one example.
7. What is the difference between authentication and authorization?
8. What is accountability, and why is it important?
9. Define threat, vulnerability, and attack.
10. Give an example that shows the relationship between a threat, a vulnerability, and an attack.
11. What are preventive controls? Give two examples.
12. What are detective controls? Give two examples.
13. What are corrective controls? Give two examples.
14. What is a security policy?
15. What is risk management?
16. Explain the formula: $Risk = Likelihood \times Impact$.
17. Why should security be considered during system design?
18. What is the purpose of the OSI Security Architecture?
19. What is the difference between a security service and a security mechanism?
20. What is the difference between a passive attack and an active attack?
21. Explain release of message contents and traffic analysis.
22. Explain masquerade, replay, message modification, and denial of service.
23. What is peer-entity authentication?
24. What is data-origin authentication?
25. List the main specific security mechanisms.
26. List the main pervasive security mechanisms.

27. Describe the main components of a network-security model.

2. Fundamentals of Cryptography

Learning Objectives

By the end of this chapter, the reader should be able to:

- Define cryptography, cryptanalysis, and cryptology.
- Define plaintext, ciphertext, encryption, decryption, cipher, algorithm, and key.
- Explain the relationship between algorithms and keys.
- Explain Kerckhoffs's principle.
- Describe how cryptography supports confidentiality, integrity, authentication, and non-repudiation evidence.
- Compare symmetric cryptography, asymmetric cryptography, and cryptographic hash functions.
- Explain key space and brute-force attacks.
- Distinguish ciphertext-only, known-plaintext, chosen-plaintext, and chosen-ciphertext attack models.
- Describe the cryptographic key lifecycle.
- Identify common applications, limitations, and implementation mistakes in cryptography.

2.1 Introduction

Information security depends on many technical and nontechnical controls. Access control, security policies, logging, backups, user awareness, secure software development, and physical protection all contribute to a secure system. Cryptography is one of the most important technical tools in this larger security program.

Cryptography can protect information while it is stored, processed, or transmitted. It can help prevent unauthorized disclosure, detect unauthorized changes, authenticate communicating parties, and provide evidence that a private-key holder approved particular data.

Modern cryptography is not based on hiding the design of an algorithm. Strong algorithms are normally published, examined, tested, and analyzed by experts. Security should depend mainly on properly generated and protected keys, secure parameters, trusted implementations, and correct protocol design.

This chapter introduces the language, goals, attack classifications, security services, security mechanisms, and general models needed before studying individual cryptographic algorithms.

2.2 What Is Cryptography?

Cryptography

Cryptography is the science and practice of protecting information using mathematical transformations, algorithms, keys, and protocols.

The word *cryptography* comes from terms associated with hidden writing. In modern computing, however, cryptography is used for more than hiding message contents. It also supports integrity checking, authentication, digital signatures, secure key establishment, and certificate-based trust.

A cryptographic system may protect:

- Text messages.
- Files and databases.
- Network traffic.
- Stored passwords.
- Financial transactions.
- Software updates.
- Digital certificates.
- Authentication tokens.
- Backups and cloud data.

Secure Communication Scenario

Suppose Alice sends a message to Bob across an untrusted network. An attacker may capture the traffic, modify it, replay an old message, or replace Bob's public key.
A secure system may combine encryption, message authentication, digital signatures, certificates, nonces, timestamps, and key-management procedures to reduce these risks.

2.3 Cryptography, Cryptanalysis, and Cryptology

Cryptography and cryptanalysis have different goals.

Cryptanalysis

Cryptanalysis is the study of techniques for analyzing or defeating cryptographic protection without being given the required secret information.

A cryptanalyst may try to:

- Recover plaintext from ciphertext.
- Recover a secret key.
- Forge an authentication value or digital signature.
- Find two inputs with the same hash digest.

- Predict a supposedly random value.
- Exploit a weakness in a protocol or implementation.

Cryptology

Cryptology is the broader field that includes both cryptography and cryptanalysis.

Cryptanalysis is valuable because it helps researchers identify weak algorithms, incorrect assumptions, and implementation errors. A cryptographic method should be studied carefully before it is trusted.

2.4 Basic Cryptographic Terminology

Plaintext

Plaintext is the original readable or usable data before encryption.

Examples include:

- A text message.
- A photograph.
- A database record.
- A document.
- A symmetric session key.

If the original message is:

```
MEET AT 10 AM
```

then this readable message is the plaintext.

Ciphertext

Ciphertext is the protected output produced by an encryption process. It should not reveal useful plaintext information to an unauthorized observer when the system is used correctly.

Encryption

Encryption

Encryption is the process of transforming plaintext into ciphertext using an encryption algorithm and a key.

The general encryption relationship is:

$$C = E_K(P)$$

where:

- P is the plaintext.
- K is the encryption key.
- E is the encryption algorithm.
- C is the ciphertext.

Decryption

Decryption

Decryption is the process of transforming ciphertext back into plaintext using the required decryption key and algorithm.

The general relationship is:

$$P = D_K(C)$$

For a correct symmetric encryption system:

$$D_K(E_K(P)) = P$$

Cipher and Algorithm

A cipher is an algorithm used to perform encryption and decryption. More generally, a cryptographic algorithm may perform encryption, hashing, message authentication, key derivation, key agreement, signing, or signature verification.

Cryptographic Key

Cryptographic Key

A cryptographic key is a value that controls the operation or output of a cryptographic algorithm.

Two users may apply the same algorithm to the same plaintext using different keys and obtain different results. The key is therefore a central part of the security system.

Key Space

The key space is the set of all possible valid keys for an algorithm. If an algorithm uses an n-bit key and all bit strings are valid, the theoretical key space contains:

$$2^n$$

possible keys.
For example, a 4-bit educational key has:

$$2^4 = 16$$

possible values. Such a key can be searched immediately and provides no real security.

R **Important Point.** Key length affects resistance to exhaustive search, but key length alone does not guarantee security. Algorithm design, randomness, implementation, protocols, and key management also matter.

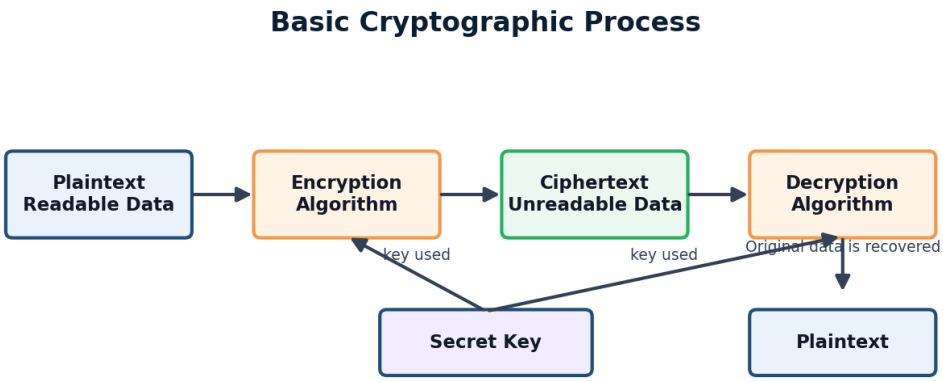


Figure 2.1: A basic cryptographic process using plaintext, an algorithm, a key, ciphertext, and decryption.

2.5 A Basic Cryptographic Process

A basic encryption process contains:

1. A sender prepares plaintext.
2. An encryption algorithm processes the plaintext using a key.
3. The system produces ciphertext.
4. The ciphertext is stored or transmitted.
5. An authorized receiver applies the correct decryption operation.
6. The receiver recovers the plaintext.

The communication channel may be insecure. An attacker may be able to observe or change the ciphertext. A secure design therefore needs more than confidentiality. It may also require authentication, integrity protection, replay protection, and trusted key distribution.

2.6 Kerckhoffs's Principle

A cryptographic system should remain secure even if an attacker knows how the system works, except for the secret key.

Kerckhoffs's Principle

A cryptographic system should not depend on secrecy of its algorithm. Its security should depend primarily on the secrecy and proper management of the required key material.

This principle is important because hidden algorithms may eventually become known through reverse engineering, leaks, documentation, or observation. Publicly studied algorithms benefit from extensive analysis by researchers and implementers.

R **Important Point.** A secret algorithm is not a substitute for a strong key and a secure design.

2.7 Main Goals of Cryptography

Cryptography supports several security goals, but different tools provide different properties.

Confidentiality

Confidentiality ensures that information is not disclosed to unauthorized parties. Encryption is the principal cryptographic tool used for confidentiality.

Examples include:

- Encrypting a laptop disk.
- Protecting messages in transit.
- Encrypting database fields.
- Protecting backups.

Integrity

Integrity allows unauthorized changes to be detected. Hash functions, message authentication codes, authenticated encryption, and digital signatures can support integrity.

Encryption without authentication does not necessarily provide reliable integrity protection.

Authentication

Authentication provides evidence about an identity or the origin of data. Authentication may involve passwords, MACs, digital signatures, certificates, or authenticated protocols.

Two related forms are:

- **Peer-entity authentication:** provides evidence about the identity of a communicating party.
- **Data-origin authentication:** provides evidence about the source of a message or data unit.

Non-Repudiation Support

Digital signatures can provide evidence that a particular private key was used to approve specific data. Strong non-repudiation also depends on identity proofing, key protection, timestamps, audit records, organizational policy, and legal context.

Access Control Support

Cryptography can support access control by protecting credentials, tokens, keys, and communication channels. However, cryptography does not decide all permissions by itself. Authorization policy and application logic are also required.

Availability Limitations

Cryptography can help protect systems, but it cannot guarantee availability. Encrypted information may still become unavailable because of hardware failure, denial-of-service attacks, deleted keys, ransomware, or operational mistakes.

2.8 Types of Cryptographic Systems

Cryptographic tools can be grouped according to their key usage and purpose.

Symmetric Key Cryptography

Symmetric cryptography uses a shared secret key. The same key, or directly related secret keys, are used for encryption and decryption.

Symmetric Key Cryptography

Symmetric key cryptography uses a shared secret key to protect and recover data.

Advantages include:

- High speed.
- Efficient bulk-data encryption.
- Suitability for files, disks, databases, and network sessions.

The main challenge is secure key distribution and management.

Examples include AES and ChaCha20. DES and Triple DES are historically important but should not be selected for new sensitive-data systems.

Asymmetric Key Cryptography

Asymmetric cryptography uses a public key and a private key.

> **Asymmetric Key Cryptography**
>
> Asymmetric key cryptography uses mathematically related public and private keys for encryption, key establishment, digital signatures, or related operations.

It supports:

- Public key encryption.
- Digital signatures.
- Authenticated key establishment.
- Certificate-based identity systems.

Examples include RSA and elliptic curve systems. Diffie–Hellman is a key-agreement method rather than a general bulk-encryption algorithm.

Cryptographic Hash Functions

A cryptographic hash function does not normally use a secret key. It maps input data to a fixed-length digest.

Hash functions support integrity checking, digital signatures, fingerprinting, message-authentication constructions, and password-hashing systems. Hashing is not encryption and is not designed to recover the original input.

Comparison

Type	Key Model	Main Purpose	Examples
Symmetric	Shared secret	Bulk confidentiality and authenticated encryption	AES, ChaCha20
Asymmetric	Public/private pair	Key establishment, signatures, and small-value protection	RSA, Diffie–Hellman, ECC
Hash function	Normally unkeyed	Digests, integrity support, and fingerprints	SHA-256, SHA-3

2.9 Cryptanalysis and Attack Models

The amount of information available to an attacker affects the attack model.

Ciphertext-Only Attack

The attacker has one or more ciphertexts but does not know the corresponding plaintexts or secret key.

Known-Plaintext Attack

The attacker knows some plaintext and the corresponding ciphertext. The attacker uses these pairs to analyze the system or recover other protected information.

Chosen-Plaintext Attack

The attacker can select plaintexts and obtain their ciphertexts. Modern encryption schemes should be designed to remain secure under strong chosen-plaintext models appropriate to their intended use.

Chosen-Ciphertext Attack

The attacker can select ciphertexts and obtain information about their decryption. Secure public key encryption schemes use carefully designed encodings and protocols to resist relevant chosen-ciphertext attacks.

Side-Channel Attack

A side-channel attack studies information leaked by an implementation, such as timing, power use, cache behavior, electromagnetic emissions, or error responses.

Important Point. A mathematically strong algorithm may still fail because of an insecure protocol, weak randomness, implementation error, or side-channel leakage.

2.10 Brute-Force Attacks

A brute-force or exhaustive-key-search attack tries possible keys until a meaningful result is found.

For an n-bit key, the theoretical number of possible keys may be:

$$2^n$$

On average, an attacker may expect to test a substantial portion of the key space before finding the correct key. Actual attack cost depends on:

- Key length.
- Hardware speed.
- Parallel processing.
- Algorithm cost per guess.
- Availability of known plaintext or validity checks.
- Weaknesses that reduce the effective key space.

Caesar Cipher Brute Force

The Caesar cipher has only 25 useful nonzero shifts. An attacker can test every shift and identify the readable plaintext. This tiny key space makes the cipher unsuitable for real security.

2.11 Cryptographic Keys and Key Management

Key management is the process of controlling cryptographic keys throughout their lifecycle.

Key Management

Key management includes secure key generation, establishment, distribution, storage, use, rotation, backup, revocation, archival, and destruction.

Key Generation

Keys should be created using an appropriate cryptographically secure random process. Predictable keys can defeat otherwise strong algorithms.

Key Distribution and Establishment

Keys may be distributed or established using:

- A protected physical process.
- A trusted key-distribution service.
- Public key encryption.
- Authenticated key agreement.
- Pre-shared keys installed securely.

Key Storage

Keys should be stored separately from ordinary application data whenever practical and protected by strong access controls. Sensitive keys may be stored in hardware security modules, trusted platform modules, smart cards, or protected key vaults.

Key Usage

Keys should be used only for approved purposes. One key should not automatically be reused for encryption, signing, message authentication, and unrelated applications.

Key Rotation and Revocation

Keys may need replacement because of expiration, policy, algorithm transition, role changes, or suspected compromise. Public-key certificates may need revocation when associated private keys are compromised.

Key Destruction

Retired keys should be destroyed securely when they are no longer needed. However, decryption keys may need controlled archival if previously encrypted organizational data must remain recoverable.

R **Important Point.** A strong algorithm cannot protect data if the required secret key is exposed, predictable, misused, or lost.

2.12 Common Applications of Cryptography

Cryptography appears in many systems:

- HTTPS and TLS.
- Secure messaging.
- Online banking and electronic payment systems.
- Virtual private networks.
- Wi-Fi security.

- Disk, file, and database encryption.
- Cloud storage protection.
- Password-verification systems.
- Digital signatures and electronic documents.
- Software and firmware updates.
- Digital certificates and public key infrastructure.
- Authentication tokens and API protection.

Most real systems combine multiple cryptographic methods. For example, TLS may use certificates and public key techniques for authentication and key establishment, then use symmetric authenticated encryption for application traffic.

2.13 Limitations of Cryptography

Cryptography is powerful, but it does not solve every security problem.

Cryptography cannot by itself:

- Correct insecure software logic.
- Prevent all phishing and social-engineering attacks.
- Guarantee availability.
- Protect plaintext after an authorized endpoint is compromised.
- Decide which users should receive permissions.
- Ensure that users follow security policies.
- Recover data after keys are destroyed without a backup.
- Make weak passwords unpredictable.

A secure system requires cryptography together with access control, secure coding, monitoring, patching, backups, user education, incident response, and risk management.

2.14 Common Mistakes in Using Cryptography

- Designing a custom cipher instead of using trusted standards.
- Using outdated algorithms or insecure parameter sizes.
- Hard-coding secret keys in source code.
- Reusing nonces or initialization values incorrectly.
- Using encryption without integrity protection.
- Using fast general-purpose hashes directly for password storage.
- Failing to authenticate public keys.
- Ignoring certificate warnings or hostname validation.
- Using predictable random numbers.
- Reusing one key for unrelated purposes.
- Logging plaintext secrets, passwords, tokens, or keys.
- Failing to plan for key compromise, rotation, and revocation.
- Assuming that a secure algorithm automatically creates a secure protocol.

2.15 Worked Security Scenario

Protecting an Online Grade Submission

Suppose an instructor submits student grades through a university portal.
The system may require:

- TLS encryption to protect confidentiality in transit.
- A trusted server certificate to authenticate the portal.
- MFA to authenticate the instructor.
- Authorization rules to permit grade changes only for assigned courses.
- Integrity protection to detect modified network messages.
- Nonces or session controls to prevent replay.
- Audit logs to record who changed each grade and when.
- Backups and recovery procedures to support availability.

This scenario demonstrates that cryptography is one part of a complete security design.

2.16 Practical Activity

Group Activity: Analyze a Secure Communication System

Group Size: 3 to 5 students
Total Marks: 15 marks
Choose one system, such as online banking, a university portal, cloud storage, secure messaging, or an e-commerce website.
Complete the following tasks:

1. Identify the sensitive assets.
2. Identify two passive attacks.
3. Identify four active attacks.
4. Identify the required security services.
5. Identify cryptographic and noncryptographic security mechanisms.
6. Describe how keys may be generated, distributed, stored, rotated, and revoked.
7. Explain one limitation that cryptography cannot solve by itself.
8. Draw a simple network-security model for the selected system.

Required Submission

Each group should submit:

- A system description.
- An asset and threat list.
- A passive-versus-active attack classification.
- A security-service and mechanism mapping.
- A key-management plan.
- A one-page network-security diagram.

Marking Scheme

Assessment Item	Marks
Correct identification of assets and threats	3
Correct classification of passive and active attacks	3
Correct selection of security services	2
Appropriate security mechanisms	2
Key-management explanation	2
Network-security model and diagram	2
Teamwork and presentation	1

2.17 Chapter Summary

This chapter introduced the fundamental language and models of cryptography. Cryptography uses algorithms, keys, and protocols to protect information. Cryptanalysis studies how cryptographic protection may be defeated, while cryptology includes both cryptography and cryptanalysis.

The chapter defined plaintext, ciphertext, encryption, decryption, algorithms, keys, and key spaces. It explained Kerckhoffs's principle and emphasized that security should not depend on hiding an algorithm.

The main cryptographic goals include confidentiality, integrity, authentication, and non-

repudiation support. The chapter also introduced the OSI security architecture, which separates security attacks, services, and mechanisms. Passive attacks include release of message contents and traffic analysis. Active attacks include masquerade, replay, message modification, and denial of service.

Security services include authentication, access control, confidentiality, integrity, non-repudiation, and availability. Security mechanisms include encryption, digital signatures, authentication exchange, integrity mechanisms, audit trails, event detection, and recovery.

Finally, the chapter compared symmetric cryptography, asymmetric cryptography, and hash functions. It introduced cryptanalytic attack models, brute-force search, the key lifecycle, common cryptographic applications, limitations, and implementation mistakes.

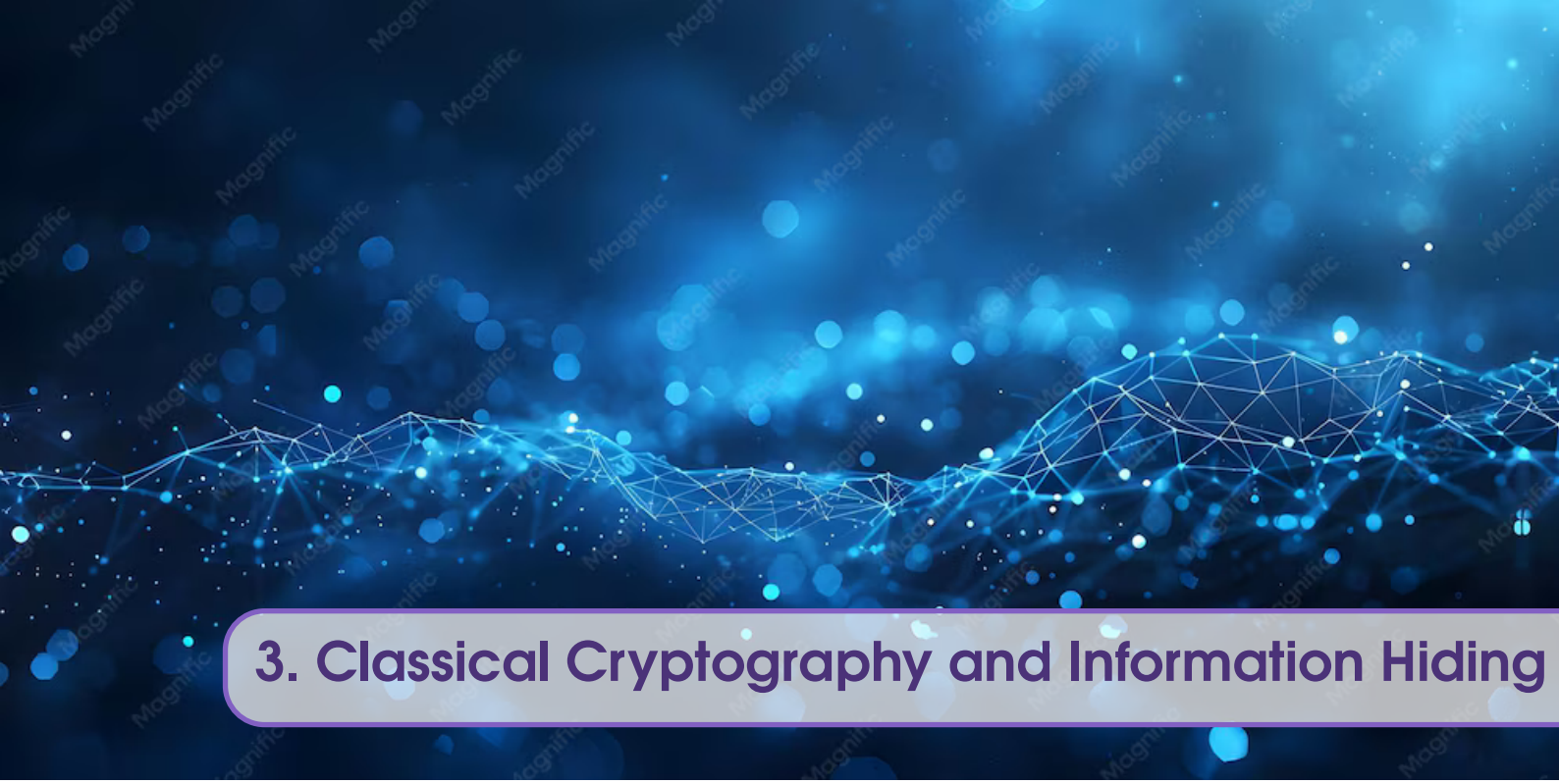
2.18 Additional Exercises

1. Explain the difference between cryptography, cryptanalysis, and cryptology.
2. Distinguish between plaintext and ciphertext.
3. Explain Kerckhoffs's principle in your own words.
4. Why is a large key space useful?
5. Compare confidentiality and integrity.
6. Compare peer-entity authentication and data-origin authentication.
7. Classify eavesdropping as passive or active and explain why.
8. Classify replay as passive or active and explain why.
9. Give an example of traffic analysis where message contents remain encrypted.
10. Explain the difference between a security service and a security mechanism.
11. Map encryption, a MAC, and a digital signature to the services each can support.
12. Create a network-security model for a secure email system.
13. Compare ciphertext-only, known-plaintext, chosen-plaintext, and chosen-ciphertext attacks.
14. Explain why encryption alone may not prevent message modification.
15. Design a simple key lifecycle for a university database-encryption key.

2.19 Review Questions

1. What is cryptography?
2. What is cryptanalysis?
3. What is cryptology?
4. What is plaintext?
5. What is ciphertext?
6. What is encryption?
7. What is decryption?
8. What is a cryptographic key?
9. What is a key space?
10. What is Kerckhoffs's principle?
11. What are the main goals of cryptography?
12. How does encryption support confidentiality?
13. Which mechanisms can support integrity?
14. What is peer-entity authentication?
15. What is data-origin authentication?
16. Why should non-repudiation be described carefully?
17. What is symmetric key cryptography?
18. What is asymmetric key cryptography?
19. What is a cryptographic hash function?

20. What is a ciphertext-only attack?
21. What is a known-plaintext attack?
22. What is a chosen-plaintext attack?
23. What is a chosen-ciphertext attack?
24. What is a brute-force attack?
25. What is key management?
26. Why is secure randomness important?
27. Why should keys be rotated or revoked?
28. Give five applications of cryptography.
29. Give five common mistakes in using cryptography.
30. Why can cryptography not solve every security problem?



3. Classical Cryptography and Information Hiding

Learning Objectives

By the end of this chapter, the reader should be able to:

- Explain the historical and educational importance of classical cryptography.
- Distinguish substitution ciphers from transposition ciphers.
- Encrypt and decrypt messages using the Caesar cipher.
- Explain monoalphabetic substitution and its weaknesses.
- Construct and use a Playfair cipher key matrix.
- Apply the same-row, same-column, and rectangle rules of Playfair encryption.
- Explain how the Hill cipher uses vectors, matrices, and modular arithmetic.
- Encrypt and decrypt small blocks using a Hill cipher key matrix.
- Explain why polyalphabetic ciphers reduce simple frequency patterns.
- Encrypt and decrypt messages using the Vigenere cipher.
- Explain the Vernam cipher and the XOR operation.
- State the conditions required for a secure one-time pad.
- Apply Rail Fence and columnar transposition techniques.
- Explain how frequency analysis attacks classical ciphers.
- Distinguish cryptography from steganography.
- Explain the basic idea of least-significant-bit information hiding.
- Describe why classical ciphers are unsuitable for modern security applications.

3.1 Introduction to Classical Cryptography

Chapter 2 introduced the basic language and principles of cryptography. This chapter applies those concepts to classical encryption techniques.

Classical cryptography refers to encryption methods developed before modern computer-based cryptography. These methods were often performed using paper, mechanical devices, or simple

mathematical operations. Classical ciphers usually protect alphabetic messages using substitution, transposition, or a combination of both.

A substitution cipher replaces plaintext symbols with different symbols. A transposition cipher keeps the original symbols but rearranges their positions. Polyalphabetic ciphers use more than one substitution rule to reduce visible language patterns.

Classical ciphers are important for education because they demonstrate fundamental ideas such as keys, encryption, decryption, modular arithmetic, matrices, key spaces, repeated patterns, and cryptanalysis. However, they are not secure for modern applications because computers can test keys and analyze language patterns very quickly.

This chapter studies the Caesar cipher, monoalphabetic substitution, Playfair cipher, Hill cipher, Vigenere cipher, Vernam cipher, one-time pad, Rail Fence cipher, columnar transposition, and frequency analysis. It also introduces steganography as a method for hiding the existence of information.

3.2 Substitution and Transposition

Classical ciphers can be classified according to how they transform plaintext.

Substitution Ciphers

A substitution cipher replaces each plaintext symbol, letter, or group of letters with another symbol or group.

Examples include:

- Caesar cipher.
- Monoalphabetic substitution cipher.
- Playfair cipher.
- Hill cipher.
- Vigenere cipher.

Transposition Ciphers

A transposition cipher does not replace plaintext symbols. Instead, it changes their positions according to a rule or key.

Examples include:

- Rail Fence cipher.
- Columnar transposition cipher.

Comparison

Feature	Substitution	Transposition
Main operation	Replaces symbols	Rearranges symbols
Letter values	Usually change	Remain unchanged
Letter positions	Usually remain in order	Change according to a rule
Frequency patterns	May remain visible under fixed substitution	Exact letter frequencies remain unchanged
Examples	Caesar and Vigenere	Rail Fence and columnar transposition

Some historical systems combine substitution and transposition to make cryptanalysis more difficult. Modern block ciphers also use repeated substitution and permutation ideas, but in a much more sophisticated way.

3.3 Caesar Cipher

The Caesar cipher is one of the simplest substitution ciphers. Each plaintext letter is shifted by a fixed number of positions in the alphabet.

Caesar Cipher

The Caesar cipher is a substitution cipher in which every plaintext letter is shifted by the same number of alphabet positions.

For a shift of 3:

$$A \to D, \quad B \to E, \quad C \to F$$

and the alphabet wraps around after Z:

$$X \to A, \quad Y \to B, \quad Z \to C$$

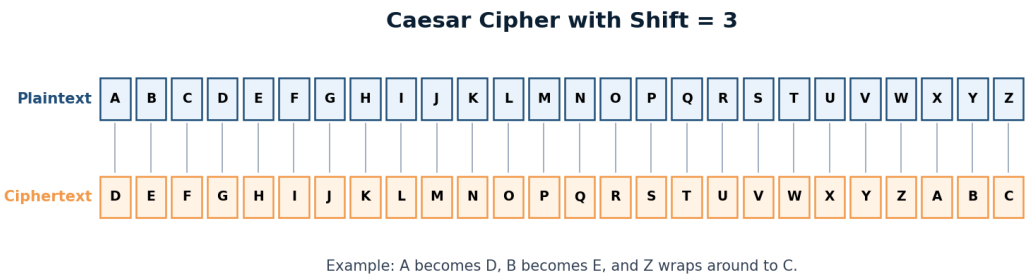


Figure 3.1: A Caesar cipher alphabet with a shift of three positions.

Encryption Formula

Represent letters using:

$$A = 0,\ B = 1,\ C = 2,\ \ldots,\ Z = 25$$

The encryption formula is:

$$C = (P + k) \bmod 26$$

where:
- P is the plaintext letter value.
- k is the shift key.
- C is the ciphertext letter value.

Decryption Formula

The decryption formula is:

$$P = (C - k) \bmod 26$$

Complete Worked Example

Caesar Encryption Example

Encrypt the message:

$$\texttt{SECURITY}$$

using key $k = 3$.
The letter-by-letter transformation is:

$$S \to V, \quad E \to H, \quad C \to F, \quad U \to X$$

$$R \to U, \quad I \to L, \quad T \to W, \quad Y \to B$$

Therefore:

$$\texttt{SECURITY} \to \texttt{VHFXULWB}$$

The letter Y wraps around the alphabet:

$$Y \to Z \to A \to B$$

Complete Decryption Example

Caesar Decryption Example

Decrypt:

$$\texttt{FRPSXWHU}$$

using key $k = 3$.
Shift each ciphertext letter three positions backward:

$$F \to C, \quad R \to O, \quad P \to M, \quad S \to P$$

$$X \to U, \quad W \to T, \quad H \to E, \quad U \to R$$

Therefore:

$$\texttt{FRPSXWHU} \to \texttt{COMPUTER}$$

Brute-Force Attack

As explained in Chapter 2, exhaustive search tests possible keys. The Caesar cipher has only 25 useful nonzero shifts. An attacker can decrypt the ciphertext with every shift and look for readable language.

R **Important Point.** Caesar cipher is useful for learning modular arithmetic and key search, but its tiny key space makes it unsuitable for real security.

3.4 Monoalphabetic Substitution Cipher

A monoalphabetic substitution cipher uses one fixed substitution alphabet for the entire message. Each plaintext letter is always replaced by the same ciphertext letter.

Monoalphabetic Substitution Cipher

A monoalphabetic substitution cipher uses one fixed substitution alphabet for all occurrences of the plaintext letters.

Suppose the normal alphabet maps to:

$$\texttt{ABCDEFGHIJKLMNOPQRSTUVWXYZ}$$

$$\texttt{QWERTYUIOPASDFGHJKLZXCVBNM}$$

This means:

$$A \to Q, \quad B \to W, \quad C \to E, \quad D \to R$$

Encryption Example

Monoalphabetic Encryption

Encrypt:

SECURITY

Using the substitution alphabet above:

$$S \to L, \quad E \to T, \quad C \to E, \quad U \to X$$

$$R \to K, \quad I \to O, \quad T \to Z, \quad Y \to N$$

Therefore:

SECURITY $\to$ LTEXKOZN

Decryption

To decrypt, the receiver reverses the mapping. If `Q` appears in ciphertext, it maps back to `A`; if `W` appears, it maps back to `B`; and so on.

Pattern Preservation

Although the number of possible substitution alphabets is very large, fixed substitution preserves language patterns. Repeated plaintext letters become repeated ciphertext letters.

For example:

SEES $\to$ LTTL

The repeated structure remains visible. This makes monoalphabetic substitution vulnerable to frequency analysis and pattern matching.

3.5 Playfair Cipher

The Playfair cipher encrypts pairs of letters rather than one letter at a time. Each pair is called a digraph. The cipher uses a 5×5 matrix constructed from a keyword.

Playfair Cipher

The Playfair cipher is a digraph substitution cipher that encrypts pairs of plaintext letters using a 5×5 key matrix.

Creating the Key Matrix

Use the keyword:

```
MONARCHY
```

Remove repeated letters, then fill the remaining matrix positions with unused alphabet letters. In this example, `I` and `J` share one cell.

$$\begin{array}{ccccc} M & O & N & A & R \\ C & H & Y & B & D \\ E & F & G & I/J & K \\ L & P & Q & S & T \\ U & V & W & X & Z \end{array}$$

Preparing the Plaintext

The plaintext is divided into pairs.

Preparation rules include:

- Convert the message to uppercase.
- Remove spaces and punctuation.
- Replace `J` with `I` when the matrix combines them.
- If a pair contains repeated letters, insert a padding letter such as `X`.
- If one letter remains at the end, add a padding letter.

For example:

```
BALLOON
```

may be prepared as:

```
BA LX LO ON
```

Same-Row Rule

If both letters are in the same row, replace each letter with the letter immediately to its right. Wrap around to the beginning of the row when necessary.

For example, in the fourth row:

$$L\ P\ Q\ S\ T$$

The pair `LP` becomes `PQ`.

Same-Column Rule

If both letters are in the same column, replace each letter with the letter immediately below it. Wrap to the top when necessary.

For example, in the first column:

$$M,\ C,\ E,\ L,\ U$$

The pair MC becomes CE.

Rectangle Rule

If the letters form opposite corners of a rectangle, keep each letter's row but replace its column with the other letter's column.

For example, H is at row 2, column 2, and I is at row 3, column 4. The other rectangle corners are B and F, so:

$$\texttt{HI} \rightarrow \texttt{BF}$$

Complete Encryption Example

Playfair Encryption Example

Use the matrix generated from MONARCHY to encrypt:

INSTRUMENTS

Prepare the pairs:

IN ST RU ME NT SX

Apply the Playfair rules:

$$\texttt{IN} \rightarrow \texttt{GA}$$

$$\texttt{ST} \rightarrow \texttt{TL}$$

$$\texttt{RU} \rightarrow \texttt{MZ}$$

$$\texttt{ME} \rightarrow \texttt{CL}$$

$$\texttt{NT} \rightarrow \texttt{RQ}$$

$$\texttt{SX} \rightarrow \texttt{XA}$$

Therefore, the ciphertext is:

GATLMZCLRQXA

Decryption

Decryption reverses the direction:

- Same row: move left.

- Same column: move upward.
- Rectangle: use the opposite corners in the same way as encryption.

Padding letters introduced during preparation may need to be removed carefully after decryption.

Limitations

Playfair hides single-letter frequencies better than monoalphabetic substitution because it encrypts pairs. However, digraph frequencies and language patterns remain. Modern computers can analyze these patterns efficiently.

3.6 Hill Cipher

The Hill cipher encrypts blocks of letters using matrix multiplication modulo 26. Letters are represented as numbers, grouped into vectors, and multiplied by a key matrix.

Hill Cipher

The Hill cipher is a polygraphic substitution cipher that uses linear algebra and modular arithmetic to encrypt blocks of letters.

Letters as Numbers

Use:

$$A = 0,\ B = 1,\ C = 2,\ \ldots,\ Z = 25$$

For a two-letter block, create a column vector.
For example:

$$\texttt{HI} \rightarrow \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Key Matrix

Choose an invertible key matrix modulo 26. For example:

$$K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$$

The encryption equation is:

$$C = KP \bmod 26$$

Encryption Example

Hill Cipher Encryption

Encrypt the block:

HI

The plaintext vector is:

$$P = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Multiply by the key matrix:

$$KP = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 7 \\ 8 \end{bmatrix} = \begin{bmatrix} 45 \\ 54 \end{bmatrix}$$

Reduce modulo 26:

$$\begin{bmatrix} 45 \\ 54 \end{bmatrix} \bmod 26 = \begin{bmatrix} 19 \\ 2 \end{bmatrix}$$

The values 19 and 2 represent T and C. Therefore:

$$\texttt{HI} \rightarrow \texttt{TC}$$

Inverse Key Matrix

Decryption requires the inverse matrix modulo 26:

$$P = K^{-1}C \bmod 26$$

A valid Hill key must have an inverse modulo 26. If the determinant is not relatively prime to 26, the key matrix cannot be used for unique decryption.

For the example key:

$$\det(K) = 3(5) - 3(2) = 9$$

Since:

$$\gcd(9, 26) = 1$$

an inverse exists.

The modular inverse of 9 modulo 26 is 3 because:

$$9 \times 3 = 27 \equiv 1 \pmod{26}$$

The inverse key is:

$$K^{-1} = \begin{bmatrix} 15 & 17 \\ 20 & 9 \end{bmatrix} \bmod 26$$

Decryption Example

Hill Cipher Decryption

Decrypt:

```
TC
```

The ciphertext vector is:

$$C = \begin{bmatrix} 19 \\ 2 \end{bmatrix}$$

Apply the inverse key:

$$P = \begin{bmatrix} 15 & 17 \\ 20 & 9 \end{bmatrix} \begin{bmatrix} 19 \\ 2 \end{bmatrix} \bmod 26$$

$$P = \begin{bmatrix} 319 \\ 398 \end{bmatrix} \bmod 26 = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

The result is:

```
HI
```

Security Limitations

The Hill cipher mixes letters inside a block and hides simple single-letter relationships. However, it is vulnerable to known-plaintext attacks because enough plaintext-ciphertext pairs can reveal the key matrix.

3.7 Polyalphabetic Ciphers

A polyalphabetic cipher uses multiple substitution alphabets. The same plaintext letter may be encrypted differently depending on its position and corresponding key symbol.

Polyalphabetic Cipher

A polyalphabetic cipher uses more than one substitution alphabet to reduce visible frequency patterns.

For example, repeated plaintext letters:

```
EEEE
```

might become:

```
HOJQ
```

if different shifts are used at the four positions.

3.8 Vigenere Cipher

The Vigenere cipher is a well-known polyalphabetic substitution cipher. It uses a repeating keyword to determine different Caesar shifts.

Vigenere Cipher

The Vigenere cipher encrypts each plaintext letter using a shift determined by the corresponding letter of a repeating keyword.

Encryption Formula

Represent letters as values from 0 to 25. The formula is:

$$C_i = (P_i + K_i) \bmod 26$$

Complete Encryption Example

Vigenere Encryption Example

Encrypt:

ATTACKATDAWN

using keyword:

LEMON

Repeat the keyword:

Plaintext:	*A*	*T*	*T*	*A*	*C*	*K*	*A*	*T*	*D*	*A*	*W*	*N*
Keyword:	*L*	*E*	*M*	*O*	*N*	*L*	*E*	*M*	*O*	*N*	*L*	*E*
Ciphertext:	*L*	*X*	*F*	*O*	*P*	*V*	*E*	*F*	*R*	*N*	*H*	*R*

Therefore:

ATTACKATDAWN $\rightarrow$ LXFOPVEFRNHR

Decryption Formula

$$P_i = (C_i - K_i) \bmod 26$$

The receiver repeats the same keyword and subtracts the corresponding key values.

Weaknesses of Repeated Keywords

If a short keyword is repeated over a long message, periodic patterns may appear. Cryptanalysts may estimate the keyword length and divide the ciphertext into groups that behave like separate Caesar ciphers.

R **Important Point.** Vigenere is stronger than a single fixed substitution, but repeated-key structure prevents it from providing modern security.

3.9 Vernam Cipher

The Vernam cipher combines plaintext units with key units. In binary systems, the combining operation is XOR.

Vernam Cipher

A Vernam cipher combines plaintext with a key stream, commonly using the XOR operation.

Encryption is:

$$C = P \oplus K$$

Decryption is:

$$P = C \oplus K$$

This works because:

$$(P \oplus K) \oplus K = P$$

The XOR Operation

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Worked Binary Example

Vernam XOR Example

Let:

$$P = 10110010$$

and:

$$K = 01101100$$

Encryption gives:

$$\begin{array}{r} 10110010 \\ \oplus\ 01101100 \\ \hline 11011110 \end{array}$$

Therefore:

$$C = 11011110$$

Decrypt using the same key:

$$\begin{array}{r} 11011110 \\ \oplus\ 01101100 \\ \hline 10110010 \end{array}$$

The original plaintext is recovered.

Key-Reuse Problem

If the same key stream is reused for two messages:

$$C_1 = P_1 \oplus K$$

$$C_2 = P_2 \oplus K$$

then:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

The key cancels. This reveals a relationship between the plaintexts and can cause serious compromise.

3.10 One-Time Pad

A one-time pad is a special form of XOR-based encryption that can provide perfect secrecy when all requirements are satisfied.

One-Time Pad

A one-time pad uses a truly random secret key that is at least as long as the message and is never reused.

The key must be:

- Truly random.
- At least as long as the plaintext.
- Kept completely secret.
- Used for only one message.
- Distributed securely to the intended parties.

Why Key Reuse Is Dangerous

Reusing a one-time-pad key destroys its perfect-secrecy guarantee. The XOR relationship between ciphertexts can reveal information about both plaintexts.

Practical Limitations

The main challenge is key management. A one-megabyte message requires a one-megabyte random secret key. The sender and receiver must securely exchange, store, synchronize, and destroy the key after use.

R **Important Point.** Not every Vernam-style cipher is a one-time pad. Perfect secrecy requires a truly random, message-length, secret, single-use key.

3.11 Transposition Ciphers

Transposition ciphers rearrange plaintext positions without replacing the symbols. This means the ciphertext contains exactly the same letters as the plaintext, but in a different order.

Transposition Cipher

A transposition cipher keeps the original plaintext symbols but changes their positions according to a rule or key.

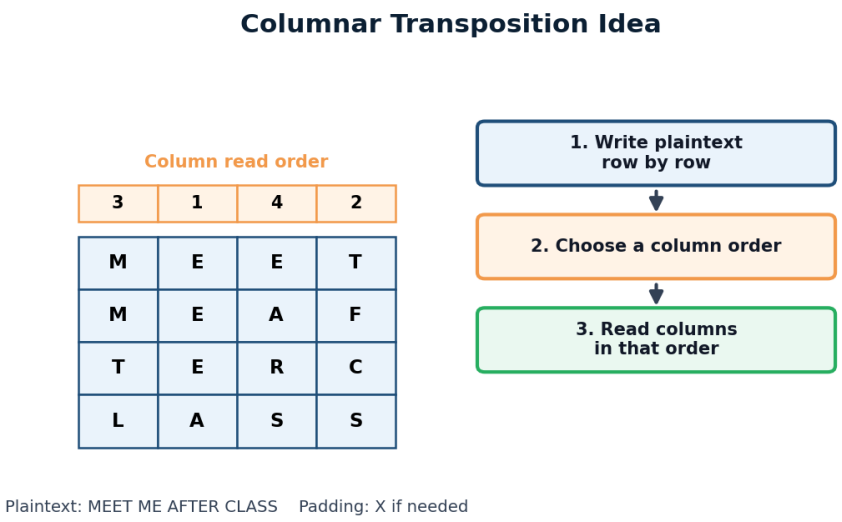


Figure 3.2: A simple grid used for columnar transposition.

3.12 Rail Fence Cipher

The Rail Fence cipher writes plaintext in a zigzag pattern across a selected number of rows, called rails. The ciphertext is produced by reading each rail from left to right.

Rail Fence Cipher

The Rail Fence cipher is a transposition cipher that writes plaintext in a zigzag pattern across multiple rails.

Encryption Example

Rail Fence Encryption

Encrypt:

```
WEAREDISCOVERED
```

using three rails.
Write the zigzag:

```
W       E       C       R
  E   R   D   S   O   E   E
    A       I       V       D
```

Read the rows from top to bottom:

```
WECRERDSOEEAIVD
```

Decryption Idea

To decrypt, determine the zigzag positions, fill those positions with ciphertext letters rail by rail, and then read the zigzag path in its original order.

Limitations

The number of plausible rails is usually small, and the ciphertext preserves all original letters. Brute-force testing and pattern analysis can break simple Rail Fence messages.

3.13 Columnar Transposition

In a columnar transposition cipher, plaintext is written into a grid row by row. Columns are then read according to a fixed or keyed order.

Simple Columnar Transposition

Simple Columnar Transposition

Encrypt:

`COMPUTER`

using four columns.
Write the plaintext row by row:

$$\begin{matrix} C & O & M & P \\ U & T & E & R \end{matrix}$$

Read each column from top to bottom:

$$CU \quad OT \quad ME \quad PR$$

Therefore:

$$\texttt{COMPUTER} \rightarrow \texttt{CUOTMEPR}$$

Keyed Column Order

Suppose the column order is:

$$3,\ 1,\ 4,\ 2$$

Using the same grid, read column 3, then 1, then 4, then 2:

$$ME \quad CU \quad PR \quad OT$$

Therefore:

$$\texttt{COMPUTER} \rightarrow \texttt{MECUPROT}$$

Padding

If plaintext does not fill the grid, a padding character such as `X` may be added.
For example:

$$\texttt{ATTACKNOW} \rightarrow \texttt{ATTACKNOWXXX}$$

with four columns:

$$\begin{matrix} A & T & T & A \\ C & K & N & O \\ W & X & X & X \end{matrix}$$

Reading columns from left to right gives:

```
ACWTKXTNXAOX
```

Decryption

The receiver must know the number of columns, number of rows, padding convention, and column order. The ciphertext letters are placed back into columns and then read row by row.

3.14 Frequency Analysis

Frequency analysis studies how often letters or groups of letters appear in ciphertext. It is especially effective against fixed substitution ciphers.

Frequency Analysis

Frequency analysis is the study of statistical symbol patterns in ciphertext to infer plaintext mappings or key information.

In English, common letters include:

$$E,\ T,\ A,\ O,\ I,\ N$$

Less common letters include:

$$Q,\ X,\ Z$$

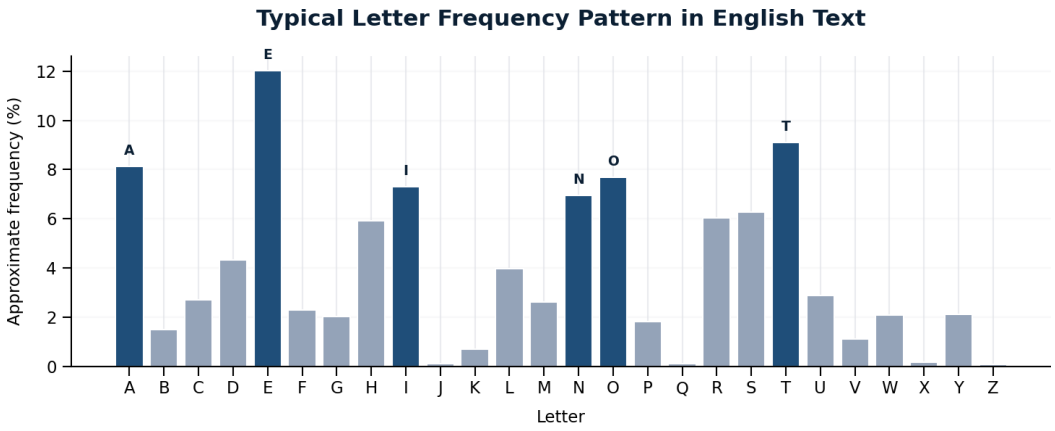


Figure 3.3: Typical English letter-frequency patterns used in classical cryptanalysis.

Repeated Patterns

A cryptanalyst studies:

- Frequent single letters.
- Repeated double letters.
- Common digraphs such as `TH`, `HE`, and `IN`.
- Common trigraphs such as `THE` and `AND`.
- One-letter words such as `A` and `I`.
- Repeated word shapes.
- Word lengths and punctuation positions.

Attacking Caesar Cipher

Frequency analysis can help identify the most likely shift, but exhaustive search is even simpler because the key space contains only 25 useful shifts.

Attacking Monoalphabetic Substitution

A large substitution key space prevents simple exhaustive search, but fixed letter mappings preserve statistical language structure. Frequency analysis and word-pattern analysis can gradually recover the substitution alphabet.

Limitations of Frequency Analysis

Frequency analysis is less reliable when:

- The ciphertext is very short.
- The plaintext language is unknown.
- The text contains unusual vocabulary.
- Multiple substitution alphabets are used.
- Plaintext has been compressed or randomized.

Mini Frequency Activity

Encrypt a paragraph using a Caesar cipher with shift 3. Count the frequency of every ciphertext letter. Compare the most frequent ciphertext letters with common English letters and explain which patterns remain visible.

3.15 Steganography

Steganography hides the existence of information by embedding a secret message inside an apparently ordinary object, such as an image, audio file, video, or document.

Steganography

Steganography is the practice of hiding secret information inside another object or communication so that the existence of the hidden message is less obvious.

Basic Terms

- **Cover object:** the original object used to carry hidden data.
- **Secret message:** the information being hidden.
- **Stego object:** the resulting object after embedding.

- **Embedding algorithm:** the method used to hide the data.
- **Extraction algorithm:** the method used to recover the data.
- **Stego key:** an optional secret value controlling embedding or extraction.

Cryptography vs. Steganography

Feature	Cryptography	Steganography
Main goal	Hide message meaning	Hide message existence
Visible protected data	Usually yes	Ideally not obvious
Typical output	Ciphertext	Stego object
If detected	Content may remain protected	Hidden channel may be exposed
Best practice	Use trusted encryption	Encrypt before embedding when appropriate

Least-Significant-Bit Hiding

A digital image stores pixel color values using binary numbers. An LSB technique changes the least significant bit of selected values to store message bits.

For example, a byte:

```
10110110
```

may be changed to:

```
10110111
```

to store a hidden bit of 1. The numerical change is small, so the visual difference may be difficult to notice.

Capacity and Imperceptibility

Two important design concerns are:

- **Capacity:** how much information can be hidden.
- **Imperceptibility:** how difficult it is to notice the modifications.

Increasing capacity may make changes easier to detect.

Steganalysis

Steganalysis attempts to detect hidden information. It may compare statistical patterns, file structures, pixel distributions, compression artifacts, or suspicious metadata.

Security and Ethical Considerations

Steganography has legitimate uses in watermarking, integrity marking, research, and covert communication. It can also be misused. Classroom activities should use synthetic data, authorized files, and clearly defined ethical boundaries.

R **Important Point.** Steganography does not replace encryption. If hidden data is discovered and was not encrypted, its contents may be immediately readable.

3.16 Comparison of Classical Techniques

Technique	Main Operation	Key Type	Main Weakness
Caesar	Single alphabet shift	Numeric shift	Very small key space
Monoalphabetic	Fixed substitution	Substitution alphabet	Frequency analysis
Playfair	Digraph substitution	Key matrix	Digraph patterns remain
Hill	Matrix transformation	Invertible matrix	Known-plaintext analysis
Vigenere	Repeated shifts	Keyword	Repeated-key patterns
Vernam	Bitwise XOR	Bit sequence	Key reuse
One-time pad	XOR with random key	Single-use random key	Difficult key distribution
Rail Fence	Zigzag rearrangement	Number of rails	Small key space
Columnar	Column rearrangement	Column order	Letter frequencies remain

3.17 Limitations of Classical Cryptography

Classical ciphers are useful for learning, but they are not secure for modern applications.

Major limitations include:

- Small key spaces in many systems.
- Visible language patterns.
- Vulnerability to frequency and pattern analysis.
- Limited resistance to known-plaintext attacks.
- Manual key distribution difficulties.
- Lack of strong integrity and authentication mechanisms.
- Poor suitability for large files, databases, multimedia, and network traffic.
- Weakness against modern computing power.

Pattern Limitation

A monoalphabetic substitution may replace the letters of `MEET`, but the repeated middle letters remain repeated. A transposition cipher preserves the exact number of every letter. These patterns provide information to an attacker.

Modern cryptography uses stronger mathematical designs, large keys, secure modes, authenticated encryption, cryptographic randomness, and carefully analyzed protocols.

3.18 Practical Group Activity

Classical Cipher Challenge

Group Size: 3 to 5 students
Total Marks: 20 marks
Each group must complete four stages.

Stage 1: Caesar Cipher

Choose a plaintext of at least 12 letters, select a shift key, encrypt it, and give only the ciphertext to another group. The receiving group must recover the plaintext by testing possible shifts.

Stage 2: Playfair or Vigenere

Choose either Playfair or Vigenere. Document the keyword, preparation rules, encryption steps, and ciphertext. Another group should verify the result.

Stage 3: Transposition

Encrypt a message using either Rail Fence or keyed columnar transposition. Clearly document the number of rails or column order.

Stage 4: Security Analysis

Compare the selected ciphers and explain:

- Which cipher was easiest to use.
- Which cipher was easiest to break.
- Which patterns remained visible.
- Why none should protect modern sensitive data.

Required Submission

- Group member names.
- Plaintexts, keys, and ciphertexts.
- Step-by-step calculations.
- Verification or decryption results.
- A one-page comparative security analysis.

Marking Scheme

Assessment Item	Marks
Correct Caesar encryption and cryptanalysis	4
Correct Playfair or Vigenere process	5
Correct transposition process	4
Clear step-by-step calculations	3
Security comparison and limitations	3
Teamwork and presentation	1

3.19 Chapter Summary

This chapter introduced classical cryptographic techniques and information hiding. Classical ciphers are historically important because they demonstrate substitution, transposition, key use, modular arithmetic, matrix operations, repeated patterns, and cryptanalysis.

The Caesar cipher uses one fixed alphabet shift and is easily broken by testing all possible keys. Monoalphabetic substitution uses a larger key space but preserves language-frequency patterns. The Playfair cipher encrypts letter pairs, while the Hill cipher encrypts letter blocks using matrix multiplication modulo 26.

The Vigenere cipher uses a repeating keyword to produce multiple Caesar shifts. It hides simple letter frequencies better than monoalphabetic substitution, but repeated keywords allow attacks. The Vernam cipher uses XOR, while the one-time pad provides perfect secrecy only when its key is truly random, as long as the message, secret, and never reused.

Rail Fence and columnar transposition ciphers rearrange plaintext positions without replacing letters. As a result, letter-frequency information remains present.

Frequency analysis studies statistical language patterns and is especially effective against fixed substitution ciphers. Steganography hides the existence of a message inside another object, but it does not replace encryption.

Classical ciphers are valuable educational tools, but they are not appropriate for protecting modern sensitive information.

3.20 Additional Exercises

1. Encrypt `NETWORK` using a Caesar shift of 5.
2. Decrypt `KHOOR` using Caesar key 3.
3. Construct a monoalphabetic substitution alphabet from a keyword of your choice.
4. Build a Playfair matrix using the keyword `SECURITY`.
5. Prepare the plaintext `BALLOON` as Playfair digraphs.
6. Encrypt three Playfair digraphs using same-row, same-column, and rectangle rules.
7. Determine whether the matrix $\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$ is invertible modulo 26.
8. Encrypt `HI` using the Hill key in this chapter.
9. Encrypt `DEFEND` using Vigenere keyword `KEY`.
10. Explain why a repeated Vigenere keyword creates periodic patterns.
11. Compute $10101100 \oplus 01100110$.
12. Explain why reusing a Vernam key is dangerous.
13. List all requirements for a secure one-time pad.
14. Encrypt `CRYPTOGRAPHY` using a three-rail Rail Fence cipher.
15. Encrypt `INFORMATION` using four-column transposition and padding if needed.
16. Explain why transposition does not hide letter frequency.
17. Identify three statistics useful in frequency analysis.
18. Compare cryptography and steganography.
19. Explain how an LSB method may hide one bit in a byte.
20. Explain why a hidden message may still need encryption.

3.21 Review Questions

1. What is classical cryptography?
2. What is the difference between substitution and transposition?
3. How does the Caesar cipher encrypt a letter?
4. What is the Caesar encryption formula?

5. Why is Caesar cipher easy to break?
6. What is a monoalphabetic substitution cipher?
7. Why does monoalphabetic substitution preserve frequency patterns?
8. What is a digraph?
9. How is a Playfair key matrix constructed?
10. What happens when a Playfair pair contains repeated letters?
11. Explain the Playfair same-row rule.
12. Explain the Playfair same-column rule.
13. Explain the Playfair rectangle rule.
14. What mathematical operations does the Hill cipher use?
15. Why must a Hill key matrix be invertible modulo 26?
16. What is a polyalphabetic cipher?
17. How does Vigenere select shifts?
18. Why is Vigenere stronger than fixed substitution?
19. Why can a repeated Vigenere keyword be attacked?
20. What is the Vernam cipher?
21. What is XOR?
22. Why does applying the same XOR key twice recover the plaintext?
23. What happens when a Vernam key stream is reused?
24. What is a one-time pad?
25. What requirements provide perfect secrecy in a one-time pad?
26. What is the main practical problem with one-time pads?
27. What is the Rail Fence cipher?
28. How does columnar transposition encrypt a message?
29. Why may transposition require padding?
30. What is frequency analysis?
31. Which English letters are commonly frequent?
32. What patterns besides single-letter frequency can cryptanalysts study?
33. When is frequency analysis less reliable?
34. What is steganography?
35. What is a cover object?
36. What is a stego object?
37. What is least-significant-bit hiding?
38. What are capacity and imperceptibility?
39. What is steganalysis?
40. Why are classical ciphers unsuitable for modern security?

Modern Cryptography and Security Applications

4. Mathematical Foundations of Cryptography

Learning Objectives

By the end of this chapter, the reader should be able to:

- Explain why mathematics is important in cryptography.
- Perform addition, subtraction, multiplication, and exponentiation modulo an integer.
- Interpret modular congruence notation.
- Identify prime and composite numbers and explain their cryptographic importance.
- Compute a greatest common divisor using the Euclidean algorithm.
- Determine whether two integers are relatively prime.
- Use the extended Euclidean algorithm to find a modular inverse.
- Compute Euler's phi function for simple values.
- Apply Fermat's little theorem and Euler's theorem to small examples.
- Compute modular powers efficiently using repeated squaring.
- Explain primitive roots and the discrete logarithm problem conceptually.
- Solve a small system of congruences using the Chinese remainder theorem.
- Explain the basic role of groups and finite fields in modern cryptography.
- Determine whether a small matrix is invertible modulo 26.

4.1 Introduction

Modern cryptography is built on mathematical operations that are efficient to perform correctly but difficult to reverse without the required secret information. The purpose of this chapter is not to provide an advanced mathematics course. Instead, it develops the specific mathematical ideas needed in later chapters.

The chapter uses small values so that calculations can be completed by hand. Real cryptographic systems use much larger values, standardized parameters, secure random-number generation, and trusted software libraries.

The main connections are:

- Modular arithmetic is used throughout classical and modern cryptography.
- Matrices and modular inverses support the Hill cipher.
- Finite-field operations are used in AES and elliptic curve cryptography.
- Prime numbers, Euler's phi function, and modular inverses support RSA.
- Modular exponentiation, primitive roots, and discrete logarithms support Diffie–Hellman and ElGamal.
- The Chinese remainder theorem can improve RSA computation.

Educational Scope. The examples in this chapter explain the mathematics. The examples are not secure cryptographic parameters and must not be used to protect real information.

4.2 Integers, Divisibility, and Factors

Cryptographic algorithms operate on numerical representations of data. Important foundational concepts include integers, divisibility, factors, remainders, powers, and inverses.

Divisibility

An integer a divides an integer b, written $a \mid b$, if there is an integer k such that $b = ak$.

For example:

$$4 \mid 20$$

because:

$$20 = 4 \times 5$$

However:

$$6 \nmid 20$$

because no integer multiplied by 6 equals 20.

A factor of a number divides that number exactly. A multiple of a number is obtained by multiplying it by an integer.

4.3 Modular Arithmetic

Modular arithmetic works with remainders after division. It keeps values within a fixed range and is one of the most important mathematical tools in cryptography.

Modulo Operation

The expression $a \bmod n$ is the remainder obtained when the integer a is divided by the positive integer n.

For example:

$$29 = 4 \times 6 + 5$$

Therefore:

$$29 \bmod 6 = 5$$

Congruence

Two integers are congruent modulo n when they have the same remainder after division by n.

Modular Congruence

The notation

$$a \equiv b \pmod{n}$$

means that n divides $a - b$. Equivalently, a and b have the same remainder modulo n.

For example:

$$38 \equiv 3 \pmod{7}$$

because:

$$38 - 3 = 35$$

and 7 divides 35.

Clock Interpretation

A clock provides a familiar modular-arithmetic example. If the time is 10 o'clock and 5 hours pass, then:

$$10 + 5 = 15 \equiv 3 \pmod{12}$$

The clock wraps around after 12.

Modular Addition

Addition may be reduced before or after the numbers are added:

$$(a + b) \bmod n$$

Modular Addition

Compute:

$$(19+17) \bmod 12$$

First add:

$$19+17=36$$

Then reduce:

$$36 \bmod 12=0$$

Therefore:

$$(19+17) \bmod 12=0$$

Modular Subtraction

A negative result can be converted to its nonnegative representative by adding the modulus.

Modular Subtraction

Compute:

$$(4-9) \bmod 7$$

Since:

$$4-9=-5$$

add 7:

$$-5+7=2$$

Therefore:

$$(4-9) \bmod 7=2$$

Modular Multiplication

Products can also be reduced modulo n:

$$(ab) \bmod n$$

Modular Multiplication

Compute:

$$(13 \times 9) \bmod 10$$

$$13 \times 9 = 117$$

$$117 \bmod 10 = 7$$

Therefore:

$$(13 \times 9) \bmod 10 = 7$$

Useful Modular Rules

The following rules allow intermediate values to be reduced:

$$(a+b) \bmod n = \big((a \bmod n) + (b \bmod n)\big) \bmod n$$

$$(a-b) \bmod n = \big((a \bmod n) - (b \bmod n)\big) \bmod n$$

$$(ab) \bmod n = \big((a \bmod n)(b \bmod n)\big) \bmod n$$

These rules prevent intermediate values from becoming unnecessarily large.

4.4 Prime and Composite Numbers

Prime Number

A prime number is an integer greater than 1 with exactly two positive divisors: 1 and itself.

Examples include:

$$2,\ 3,\ 5,\ 7,\ 11,\ 13,\ 17,\ 19$$

The number 2 is the only even prime.

Composite Number

A composite number is an integer greater than 1 with more than two positive divisors.

For example, 15 is composite because:

$$15 = 3 \times 5$$

The number 1 is neither prime nor composite.

Testing Small Numbers

To determine whether a small integer n is prime, it is sufficient to test possible prime divisors up to $\sqrt{n}$.

Small Primality Test

Determine whether 29 is prime.
Since:

$$\sqrt{29} \approx 5.39$$

it is sufficient to test the primes 2, 3, and 5. The number 29 is divisible by none of them. Therefore, 29 is prime.

Prime Factorization

Prime factorization expresses a composite number as a product of primes.

For example:

$$60 = 2^2 \times 3 \times 5$$

Fundamental Theorem of Arithmetic

Every integer greater than 1 can be written as a product of prime numbers, and this factorization is unique apart from the order of the factors.

Why Prime Numbers Matter

Prime numbers create useful modular structures and support hard mathematical problems. Later chapters use primes in RSA, Diffie–Hellman, ElGamal, and some digital-signature systems.

The important security distinction is:

- Multiplying large primes is efficient.

- Recovering the original prime factors from a sufficiently large product is computationally difficult using known classical methods.

This relationship supports RSA, but the complete RSA process is explained in Chapter 6 rather than repeated here.

4.5 Greatest Common Divisor

Greatest Common Divisor

The greatest common divisor of integers a and b, written $\gcd(a,b)$, is the largest positive integer that divides both numbers.

For small values, the GCD can be found by listing factors. For larger values, the Euclidean algorithm is more efficient.

Euclidean Algorithm

The Euclidean algorithm repeatedly replaces a pair of numbers with the smaller number and the remainder.

The key identity is:

$$\gcd(a,b) = \gcd(b, a \bmod b)$$

The process stops when the remainder becomes zero. The last nonzero remainder is the GCD.

Euclidean Algorithm Example

Find:

$$\gcd(252, 105)$$

Divide 252 by 105:

$$252 = 2 \times 105 + 42$$

Then:

$$105 = 2 \times 42 + 21$$

Then:

$$42 = 2 \times 21 + 0$$

The last nonzero remainder is 21. Therefore:

$$\gcd(252, 105) = 21$$

4.6 Relatively Prime Numbers

Relatively Prime Integers

Two integers are relatively prime, or coprime, when their greatest common divisor is 1.

For example:

$$\gcd(35,64) = 1$$

Therefore, 35 and 64 are relatively prime even though neither number must itself be prime. This concept is essential because a modular inverse of a modulo n exists exactly when:

$$\gcd(a,n) = 1$$

4.7 Extended Euclidean Algorithm

The extended Euclidean algorithm computes the GCD and also finds integers x and y satisfying Bezout's identity:

$$ax + by = \gcd(a,b)$$

When $\gcd(a,n) = 1$, the coefficient of a provides its inverse modulo n.

Worked Example

Finding the Inverse of 7 Modulo 26

First apply the Euclidean algorithm:

$$26 = 3 \times 7 + 5$$

$$7 = 1 \times 5 + 2$$

$$5 = 2 \times 2 + 1$$

$$2 = 2 \times 1 + 0$$

The GCD is 1, so the inverse exists. Now work backward:

$$1 = 5 - 2 \times 2$$

Since:

$$2 = 7 - 1 \times 5$$

substitute:

$$1 = 5 - 2(7 - 5) = 3 \times 5 - 2 \times 7$$

Since:

$$5 = 26 - 3 \times 7$$

substitute again:

$$1 = 3(26 - 3 \times 7) - 2 \times 7$$

$$1 = 3 \times 26 - 11 \times 7$$

Therefore:

$$-11 \times 7 \equiv 1 \pmod{26}$$

Since:

$$-11 \equiv 15 \pmod{26}$$

We obtain:

$$7^{-1} \equiv 15 \pmod{26}$$

Check:

$$7 \times 15 = 105 \equiv 1 \pmod{26}$$

4.8 Modular Inverses

Modular Inverse

The modular inverse of a modulo n is an integer a^{-1} satisfying:

$$a \times a^{-1} \equiv 1 \pmod{n}$$

A modular inverse exists if and only if:

$$\gcd(a,n) = 1$$

Inverse That Does Not Exist

Does 6 have an inverse modulo 15?

$$\gcd(6,15) = 3$$

Since the GCD is not 1, no modular inverse exists.

Modular inverses are required in the Hill cipher, RSA key generation, and several signature calculations.

4.9 Euler's Phi Function

Euler's Phi Function

Euler's phi function $\phi(n)$ counts the integers from 1 through n that are relatively prime to n.

For a prime p:

$$\phi(p) = p - 1$$

because every integer from 1 to $p-1$ is relatively prime to p.
If p and q are distinct primes:

$$\phi(pq) = (p-1)(q-1)$$

Phi of a Product of Two Primes

Let:

$$p = 5, \qquad q = 11$$

Then:

$$n = pq = 55$$

and:

$$\phi(55) = (5-1)(11-1) = 4 \times 10 = 40$$

For a prime power:

$$\phi(p^k) = p^k - p^{k-1}$$

For example:

$$\phi(9) = \phi(3^2) = 9 - 3 = 6$$

4.10 Fermat's Little Theorem

Fermat's Little Theorem

If p is prime and $p \nmid a$, then:

$$a^{p-1} \equiv 1 \pmod{p}$$

An equivalent form is:

$$a^p \equiv a \pmod{p}$$

Fermat's Little Theorem Example

Let $a = 3$ and $p = 7$. Since 7 is prime and does not divide 3:

$$3^6 \equiv 1 \pmod{7}$$

Indeed:

$$3^6 = 729$$

and:

$$729 \bmod 7 = 1$$

Fermat's little theorem helps simplify modular powers and motivates primality tests.

4.11 Euler's Theorem

Euler's theorem generalizes Fermat's little theorem to composite moduli.

Euler's Theorem

If $\gcd(a,n) = 1$, then:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

Euler's Theorem Example

Let $a = 2$ and $n = 15$. Since:

$$\gcd(2, 15) = 1$$

and:

$$\phi(15) = \phi(3 \times 5) = (3-1)(5-1) = 8$$

Euler's theorem gives:

$$2^8 \equiv 1 \pmod{15}$$

Since $2^8 = 256$:

$$256 \bmod 15 = 1$$

This theorem is one of the mathematical ideas that explains why RSA decryption recovers the original message under appropriate conditions.

4.12 Efficient Modular Exponentiation

Public key algorithms frequently compute:

$$a^b \bmod n$$

Calculating a^b first may create an unnecessarily large intermediate value. Repeated squaring reduces values after each multiplication.

Repeated Squaring

Repeated-Squaring Example

Compute:

$$3^{13} \bmod 17$$

Write the exponent as:

$$13 = 8 + 4 + 1$$

Calculate powers by repeated squaring:

$$3^1 \bmod 17 = 3$$

$$3^2 \bmod 17 = 9$$

$$3^4 \bmod 17 = 9^2 \bmod 17 = 81 \bmod 17 = 13$$

$$3^8 \bmod 17 = 13^2 \bmod 17 = 169 \bmod 17 = 16$$

Combine the required powers:

$$3^{13} = 3^8 \times 3^4 \times 3$$

Therefore:

$$3^{13} \bmod 17 = (16 \times 13 \times 3) \bmod 17$$

$$16 \times 13 \times 3 = 624$$

$$624 \bmod 17 = 12$$

Thus:

$$3^{13} \equiv 12 \pmod{17}$$

Repeated squaring is used in RSA, Diffie–Hellman, ElGamal, and digital-signature algorithms.

4.13 Primitive Roots and Generators

Some modular groups contain an element whose powers produce every nonzero value in the group. Such an element is called a generator or primitive root in the relevant setting.

Primitive Root Modulo a Prime

A number g is a primitive root modulo a prime p if the powers of g generate every nonzero residue modulo p.

Primitive Root Example

Consider powers of 3 modulo 7:

$$3^1 \bmod 7 = 3$$

$$3^2 \bmod 7 = 2$$

$$3^3 \bmod 7 = 6$$

$$3^4 \bmod 7 = 4$$

$$3^5 \bmod 7 = 5$$

$$3^6 \bmod 7 = 1$$

The results are:

$$1, 2, 3, 4, 5, 6$$

Therefore, 3 is a primitive root modulo 7.

Generators are important in Diffie–Hellman and ElGamal because they define the set of values produced by modular exponentiation.

4.14 Discrete Logarithms

Given:

$$y \equiv g^x \pmod{p}$$

computing y from g, x, and p is efficient. The reverse problem asks for x when g, y, and p are known.

Discrete Logarithm Problem

The discrete logarithm problem is the problem of finding x from:

$$g^x \equiv y \pmod{p}$$

when g, y, and p are known.

Small Discrete Logarithm

Solve:

$$3^x \equiv 6 \pmod{7}$$

Test small exponents:

$$3^1 \equiv 3 \pmod{7}$$

$$3^2 \equiv 2 \pmod{7}$$

$$3^3 \equiv 6 \pmod{7}$$

Therefore:

$$x = 3$$

This small example is easy by inspection. With correctly selected large parameters, recovering the exponent is intended to be computationally difficult. Diffie–Hellman and ElGamal use this asymmetry.

4.15 Chinese Remainder Theorem

The Chinese remainder theorem combines congruences with pairwise coprime moduli into one solution modulo the product of those moduli.

Chinese Remainder Theorem

If the moduli are pairwise relatively prime, a system of congruences has a unique solution modulo the product of the moduli.

Worked Example

Solve:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

Numbers congruent to 2 modulo 3 include:

$$2, 5, 8, 11, 14, 17, \ldots$$

The first value in this list that is congruent to 3 modulo 5 is 8. Therefore:

$$x \equiv 8 \pmod{15}$$

Check:

$$8 \bmod 3 = 2$$

$$8 \bmod 5 = 3$$

The solution is unique modulo:

$$3 \times 5 = 15$$

The Chinese remainder theorem can be used to perform RSA private-key computations more efficiently, although those implementation details are deferred to Chapter 6.

4.16 Primality Testing

Cryptographic systems need efficient methods for finding large probable primes. Trial division is suitable only for small classroom examples.

Fermat Primality Test Concept

If p is prime and a is not divisible by p, Fermat's little theorem predicts:

$$a^{p-1} \equiv 1 \pmod{p}$$

If the congruence fails, the candidate is definitely composite. If it succeeds, the candidate is only a probable prime because some composite numbers can also pass certain Fermat tests.

Miller–Rabin Test Concept

Miller–Rabin is a stronger probabilistic primality test. It tests a candidate using carefully structured modular exponentiation and multiple bases. A failed round proves that the candidate is composite. Repeated successful rounds provide high confidence that the candidate is prime.

R **Implementation Rule.** Applications should use trusted cryptographic libraries to generate and test primes. Developers should not implement custom primality testing for production security.

4.17 Basic Algebraic Structures

Later algorithms use sets of values together with operations that follow specific rules. Only the basic concepts are required here.

Groups

Group

A group is a set with an operation that is closed, associative, has an identity element, and gives every element an inverse.

If the operation also satisfies:

$$a \circ b = b \circ a$$

then the group is called an Abelian group.

Modular addition forms an Abelian group. For example, the integers modulo 5 under addition are:

$$\{0,1,2,3,4\}$$

The identity is 0, and every element has an additive inverse. For example:

$$2+3 \equiv 0 \pmod 5$$

so 3 is the additive inverse of 2 modulo 5.

Cyclic Groups

A cyclic group is generated by repeated application of the group operation to one element. Diffie–Hellman and elliptic curve systems operate in carefully selected cyclic groups.

Fields and Finite Fields

Field

A field is a set in which addition, subtraction, multiplication, and division by nonzero elements are defined and follow the usual algebraic rules.

A finite field contains a finite number of elements. The integers modulo a prime p, written $\mathbb{Z}_p$, form a finite field.

For example, modulo 5:

$$\mathbb{Z}_5 = \{0,1,2,3,4\}$$

Every nonzero value has a multiplicative inverse:

$$2^{-1} \equiv 3 \pmod 5$$

because:

$$2 \times 3 \equiv 1 \pmod 5$$

Finite fields are used in AES, elliptic curve cryptography, and several coding and authentication techniques. Detailed field arithmetic for AES is introduced only when needed in Chapter 5.

4.18 Matrices Modulo 26

Chapter 3 introduced the Hill cipher. The Hill cipher requires a key matrix that is invertible modulo 26.

For a 2×2 matrix:

$$K = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

its determinant is:

$$\det(K) = ad - bc$$

The matrix is invertible modulo 26 when:

$$\gcd(\det(K), 26) = 1$$

Testing a Hill Cipher Key Matrix

Consider:

$$K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$$

Its determinant is:

$$\det(K) = 3(5) - 3(2) = 9$$

Since:

$$\gcd(9, 26) = 1$$

9 has an inverse modulo 26, and the matrix is suitable for Hill cipher decryption.

The detailed Hill cipher encryption and decryption process remains in Chapter 3. This section provides only the mathematical condition that makes decryption possible.

4.19 Fermat's Factorization Method

Fermat's method represents an odd composite number as a difference of two squares:

$$n = a^2 - b^2$$

Then:

$$n = (a-b)(a+b)$$

Fermat Factorization Example

Factor 5959.
Start near:

$$\sqrt{5959} \approx 77.2$$

Try $a = 78$:

$$78^2 - 5959 = 6084 - 5959 = 125$$

125 is not a square. Try $a = 80$:

$$80^2 - 5959 = 6400 - 5959 = 441 = 21^2$$

Therefore:

$$5959 = 80^2 - 21^2$$

$$5959 = (80-21)(80+21)$$

$$5959 = 59 \times 101$$

Fermat's method is especially effective when the two factors are close. This illustrates why cryptographic prime selection must follow trusted standards and library procedures.

4.20 Common Student Mistakes

- Confusing a mod n with division.

- Confusing equality with modular congruence.
- Forgetting to reduce negative results to a standard nonnegative residue.
- Assuming that every odd number is prime.
- Assuming that every integer has an inverse modulo n.
- Forgetting to check that a GCD equals 1 before finding an inverse.
- Multiplying huge powers before applying the modulus.
- Using Fermat's little theorem when the modulus is not prime.
- Assuming that passing one probable-prime test proves primality.
- Treating small educational values as secure parameters.

4.21 Practical Activity

Cryptographic Mathematics Workshop

Total Marks: 20 marks

Complete the following tasks and show every important calculation.

1. Compute $137 \bmod 12$.
2. Compute $(19+28) \bmod 11$.
3. Compute $(7 \times 13) \bmod 10$.
4. Use the Euclidean algorithm to find $\gcd(252, 105)$.
5. Determine whether 35 and 64 are relatively prime.
6. Use the extended Euclidean algorithm to find $11^{-1} \bmod 26$.
7. Compute $\phi(21)$.
8. Use repeated squaring to compute $5^{13} \bmod 23$.
9. Determine whether 3 is a primitive root modulo 7.
10. Solve $x \equiv 1 \pmod 4$ and $x \equiv 3 \pmod 5$.
11. Determine whether $\begin{bmatrix} 7 & 8 \\ 11 & 11 \end{bmatrix}$ is invertible modulo 26.

Marking Scheme

Assessment Item	Marks
Correct modular arithmetic	3
Correct Euclidean algorithm	3
Correct extended Euclidean algorithm and inverse	4
Correct phi and modular exponentiation work	4
Correct primitive-root and CRT reasoning	3
Correct matrix invertibility test	2
Clear presentation	1

4.22 Chapter Summary

This chapter introduced the mathematical foundations required by the later cryptography chapters. Modular arithmetic works with remainders and congruence classes. Modular addition, subtraction, multiplication, and exponentiation keep calculations within a fixed range.

Prime numbers support useful finite structures and hard factorization problems. The Euclidean algorithm computes greatest common divisors efficiently, while the extended Euclidean algorithm

finds Bezout coefficients and modular inverses. An inverse modulo n exists exactly when the value and n are relatively prime.

Euler's phi function counts values relatively prime to a modulus. Fermat's little theorem applies to prime moduli, and Euler's theorem generalizes the idea to relatively prime values with composite moduli. Repeated squaring performs modular exponentiation efficiently.

Primitive roots and cyclic groups support modular key-agreement systems. The discrete logarithm problem is easy to state but difficult to reverse for appropriately selected large groups. The Chinese remainder theorem combines compatible congruences and can improve RSA computation.

The chapter also introduced probable-prime testing, finite fields, modular matrix invertibility, and Fermat factorization at the level required for later chapters. These concepts prepare the reader for symmetric encryption, RSA, Diffie–Hellman, ElGamal, ECC, and digital signatures.

4.23 Additional Exercises

1. Compute $85 \bmod 9$.
2. Compute $(-8) \bmod 13$ as a nonnegative residue.
3. Determine whether $41 \equiv 5 \pmod{12}$.
4. Find the prime factorization of 84.
5. Use the Euclidean algorithm to compute $\gcd(414, 662)$.
6. Determine whether 27 and 40 are relatively prime.
7. Find $9^{-1} \bmod 26$, if it exists.
8. Use the extended Euclidean algorithm to find $17^{-1} \bmod 43$.
9. Compute $\phi(13)$, $\phi(15)$, and $\phi(25)$.
10. Verify Fermat's little theorem for $a = 2$ and $p = 11$.
11. Verify Euler's theorem for $a = 5$ and $n = 12$.
12. Use repeated squaring to compute $7^{11} \bmod 19$.
13. List the powers of 2 modulo 11 and determine their order.
14. Solve $2^x \equiv 8 \pmod{11}$ by testing small exponents.
15. Solve $x \equiv 2 \pmod{3}$ and $x \equiv 4 \pmod{7}$.
16. Explain why a Fermat probable-prime result is not an absolute proof.
17. Find the multiplicative inverses of 2, 3, and 4 in $\mathbb{Z}_5$.
18. Determine whether $\begin{bmatrix} 2 & 4 \\ 2 & 6 \end{bmatrix}$ is invertible modulo 26.
19. Use Fermat's method to factor 2021.
20. Explain which mathematical problem supports RSA and which supports Diffie–Hellman.

4.24 Review Questions

1. What does $a \bmod n$ mean?
2. What does $a \equiv b \pmod{n}$ mean?
3. How are negative residues converted to nonnegative representatives?
4. State the main modular addition and multiplication rules.
5. What is a prime number?
6. What is a composite number?
7. Why is 1 neither prime nor composite?
8. Why are prime numbers important in cryptography?
9. What is prime factorization?
10. What is the greatest common divisor?
11. How does the Euclidean algorithm compute a GCD?
12. When are two integers relatively prime?

13. What is Bezout's identity?
14. What additional information does the extended Euclidean algorithm provide?
15. What is a modular inverse?
16. When does a modular inverse exist?
17. What does Euler's phi function count?
18. What is $\phi(p)$ when p is prime?
19. What is $\phi(pq)$ for distinct primes p and q?
20. State Fermat's little theorem.
21. State Euler's theorem.
22. How is Euler's theorem related to Fermat's little theorem?
23. Why is repeated squaring useful?
24. What is a primitive root?
25. What is a cyclic group?
26. What is the discrete logarithm problem?
27. Which cryptographic systems rely on discrete logarithms?
28. What does the Chinese remainder theorem provide?
29. How can the Chinese remainder theorem support RSA computation?
30. What is a probable prime?
31. Why can a Fermat test produce a misleading result?
32. What is the basic purpose of the Miller–Rabin test?
33. What is a group?
34. What is an Abelian group?
35. What is a finite field?
36. Why are finite fields useful in cryptography?
37. When is a 2×2 matrix invertible modulo 26?
38. What is Fermat's factorization method?
39. When is Fermat's factorization method particularly effective?
40. Why must small classroom examples never be treated as secure parameters?

5. Symmetric Key Cryptography

Learning Objectives

By the end of this chapter, the reader should be able to:

- Explain symmetric key encryption and why it is used for bulk data protection.
- Describe the basic symmetric encryption and decryption process.
- Distinguish between stream ciphers and block ciphers.
- Explain the importance of keystream uniqueness in stream ciphers.
- Describe the Feistel structure and the main stages of DES.
- Explain why DES is no longer secure for modern applications.
- Describe multiple encryption, the meet-in-the-middle attack, and Triple DES.
- Explain the AES state, key sizes, round counts, and round transformations.
- Describe AES key expansion and the inverse transformations used in decryption.
- Explain why block cipher modes are required.
- Compare ECB, CBC, CFB, OFB, CTR, and GCM conceptually.
- Explain padding, initialization vectors, nonces, and authentication tags.
- Explain authenticated encryption and associated data.
- Describe the role of cryptographically secure pseudorandom number generators.
- Explain the symmetric key-distribution problem and the key lifecycle.
- Identify common attacks and implementation mistakes.

5.1 Introduction

Symmetric key cryptography uses a shared secret key to protect and recover data. It is generally much faster than public key cryptography, which makes it suitable for files, disks, databases, backups, network sessions, wireless communication, and cloud storage.

The main challenge is that communicating parties must obtain the same secret key securely. The key must then be protected throughout its lifecycle. If an attacker obtains the key, the attacker may

be able to decrypt data or create apparently valid protected messages, depending on the algorithm and mode.

Modern systems normally use a hybrid design. Public key techniques or authenticated key-agreement protocols establish temporary key material. Symmetric encryption then protects the high-volume application data efficiently.

This chapter focuses on symmetric algorithms and the structures required to use them safely. It introduces stream and block ciphers, DES, Triple DES, AES, block cipher modes, authenticated encryption, randomness, and symmetric key management.

5.2 Basic Symmetric Encryption Process

Symmetric Key Cryptography

Symmetric key cryptography uses a shared secret key, or closely related secret keys, for encryption and decryption.

The general encryption process is:

$$C = E_K(P)$$

The general decryption process is:

$$P = D_K(C)$$

where:
- P is the plaintext.
- C is the ciphertext.
- K is the shared secret key.
- E is the encryption operation.
- D is the decryption operation.

For correct decryption:

$$D_K(E_K(P)) = P$$

Symmetric Encryption Scenario

Alice and Bob share a secret session key. Alice encrypts a message using that key and sends the ciphertext. Bob uses the corresponding secret key to recover the plaintext.
An observer may see the ciphertext but should not recover the plaintext without the key. However, confidentiality alone does not prove that the ciphertext was not modified. A secure modern design should also provide integrity and authentication.

R **Important Point.** Protecting the secret key is as important as protecting the encrypted data. Loss of the key may make authorized recovery impossible, while disclosure of the key may expose protected information.

5.3 Stream Ciphers

A stream cipher processes data continuously by combining plaintext with a generated keystream. The combining operation is commonly XOR.

Stream Cipher

A stream cipher encrypts plaintext units, such as bits or bytes, by combining them with a pseudorandom keystream generated from secret key material and a nonce or similar input.

The conceptual equations are:

$$C = P \oplus Z$$

$$P = C \oplus Z$$

where Z is the keystream.

Stream Cipher Concept

Let:

$$P = 101100$$

and:

$$Z = 011010$$

Then:

$$C = P \oplus Z = 110110$$

Applying the same keystream during decryption recovers the plaintext:

$$110110 \oplus 011010 = 101100$$

Keystream Uniqueness

The same keystream must not be reused for different messages. If:

$$C_1 = P_1 \oplus Z$$

and:

$$C_2 = P_2 \oplus Z$$

then:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

The keystream cancels, exposing a relationship between the plaintexts.

Synchronous and Self-Synchronizing Designs

A synchronous stream cipher generates the keystream from the key and internal state independently of the plaintext and ciphertext. Sender and receiver must remain synchronized.

A self-synchronizing stream cipher derives future keystream state partly from previous ciphertext symbols. Some block cipher modes, such as CFB, create stream-like behavior of this kind.

Modern Use

Stream ciphers can be efficient in software and communication protocols. ChaCha20 is a modern stream cipher frequently paired with Poly1305 authentication. Secure use requires correct nonce handling and authenticated-encryption construction.

5.4 Block Ciphers

A block cipher transforms a fixed-size plaintext block into a ciphertext block of the same size under a secret key.

Block Cipher

A block cipher is a keyed permutation that maps a fixed-size plaintext block to a ciphertext block of the same size.

For a key K:

$$C_i = E_K(P_i)$$

and:

$$P_i = D_K(C_i)$$

AES uses a 128-bit block size. DES uses a 64-bit block size.

Why Modes of Operation Are Required

A block cipher directly protects only one block. Real messages contain many blocks and may not be an exact multiple of the block size. A mode of operation defines how blocks, initialization values, nonces, counters, and authentication data are processed.

The block cipher and mode must be considered together. A strong block cipher used with an inappropriate mode can produce an insecure system.

Stream Ciphers and Block Ciphers Compared

Feature	Stream Cipher	Block Cipher
Processing unit	Bit, byte, or stream unit	Fixed-size block
Core output	Keystream	Encrypted block
Typical operation	XOR with keystream	Keyed block transformation
Reuse concern	Keystream or nonce reuse	IV, nonce, mode, and padding misuse
Examples	ChaCha20	AES and historical DES

5.5 Data Encryption Standard

The Data Encryption Standard, or DES, is a historical symmetric block cipher. It introduced many students and practitioners to Feistel networks, round functions, permutations, substitution boxes, and key schedules.

Data Encryption Standard

DES encrypts 64-bit blocks using a 56-bit effective secret key and a 16-round Feistel structure.

A DES key is represented using 64 bits, but eight positions are parity bits. The effective cryptographic key length is 56 bits.

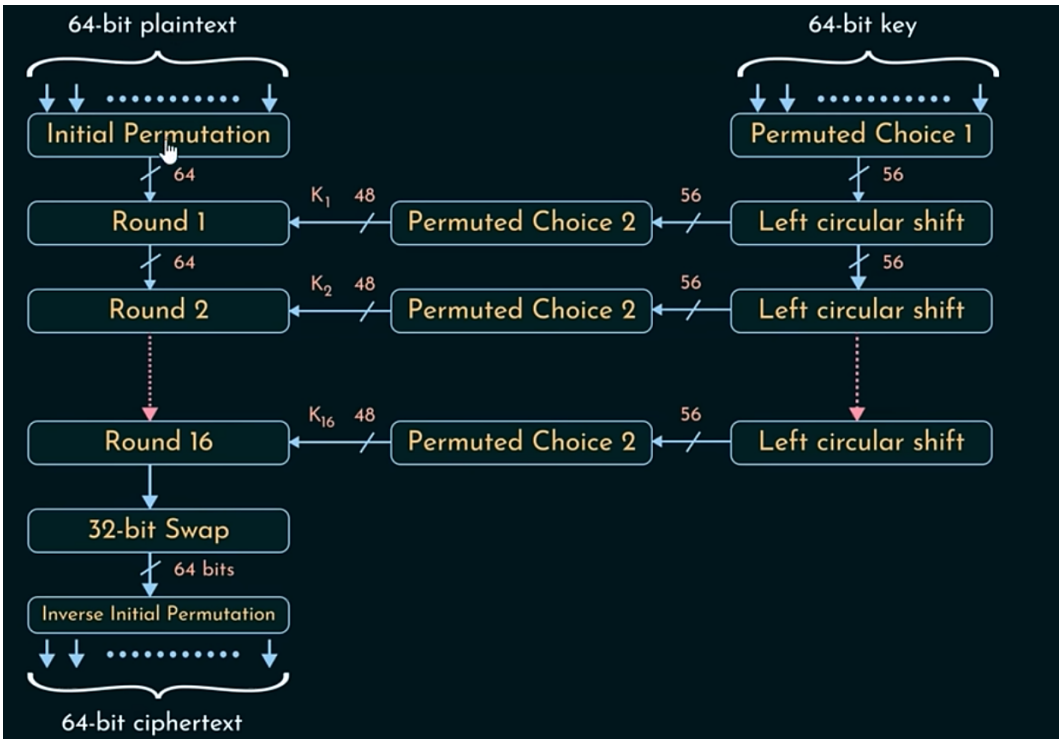


Figure 5.1: General DES encryption structure and key schedule.

General DES Process

DES performs the following stages:

1. Accept a 64-bit plaintext block.
2. Apply the initial permutation.
3. Split the result into two 32-bit halves, L_0 and R_0.
4. Generate sixteen 48-bit round keys.
5. Perform sixteen Feistel rounds.
6. Swap the final halves.
7. Apply the inverse initial permutation.
8. Produce a 64-bit ciphertext block.

Feistel Round

For round i:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

The round function expands the right half, combines it with a round key, applies S-box substitution, and applies a permutation.

DES Round Function

The DES round function performs:

1. Expansion of 32 bits to 48 bits.
2. XOR with a 48-bit round key.
3. Division into eight 6-bit groups.
4. Substitution through eight S-boxes, producing 32 bits.
5. P-permutation of the 32-bit result.

The S-boxes provide nonlinearity. Without nonlinear substitution, the cipher would be much easier to analyze algebraically.

DES Key Schedule

The key schedule performs:

1. Permuted Choice 1, selecting 56 bits from the 64-bit key input.
2. Division into 28-bit halves C_0 and D_0.
3. Round-dependent circular left shifts.
4. Permuted Choice 2, selecting a 48-bit round key.

This process produces:

$$K_1, K_2, \ldots, K_{16}$$

5.6 Worked DES Example

The standard educational example uses:

$$P = \texttt{0123456789ABCDEF}$$

$K = \texttt{133457799BBCDFF1}$

and produces:

$C = \texttt{85E813540F0AB405}$

Initial Permutation

The DES initial permutation is:

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Applying it to the plaintext gives:

$\texttt{CC00CCFFF0AAF0AA}$

Therefore:

$L_0 = \texttt{CC00CCFF}$

$R_0 = \texttt{F0AAF0AA}$

The first four output bits are selected from positions 58, 50, 42, and 34. Their values are `1100`, which is hexadecimal `C`. This illustrates that a permutation only selects and rearranges existing bits.

First Round Key

After PC-1:

$C_0 = \texttt{F0CCAAF}$

$D_0 = \texttt{556678F}$

After one circular left shift:

$C_1 = \texttt{E19955F}$

$D_1 = \texttt{AACCF1E}$

Combining the halves gives:

$C_1D_1 = \texttt{E19955FAACCF1E}$

After PC-2:

$K_1 = \texttt{1B02EFFC7072}$

Expansion and XOR

Expand R_0:

$E(R_0) = \texttt{7A15557A1555}$

Then combine with the first round key:

$E(R_0) \oplus K_1 = \texttt{6117BA866527}$

S-Boxes and P-Permutation

The 48-bit value is divided into eight 6-bit groups. The first group is:

`011000`

The outer bits select row 0, and the inner bits `1100` select column 12. DES S-box S_1 returns decimal 5, or binary `0101`.

After all eight S-boxes:

`5C82B597`

After the P-permutation:

$F(R_0, K_1) = \texttt{234AA9BB}$

Round 1 Output

$L_1 = R_0 = \texttt{F0AAF0AA}$

$R_1 = L_0 \oplus F(R_0, K_1)$

$R_1 = \texttt{CC00CCFF} \oplus \texttt{234AA9BB} = \texttt{EF4A6544}$

After sixteen rounds, the final halves are swapped and the inverse initial permutation produces:

`85E813540F0AB405`

Important Point. The detailed example demonstrates DES structure. It does not make DES suitable for modern use. The effective 56-bit key is too small.

Why DES Is No Longer Secure

The main weakness is the 56-bit key space. Exhaustive key search became practical with specialized hardware. DES also has a small 64-bit block size, which is unsuitable for large modern data volumes.

DES should be studied historically and conceptually, not deployed for new sensitive systems.

5.7 Multiple Encryption and Triple DES

Applying a block cipher more than once appears to increase security. However, multiple encryption must be analyzed carefully.

Double DES and Meet-in-the-Middle

Double DES would compute:

$$C = E_{K_2}(E_{K_1}(P))$$

A meet-in-the-middle attack stores values computed forward from known plaintext under possible K_1 keys and compares them with values computed backward from ciphertext under possible K_2 keys. This greatly reduces the expected security compared with a naive 112-bit estimate.

Triple DES

Triple DES applies DES three times, commonly using an encrypt-decrypt-encrypt structure:

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$

The EDE form supported compatibility with single DES when appropriate keys were equal.

Triple DES provided a transition path away from DES, but it is slow and retains the 64-bit DES block size. New systems should use modern authenticated encryption based on AES or another approved modern construction.

5.8 Advanced Encryption Standard

AES is the principal modern block cipher taught and used in contemporary systems.

Advanced Encryption Standard

AES is a symmetric block cipher with a 128-bit block size and key sizes of 128, 192, or 256 bits.

AES Parameters

Version	Key Size	Block Size	Rounds
AES-128	128 bits	128 bits	10
AES-192	192 bits	128 bits	12
AES-256	256 bits	128 bits	14

AES is a substitution-permutation network rather than a Feistel cipher. Encryption and decryption use related but different transformation sequences.

The AES State

AES organizes the 128-bit input block as 16 bytes in a 4×4 state matrix. Input bytes are placed column by column:

$$\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}$$

Each round transforms this state.

Initial AddRoundKey

Before the numbered rounds, AES XORs the state with the initial round key:

$$\text{State} = \text{State} \oplus \text{RoundKey}_0$$

Every bit of the state is influenced by key material from the beginning.

SubBytes

SubBytes replaces every byte using the AES S-box. The S-box is a fixed nonlinear transformation derived using finite-field inversion and an affine transformation.

The purpose is to introduce nonlinearity and confusion. A byte value is used as an index into the S-box, and the corresponding substituted byte replaces it.

ShiftRows

ShiftRows rotates the rows of the state:

- Row 0 is unchanged.
- Row 1 shifts left by one byte.
- Row 2 shifts left by two bytes.
- Row 3 shifts left by three bytes.

This moves bytes between columns and contributes to diffusion.

ShiftRows Example

Before ShiftRows:

$$\begin{bmatrix} a_0 & a_1 & a_2 & a_3 \\ b_0 & b_1 & b_2 & b_3 \\ c_0 & c_1 & c_2 & c_3 \\ d_0 & d_1 & d_2 & d_3 \end{bmatrix}$$

After ShiftRows:

$$\begin{bmatrix} a_0 & a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 & b_0 \\ c_2 & c_3 & c_0 & c_1 \\ d_3 & d_0 & d_1 & d_2 \end{bmatrix}$$

MixColumns

MixColumns transforms each state column using arithmetic in the finite field $GF(2^8)$. Conceptually, each output byte depends on all four input bytes in the same column.

The fixed matrix is:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Multiplied by a state column in $GF(2^8)$, this transformation spreads byte changes throughout the column.

The detailed polynomial arithmetic is not required at this stage. Students should understand that AES byte multiplication is finite-field multiplication, not ordinary integer multiplication.

AddRoundKey

AddRoundKey XORs the state with a round key generated by the AES key schedule:

$$\text{State} = \text{State} \oplus \text{RoundKey}_i$$

AddRoundKey is the only AES round operation that directly introduces secret key material.

AES Round Structure

For AES-128:

1. Initial AddRoundKey.
2. Nine full rounds containing SubBytes, ShiftRows, MixColumns, and AddRoundKey.
3. One final round containing SubBytes, ShiftRows, and AddRoundKey.

The final round omits MixColumns.

AES Key Expansion

AES derives a different round key for every round. AES-128 expands a 128-bit cipher key into eleven 128-bit round keys, including the initial key.

Important key-schedule operations include:

- **RotWord:** rotates a four-byte word.
- **SubWord:** applies the AES S-box to each byte.
- **Rcon:** adds a round-dependent constant.
- XOR with earlier words to generate new words.

The key schedule ensures that each encryption round uses different derived key material.

AES Decryption

AES decryption applies inverse transformations with round keys in reverse order:

- InvShiftRows.
- InvSubBytes.
- AddRoundKey.
- InvMixColumns in the applicable rounds.

The inverse operations recover the original state when the correct key is used.

AES Security and Implementation

AES is widely trusted when used with appropriate keys, modes, nonces, and implementations. Practical risks often arise from:

- Side-channel leakage.
- Incorrect modes.
- Nonce reuse.
- Weak key generation.
- Exposed keys.
- Missing authentication.
- Incorrect error handling.

AES software should use trusted, reviewed cryptographic libraries and platform-supported hardware acceleration where appropriate.

5.9 Block Cipher Modes of Operation

A mode defines how a block cipher protects messages longer than one block and how it uses IVs, nonces, counters, feedback, padding, and authentication.

5.10 Electronic Codebook Mode

ECB encrypts blocks independently:

$$C_i = E_K(P_i)$$

Identical plaintext blocks under the same key produce identical ciphertext blocks. This reveals patterns and relationships.

ECB may be useful for explaining direct block encryption, but it should not be used to protect structured or multi-block sensitive data.

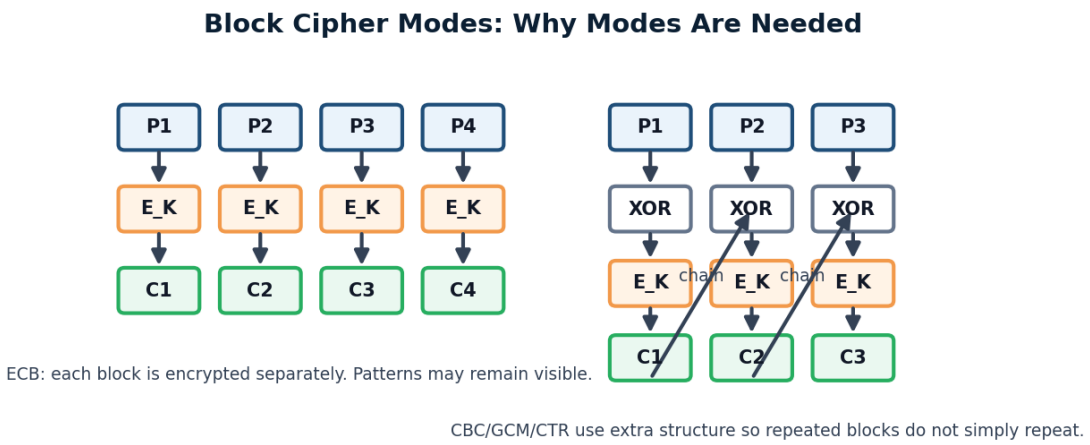


Figure 5.2: Conceptual structures used by common block cipher modes.

5.11 Cipher Block Chaining Mode

CBC XORs each plaintext block with the previous ciphertext block before encryption:

$$C_i = E_K(P_i \oplus C_{i-1})$$

For the first block:

$$C_0 = IV$$

Decryption is:

$$P_i = D_K(C_i) \oplus C_{i-1}$$

CBC hides repeated plaintext blocks when the IV is used correctly. It usually requires padding and separate authentication. Incorrect padding-error handling can create padding-oracle vulnerabilities.

5.12 Cipher Feedback Mode

CFB uses a block cipher to create stream-like encryption. The previous ciphertext value is encrypted, and the result is combined with plaintext.

Conceptually:

$$C_i = P_i \oplus E_K(C_{i-1})$$

with an IV used before the first segment.

CFB can process units smaller than the block size and is self-synchronizing under certain designs. It does not provide authentication by itself.

5.13 Output Feedback Mode

OFB repeatedly encrypts an internal feedback value to generate a keystream:

$$O_i = E_K(O_{i-1})$$

$$C_i = P_i \oplus O_i$$

The initial feedback value is derived from an IV. Because ciphertext is not fed back, transmission errors do not propagate in the same way as CFB. Reusing the same IV and key repeats the keystream and is dangerous.

5.14 Counter Mode

CTR encrypts a sequence of unique counter blocks to produce a keystream:

$$Z_i = E_K(\text{Nonce} \parallel \text{Counter}_i)$$

$$C_i = P_i \oplus Z_i$$

Advantages include:

- Parallel encryption and decryption.
- Random access to blocks.
- No padding requirement.
- Precomputation of keystream blocks.

The nonce-counter input must never repeat under the same key. CTR provides confidentiality but not authentication by itself.

5.15 Galois Counter Mode

GCM combines counter-mode encryption with an authentication function. It provides authenticated encryption and can also authenticate unencrypted associated data.

GCM produces:

- Ciphertext.
- An authentication tag.

The receiver verifies the tag before accepting the plaintext.

Associated Data

Associated data is information that is authenticated but not encrypted, such as protocol headers, message types, or routing information.

Nonce uniqueness is critical. Reusing a nonce under the same GCM key can expose relationships between plaintexts and undermine authentication.

5.16 Mode Comparison

Mode	Pattern Hiding	Padding	Authentication	Important Rule
ECB	No	Usually yes	No	Avoid for sensitive multi-block data
CBC	Yes	Usually yes	No	Use IV correctly and authenticate separately
CFB	Yes	No	No	Use the IV correctly
OFB	Yes	No	No	Never repeat keystream input
CTR	Yes	No	No	Never repeat nonce-counter values
GCM	Yes	No	Yes	Never reuse a nonce with the same key

5.17 Padding

Some modes require plaintext length to be an exact multiple of the block size. A padding scheme adds bytes that can be removed unambiguously after decryption.

A common conceptual rule adds N bytes, each containing the value N. If five bytes are required, five bytes with value 5 are added.

Padding must be validated carefully. Different error responses for invalid padding can reveal information to attackers. Authenticated encryption avoids many of the design problems associated with unauthenticated padded modes.

5.18 Initialization Vectors, Nonces, and Counters

These values provide uniqueness or unpredictability required by a mode.

- An **IV** is an initialization value used by modes such as CBC, CFB, and OFB.
- A **nonce** is a value intended for one-time use under a given key.
- A **counter** changes for each block in counter-based modes.

These values usually do not need to be secret, but their requirements differ by mode. Some must be unpredictable, some must be unique, and some require both careful formatting and uniqueness.

Important Point. Do not assume that all IVs and nonces have the same requirements. Follow the exact specification for the selected algorithm and mode.

5.19 Authenticated Encryption

Encryption alone does not provide a complete guarantee that ciphertext has not been modified. Some encryption modes are malleable, meaning an attacker can change ciphertext in ways that cause controlled or unpredictable plaintext changes.

Authenticated encryption provides confidentiality and integrity together. The receiver must verify the authentication tag before using the plaintext.

Examples include:

- AES-GCM.
- ChaCha20-Poly1305.
- AES-CCM.

Authentication Tag

An authentication tag is a value produced by an authenticated-encryption algorithm and verified to detect unauthorized modification of ciphertext and associated data.

If tag verification fails, the message must be rejected. Applications should not use unauthenticated plaintext produced from a failed verification attempt.

5.20 Randomness and Pseudorandom Generation

Symmetric cryptography requires unpredictable keys and often requires nonces, IVs, challenges, salts, and temporary values.

Cryptographically Secure Pseudorandom Number Generator

A CSPRNG is a deterministic generator designed so that its output is computationally unpredictable without knowledge of its internal state.

A CSPRNG is initialized or refreshed using entropy from physical and system sources. Ordinary simulation-oriented random functions may be predictable and unsuitable for key generation.

Important Randomness Properties

A secure generator should aim to provide:

- Unpredictable output.
- Sufficient entropy in its seed.
- Resistance to state compromise when supported.
- Correct reseeding behavior.
- Separation between unrelated cryptographic purposes.

Golomb's Randomness Postulates

For periodic binary sequences, Golomb described useful statistical properties:

- **Balance:** the number of ones and zeros should be nearly equal.
- **Run distribution:** runs of different lengths should occur in an expected pattern.
- **Autocorrelation:** shifted versions of the sequence should not correlate strongly except at full-period alignment.

These properties help explain desirable sequence behavior, but statistical appearance alone does not prove cryptographic unpredictability.

5.21 The Symmetric Key-Distribution Problem

Both parties need the same secret key. Secure key establishment may use:

- Secure physical provisioning.
- A trusted key-distribution center.
- Public key encryption of a symmetric key.
- Authenticated Diffie–Hellman or elliptic curve key agreement.
- Pre-shared keys installed through a protected process.

The key-establishment method must authenticate the parties. Otherwise, an attacker may perform a man-in-the-middle attack and establish separate keys with each side.

5.22 Symmetric Key Management

A complete key lifecycle includes:

1. Generate the key using a CSPRNG.
2. Establish or distribute the key securely.
3. Store the key in protected cryptographic storage.
4. Limit access and permitted operations.
5. Use the key with an approved algorithm and mode.
6. Rotate or replace the key according to policy and usage limits.
7. Revoke or disable the key after suspected compromise.
8. Archive a recovery key only when organizational requirements justify it.
9. Destroy retired key material securely.

Keys should be separated by purpose. An encryption key should not automatically be reused as a MAC key, key-encryption key, or unrelated application key.

5.23 Common Attacks and Implementation Mistakes

- Selecting DES, Triple DES, or another outdated algorithm for a new system.
- Using ECB for structured sensitive data.
- Reusing a stream-cipher keystream.
- Reusing a CTR or GCM nonce under the same key.
- Using CBC without authentication.
- Revealing padding-validation details.
- Hard-coding keys in source code.
- Storing keys next to ciphertext without separate protection.
- Generating keys with a predictable random function.
- Using encryption without an authentication tag or MAC.
- Ignoring authentication-tag verification failures.
- Reusing one key for unrelated purposes.
- Encrypting excessive data under one key without considering usage limits.
- Logging keys, plaintext secrets, IV-state information, or sensitive intermediate values.
- Implementing AES, DES, modes, or padding manually instead of using a trusted library.
- Ignoring timing, cache, power, or electromagnetic side channels.

5.24 Case Study: Encrypting Student Records

Suppose a university stores student identities, addresses, payment records, and academic documents.

A suitable design may include:

- AES-based authenticated encryption for sensitive database fields or encrypted files.
- Unique nonces generated according to the selected mode.
- Keys stored in a protected key-management service rather than in the database.
- Separate keys for development, testing, and production.
- Access controls restricting which application components may request cryptographic operations.
- Audit logs for key use and administrative actions.
- Encrypted backups with tested restoration procedures.
- Planned key rotation and response to suspected compromise.

Encryption should be combined with authorization, secure coding, monitoring, backup, and incident response. Encrypting a database does not help if an attacker compromises an authorized application and requests legitimate decryption operations.

5.25 Practical Group Activity

Design a Symmetric Encryption Solution

Group Size: 3 to 5 students
Total Marks: 20 marks
Design a solution for protecting one of the following:

- Student records.
- Cloud backups.
- A secure messaging session.
- A laptop disk.
- Files exchanged between two departments.

Your design must identify:

1. The protected assets.
2. Whether protection is needed at rest, in transit, or both.
3. The selected modern algorithm and mode.
4. The IV or nonce requirements.
5. Whether padding is required.
6. How integrity and authentication are provided.
7. How keys are generated, distributed, stored, rotated, and revoked.
8. At least four implementation mistakes that must be avoided.

Marking Scheme

Assessment Item	Marks
Correct asset and requirement analysis	3
Appropriate algorithm and mode	4
Correct nonce, IV, and padding explanation	3
Integrity and authentication design	3
Complete key-management lifecycle	4
Attack and misuse analysis	2
Teamwork and presentation	1

5.26 Chapter Summary

This chapter introduced symmetric key cryptography as the primary method for protecting large amounts of data efficiently. Stream ciphers generate a keystream that is combined with plaintext, while block ciphers transform fixed-size blocks under a secret key.

DES demonstrated the Feistel structure, round functions, S-boxes, permutations, and key schedules, but its 56-bit key and 64-bit block size make it unsuitable for modern applications. Double DES is weakened by a meet-in-the-middle attack, and Triple DES is a legacy transition mechanism rather than the preferred choice for new systems.

AES is a modern 128-bit block cipher supporting 128-bit, 192-bit, and 256-bit keys. Its encryption process uses SubBytes, ShiftRows, MixColumns, and AddRoundKey. The final round omits MixColumns, and the key schedule creates a separate round key for each round.

Block cipher modes define how longer messages are processed. ECB reveals repeated patterns. CBC, CFB, OFB, and CTR provide confidentiality under different IV and nonce requirements but

do not inherently authenticate messages. GCM provides authenticated encryption and supports associated data.

Secure implementation requires correct padding, unique or unpredictable initialization values as specified, verified authentication tags, cryptographically secure randomness, protected keys, and trusted libraries. Strong algorithms cannot compensate for nonce reuse, exposed keys, weak randomness, missing authentication, or faulty key management.

5.27 Additional Exercises

1. Explain why symmetric encryption is suitable for bulk data.
2. Compare stream ciphers and block ciphers.
3. Show algebraically why reusing a stream-cipher keystream is dangerous.
4. State the DES block size, effective key size, and number of rounds.
5. Explain the purpose of the DES S-boxes.
6. Explain why the initial permutation is not the main source of DES security.
7. Describe the meet-in-the-middle idea against Double DES.
8. Explain the EDE structure of Triple DES.
9. State the AES block size, key sizes, and round counts.
10. Write a 4×4 AES state using sixteen labeled bytes.
11. Apply ShiftRows to a labeled AES state.
12. Explain the purpose of SubBytes.
13. Explain the purpose of MixColumns.
14. Explain why AddRoundKey is necessary.
15. List the transformations in a full AES encryption round.
16. Explain how the final AES round differs from the earlier rounds.
17. Compare ECB, CBC, CFB, OFB, CTR, and GCM.
18. Explain why CBC commonly requires padding.
19. Explain why CTR does not require padding.
20. Explain why GCM nonce reuse is dangerous.
21. Give three examples of associated data.
22. Explain the difference between an IV, nonce, and counter.
23. Explain why statistical randomness is not sufficient for cryptographic keys.
24. Describe the symmetric key lifecycle.
25. Design a key-rotation plan for encrypted university backups.

5.28 Review Questions

1. What is symmetric key cryptography?
2. Why is symmetric encryption generally faster than public key encryption?
3. What is a stream cipher?
4. What is a keystream?
5. Why must a keystream not be reused?
6. What is a block cipher?
7. Why are modes of operation required?
8. What are the DES block size and effective key size?
9. How many rounds does DES use?
10. What is a Feistel network?
11. What operations appear in the DES round function?
12. What is the purpose of PC-1?
13. What is the purpose of PC-2?

14. Why is DES no longer secure?
15. What is a meet-in-the-middle attack?
16. What is Triple DES?
17. Why is AES preferred over Triple DES for new systems?
18. What is the AES block size?
19. Which AES key sizes are supported?
20. How many rounds do AES-128, AES-192, and AES-256 use?
21. What is the AES state?
22. What does SubBytes do?
23. What does ShiftRows do?
24. What does MixColumns do?
25. What does AddRoundKey do?
26. Which operation is omitted from the final AES round?
27. What is AES key expansion?
28. Why is ECB mode weak?
29. How does CBC connect blocks?
30. How does CFB create stream-like encryption?
31. How does OFB generate a keystream?
32. How does CTR generate a keystream?
33. What does GCM add to CTR-style encryption?
34. What is an authentication tag?
35. What is associated data?
36. Why is padding required in some modes?
37. What is a padding-oracle risk?
38. What is an initialization vector?
39. What is a nonce?
40. Why do IV and nonce requirements depend on the mode?
41. What is authenticated encryption?
42. What is a CSPRNG?
43. What are Golomb's three postulates?
44. What is the symmetric key-distribution problem?
45. Name four secure key-establishment methods.
46. Why should keys be separated by purpose?
47. List five common symmetric encryption mistakes.

Three

6. Public Key Cryptography

Learning Objectives

By the end of this chapter, the reader should be able to:

- Define public key cryptography and explain why it is called asymmetric cryptography.
- Distinguish public keys from private keys.
- Explain why a public key must be authentic even though it is not secret.
- Distinguish public key encryption, key agreement, and digital signatures.
- Describe the main stages of RSA key generation.
- Encrypt and decrypt a small educational RSA message.
- Explain why textbook RSA is unsafe for real applications.
- Describe Diffie–Hellman key agreement and calculate a small example.
- Explain why unauthenticated Diffie–Hellman is vulnerable to a man-in-the-middle attack.
- Explain ephemeral key agreement and forward secrecy.
- Describe ElGamal encryption and the importance of fresh randomness.
- Explain elliptic curve cryptography, ECDH, and ECDSA conceptually.
- Explain how hybrid encryption combines public key and symmetric techniques.
- Identify common attacks and implementation mistakes in public key systems.

6.1 Introduction

Public key cryptography, also called asymmetric cryptography, uses a pair of mathematically related keys. The public key may be distributed, while the private key must remain secret and under the control of its owner.

Different public key algorithms support different operations. Some provide encryption, some establish shared key material, and others create digital signatures. A system should use each key only for its approved purpose.

Public key operations are usually slower than symmetric encryption. Modern systems therefore use public key methods for authentication, key establishment, or protection of small values, then use symmetric authenticated encryption for application data.

Public Key Cryptography

Public key cryptography uses mathematically related public and private keys for encryption, key establishment, digital signatures, or related security operations.

The mathematical foundations required in this chapter, including prime numbers, modular inverses, Euler's phi function, repeated squaring, primitive roots, and discrete logarithms, were introduced in Chapter 4.

6.2 Public and Private Keys

Public and Private Key Pair

A public key may be distributed to other parties. The related private key must remain protected by its owner.

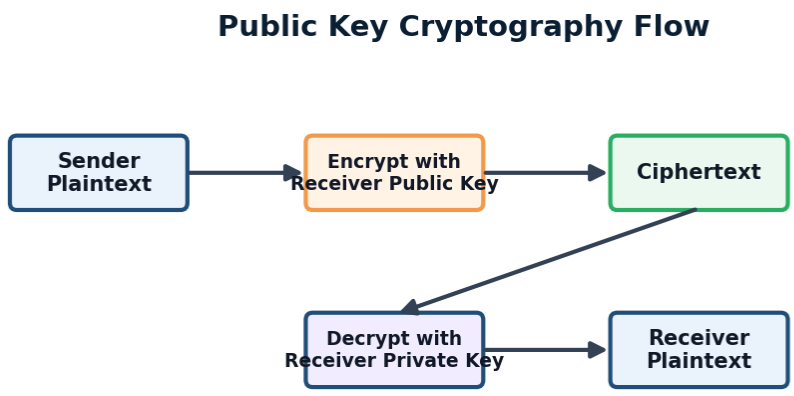


Figure 6.1: Public key encryption uses a public key for encryption and the corresponding private key for decryption.

A public and private key pair has the following general properties:

- The keys are mathematically related.
- The public key does not require secrecy.
- The private key must remain confidential and protected from unauthorized use.
- Recovering the private key from the public key should be computationally infeasible.
- Public keys must be authenticated before they are trusted.
- Keys should be restricted to their intended purpose.

Why Public-Key Authenticity Matters

An attacker can generate a key pair and claim that the public key belongs to another person or server. If Alice encrypts a session key using an attacker's substituted public key, the attacker may

decrypt it.

Public-key authenticity may be established through:

- Digital certificates and certificate authorities.
- Verified public-key fingerprints.
- Trusted directories.
- Secure physical provisioning.
- Previously authenticated communication channels.

Certificates and public key infrastructure are discussed fully in Chapter 8.

6.3 Three Main Public Key Operations

Public key encryption, key agreement, and digital signatures should not be treated as the same operation.

Public Key Encryption

A sender encrypts a small value using the receiver's authentic public encryption key. The receiver decrypts it using the corresponding private key.

$$C = E_{K_{\text{pub}}}(M)$$

$$M = D_{K_{\text{priv}}}(C)$$

Public key encryption is commonly used to protect a symmetric session key or another short secret rather than a large file.

Key Agreement

In key agreement, both parties contribute information and calculate the same shared secret. The shared secret is not normally transmitted directly. Diffie–Hellman and ECDH are key-agreement methods.

Digital Signatures

A signer uses a private signing key to create a signature. A verifier uses the corresponding public key to verify it. Digital signatures are explained in detail in Chapter 8.

Important Point. Encryption, key agreement, and digital signatures have different purposes and security requirements. A library or protocol should use the exact algorithm designed for the required operation.

6.4 RSA

RSA is a public key algorithm based on arithmetic with large integers. Its security is associated with the difficulty of factoring a large modulus formed from carefully generated prime numbers.

RSA may support encryption and digital signatures, but each operation requires an approved encoding scheme. The simple mathematical form presented here is only for education.

RSA Key Generation

A simplified RSA key-generation process is:

1. Choose two distinct prime numbers p and q.
2. Compute the modulus:

$$n = pq$$

3. Compute:

$$\phi(n) = (p-1)(q-1)$$

4. Choose a public exponent e satisfying:

$$\gcd(e, \phi(n)) = 1$$

5. Compute the private exponent d as the inverse of e modulo $\phi(n)$:

$$ed \equiv 1 \pmod{\phi(n)}$$

The public key is:

$$(e, n)$$

The private key includes:

$$(d, n)$$

Real implementations may retain additional prime-related values to accelerate private-key operations using the Chinese remainder theorem.

Educational Encryption and Decryption Equations

The simplified encryption equation is:

$$C = M^e \bmod n$$

The simplified decryption equation is:

$$M = C^d \bmod n$$

The message representative must satisfy the requirements of the real encoding scheme. Direct use of these equations without secure encoding is unsafe.

6.5 Complete Small RSA Example

This example uses small values so that students can follow the mathematics. The values are completely insecure.

Step 1: Choose the Primes

Choose:

$$p = 5, \qquad q = 11$$

Step 2: Compute the Modulus

$$n = pq = 5 \times 11 = 55$$

Step 3: Compute Euler's Phi Function

$$\phi(n) = (p-1)(q-1)$$

$$\phi(55) = 4 \times 10 = 40$$

Step 4: Choose the Public Exponent

Choose:

$$e = 3$$

because:

$$\gcd(3, 40) = 1$$

Step 5: Find the Private Exponent

Find d such that:

$$3d \equiv 1 \pmod{40}$$

A solution is:

$$d = 27$$

because:

$$3 \times 27 = 81 \equiv 1 \pmod{40}$$

Therefore:

$$\text{Public key} = (3, 55)$$

$$\text{Private key} = (27, 55)$$

Step 6: Encrypt a Message

Let:

$$M = 7$$

Then:

$$C = 7^3 \bmod 55$$

$$C = 343 \bmod 55 = 13$$

Therefore:

$$C = 13$$

Step 7: Decrypt the Ciphertext

$$M = 13^{27} \bmod 55$$

Use repeated squaring:

$$13^2 \bmod 55 = 4$$

$$13^4 \bmod 55 = 16$$

$$13^8 \bmod 55 = 36$$

$$13^{16} \bmod 55 = 31$$

Since:

$$27 = 16 + 8 + 2 + 1$$

then:

$$13^{27} \bmod 55 = (31 \times 36 \times 4 \times 13) \bmod 55 = 7$$

The original message is recovered:

$$M = 7$$

R **Educational Warning.** Real RSA uses large keys, carefully generated primes, secure encoding, protected private-key operations, and trusted libraries.

6.6 Why Textbook RSA Is Unsafe

Direct textbook RSA has several serious problems:

- It is deterministic. The same message produces the same ciphertext.
- It exposes algebraic structure.
- It does not encode messages safely.
- It does not provide integrity or authentication automatically.
- It can be vulnerable to chosen-ciphertext and related attacks.
- It is unsuitable for encrypting large application messages directly.

Real RSA encryption uses a secure randomized encoding such as OAEP. Real RSA signatures use an approved signature encoding such as RSA-PSS. Applications should rely on trusted libraries rather than implementing RSA mathematics directly.

6.7 RSA Security and Implementation

RSA security depends on more than modulus size. Important requirements include:

- Strong prime generation.
- Sufficient key size.
- Secure padding and encoding.
- Validation of inputs and parameters.
- Protection against timing and fault attacks.
- Secure private-key storage.
- Correct handling of errors.

RSA blinding can reduce information leaked through private-key timing behavior. Hardware security modules can restrict direct access to sensitive private-key material.

6.8 Diffie–Hellman Key Agreement

Diffie–Hellman allows two parties to calculate the same shared secret across an insecure channel without transmitting that shared secret directly.

The public parameters are:

- A suitable prime modulus p.
- A generator g of an appropriate subgroup.

Alice chooses private value a and computes:

$$A = g^a \bmod p$$

Bob chooses private value b and computes:

$$B = g^b \bmod p$$

After exchanging A and B, Alice computes:

$$S = B^a \bmod p$$

Bob computes:

$$S = A^b \bmod p$$

Both obtain:

$$g^{ab} \bmod p$$

The shared value is normally processed by a key-derivation function rather than used directly as an encryption key.

6.9 Small Diffie–Hellman Example

Use the public parameters:

$$p = 23, \qquad g = 5$$

Alice chooses:

$$a = 6$$

Bob chooses:

$$b = 15$$

Alice's Public Value

$A = 5^6 \bmod 23$

Using repeated squaring:

$$5^2 \equiv 2 \pmod{23}$$

$$5^4 \equiv 4 \pmod{23}$$

$$5^6 \equiv 5^4 \times 5^2 \equiv 4 \times 2 = 8 \pmod{23}$$

Therefore:

$$A = 8$$

Bob's Public Value

$B = 5^{15} \bmod 23 = 19$

Shared Secret

Alice computes:

$$S = 19^6 \bmod 23 = 2$$

Bob computes:

$$S = 8^{15} \bmod 23 = 2$$

Both obtain:

$$S = 2$$

R **Important Point.** Alice and Bob exchange public values, not the shared secret or private exponents.

6.10 Authenticated Diffie–Hellman

Basic Diffie–Hellman does not identify the parties. An attacker can replace the exchanged public values and create one secret with Alice and another with Bob.

Authentication may be added using:

- Digital signatures.
- Certificates.
- Pre-shared authentication keys.
- Authenticated key-exchange protocols.
- Verified public-key fingerprints.

The parties should also validate received public values and group parameters. Failure to validate them may enable small-subgroup or invalid-parameter attacks.

6.11 Ephemeral Diffie–Hellman and Forward Secrecy

Ephemeral Diffie–Hellman generates temporary private exponents for a session. The temporary secrets should be removed after the session.

Forward Secrecy

Forward secrecy is the property that compromise of a long-term private key should not reveal previously established session keys.

Forward secrecy depends on correct protocol design, authenticated ephemeral exchange, secure random generation, and deletion of temporary secrets.

6.12 Key Derivation after Key Agreement

The raw Diffie–Hellman shared value should not normally be used directly as an application key. A key-derivation function processes the shared value together with contextual information.

A KDF may derive separate keys for:

- Client-to-server encryption.
- Server-to-client encryption.
- Authentication or transcript binding.
- IV or nonce derivation when specified.

Key separation prevents one key from being reused for unrelated directions or cryptographic purposes.

6.13 ElGamal Encryption

ElGamal is a randomized public key encryption system based on the discrete logarithm problem.

Key Generation

Choose suitable public parameters p and g. Select a private key x, then compute:

$$y = g^x \bmod p$$

The public key contains (p, g, y), while x remains private.

Encryption

For message representative M, choose a fresh random value k. Compute:

$$c_1 = g^k \bmod p$$

$$c_2 = M y^k \bmod p$$

The ciphertext is:

$$(c_1, c_2)$$

Decryption

The receiver computes:

$$s = c_1^x \bmod p$$

and recovers:

$$M = c_2 s^{-1} \bmod p$$

Small ElGamal Example

Use:

$$p = 23, \qquad g = 5$$

Let the private key be:

$$x = 6$$

Then the public value is:

$$y = 5^6 \bmod 23 = 8$$

Encrypt:

$$M = 10$$

using fresh random value:

$$k = 3$$

Compute:

$$c_1 = 5^3 \bmod 23 = 10$$

$$y^k = 8^3 \bmod 23 = 6$$

$$c_2 = 10 \times 6 \bmod 23 = 14$$

The ciphertext is:

$$(10, 14)$$

For decryption:

$$s = c_1^x \bmod 23 = 10^6 \bmod 23 = 6$$

The inverse of 6 modulo 23 is 4 because:

$$6 \times 4 \equiv 1 \pmod{23}$$

Therefore:

$$M = 14 \times 4 \bmod 23 = 10$$

The original message is recovered.

Important Point. The random value k must be fresh and unpredictable for every ElGamal encryption. Reuse or prediction can compromise security.

6.14 Elliptic Curve Cryptography

Elliptic curve cryptography uses groups formed from points on carefully selected curves over finite fields.

A commonly presented curve form is:

$$y^2 = x^3 + ax + b$$

The curve parameters must satisfy:

$$4a^3 + 27b^2 \neq 0$$

in the relevant field so that the curve is nonsingular.

Points and the Point at Infinity

An elliptic curve group contains:

- Points (x, y) satisfying the curve equation.
- A special identity element called the point at infinity.

The group operation is point addition. Repeated addition of a point P is scalar multiplication:

$$Q = kP$$

Computing Q from k and P is efficient. Recovering k from P and Q is the elliptic curve discrete logarithm problem.

Benefits of ECC

ECC can provide strong security using comparatively smaller keys than traditional integer-based public key systems. Smaller keys may reduce storage, bandwidth, and computational cost.

ECC is used in:

- TLS and secure network protocols.
- Mobile and embedded systems.
- Smart cards and constrained devices.
- Key agreement.
- Digital signatures.

ECDH

Elliptic Curve Diffie–Hellman uses elliptic curve scalar multiplication for key agreement. Alice and Bob exchange public points and calculate the same shared point using their private scalars.

ECDH must be authenticated and received points must be validated according to the selected protocol and curve.

ECDSA

ECDSA is an elliptic curve digital-signature algorithm. It uses a private signing scalar and corresponding public point. Signature generation requires secure per-signature values. Reusing or predicting these values can reveal the private signing key.

Digital signatures are developed fully in Chapter 8.

R **Important Point.** ECC security depends on approved curves, correct point validation, secure randomness, constant-time implementations, and protected private keys.

6.15 Hybrid Encryption

Hybrid encryption combines public key techniques with symmetric authenticated encryption.

A typical process is:

1. Generate or establish fresh symmetric key material.
2. Authenticate the public keys or key-agreement messages.
3. Protect or derive the session key using a public key method.
4. Encrypt application data using an authenticated symmetric algorithm.
5. Transmit the required public information, nonce, ciphertext, associated data, and tag.
6. Recover or derive the session key at the receiver.
7. Verify the authentication tag before accepting the plaintext.

Secure File Transfer

A sender generates a random AES session key and encrypts a large file using AES-GCM. The sender protects the session key using the receiver's authentic public key or derives it through authenticated key agreement.

The receiver recovers the session key, verifies the GCM tag, and decrypts the file. Public key cryptography solves the session-key problem, while symmetric encryption protects the large file efficiently.

6.16 Public Key and Symmetric Cryptography Compared

Feature	Symmetric Cryptography	Public Key Cryptography
Key model	Shared secret	Public and private key pair
Speed	Fast	Relatively slower
Main use	Bulk-data protection	Key establishment and signatures
Distribution challenge	Secret must be shared securely	Public key must be authenticated
Examples	AES and ChaCha20	RSA, Diffie–Hellman, and ECC

Modern systems normally combine both approaches.

6.17 Private-Key Protection

Public keys may be distributed, but private keys require strong protection.

Controls may include:

- Cryptographically secure key generation.
- Hardware security modules or trusted platform modules.
- Smart cards or protected device key stores.
- Strong access control and administrative separation.
- Backup only when justified and protected.
- Rotation, certificate renewal, and revocation procedures.
- Audit logging of sensitive operations.
- Secure destruction of retired keys.

The complete certificate lifecycle is addressed in Chapter 8. This section emphasizes that a strong algorithm cannot protect an exposed private key.

6.18 Side-Channel and Fault Attacks

Public key operations may expose implementation information through:

- Execution time.
- Cache and memory-access patterns.
- Power consumption.
- Electromagnetic emissions.
- Faulty computations induced by an attacker.
- Different error messages.

Defenses include constant-time implementations, RSA blinding, protected hardware, input validation, uniform error handling, and verification of sensitive computed results.

6.19 Common Attacks and Mistakes

- Using small keys or outdated parameters.
- Generating private keys with predictable randomness.
- Failing to authenticate a public key.
- Using textbook RSA without secure encoding.
- Encrypting large files directly with RSA.
- Using unauthenticated Diffie–Hellman.
- Failing to validate Diffie–Hellman public values.
- Reusing an ElGamal ephemeral value.
- Reusing or predicting an ECDSA per-signature value.
- Accepting invalid elliptic curve points.
- Using one key pair for unrelated purposes.
- Storing private keys in source code or public repositories.
- Ignoring side-channel and fault attacks.
- Failing to revoke credentials after private-key compromise.
- Designing custom cryptographic protocols.

6.20 Case Study: Authenticated Secure Chat

A secure chat application may use the following design:

1. Each user has a long-term identity key pair.
2. Public identity keys are authenticated through a trusted directory, certificate, or verified fingerprint.
3. The users perform an authenticated ephemeral key agreement.
4. A KDF derives separate session keys for each communication direction.
5. Messages are protected using authenticated symmetric encryption.
6. Nonces are managed so that they never repeat under a session key.
7. Session keys are rotated and temporary private values are erased.
8. Compromised identity keys are revoked or replaced.

This design demonstrates how identity, public key agreement, key derivation, and symmetric authenticated encryption work together.

6.21 Practical Group Activity

Hybrid Encryption and Key Agreement Design

Group Size: 3 to 5 students
Total Marks: 20 marks
Design a system for one of the following:

- Secure chat.
- Protected file transfer.
- A university portal session.
- Encrypted cloud backup sharing.

The design must include:

1. The required long-term and temporary keys.
2. A method for authenticating public keys.
3. A public key encryption or authenticated key-agreement method.
4. A KDF and key-separation plan.
5. A symmetric authenticated-encryption method.
6. Private-key protection.
7. Response to key compromise.
8. Mitigation of man-in-the-middle and replay attacks.

Required Submission

- A labeled protocol diagram.
- A step-by-step description.
- A list of protected assets.
- At least five possible attacks.
- A security control for every attack.

Marking Scheme

Assessment Item	Marks
Correct public-key operation and key roles	4
Public-key authentication and MITM protection	4
Correct KDF and key separation	3
Appropriate authenticated symmetric encryption	3
Private-key protection and compromise response	3
Attack analysis and controls	2
Diagram and presentation	1

6.22 Chapter Summary

This chapter introduced public key cryptography and the different roles of public and private keys. Public keys may be distributed, but their authenticity must be established. Private keys must remain protected.

RSA uses modular arithmetic with a modulus formed from two primes. The educational example demonstrated key generation, encryption, decryption, and repeated squaring. Real RSA requires approved encoding such as OAEP for encryption or PSS for signatures.

Diffie–Hellman allows two parties to calculate the same shared secret without transmitting it directly. The protocol must be authenticated to prevent man-in-the-middle attacks. Ephemeral key agreement can provide forward secrecy, and a KDF converts the shared value into separated session keys.

ElGamal is a randomized encryption system based on discrete logarithms. Fresh randomness is essential for every encryption. Elliptic curve cryptography uses point groups over finite fields and supports ECDH key agreement and ECDSA signatures with comparatively small keys.

Modern applications use hybrid encryption. Public key techniques establish identity or session keys, while symmetric authenticated encryption protects application data efficiently. Secure implementations require public-key authentication, private-key protection, input validation, secure randomness, trusted libraries, and resistance to side-channel attacks.

6.23 Additional Exercises

1. Explain why a public key must be authentic.
2. Compare public key encryption, key agreement, and digital signatures.
3. List the main steps of RSA key generation.
4. Verify that 27 is the inverse of 3 modulo 40.
5. Recalculate the RSA example using message $M = 8$.
6. Explain why textbook RSA is deterministic.
7. Explain the purpose of RSA-OAEP.
8. Compute Alice's public Diffie–Hellman value for $p = 23$, $g = 5$, and $a = 4$.
9. Explain why both Diffie–Hellman parties obtain the same shared value.
10. Draw a man-in-the-middle attack against unauthenticated Diffie–Hellman.
11. Explain how digital signatures authenticate Diffie–Hellman values.
12. Explain forward secrecy in your own words.
13. Explain why a KDF is used after key agreement.
14. Derive two different conceptual keys from one shared secret and explain why separation is useful.
15. Recalculate the ElGamal example using a different valid ephemeral value.
16. Explain the effect of reusing ElGamal randomness.
17. Explain scalar multiplication in elliptic curve cryptography.
18. Compare ECDH and ECDSA.
19. Explain why invalid elliptic curve points must be rejected.
20. Design a hybrid encryption process for a large university backup file.

6.24 Review Questions

1. What is public key cryptography?
2. Why is it called asymmetric cryptography?
3. What is the difference between a public key and a private key?
4. Why must public keys be authenticated?
5. What are the three main categories of public key operation?
6. How does public key encryption provide confidentiality?
7. What is key agreement?
8. How are private and public keys used in digital signatures?
9. What mathematical problem supports RSA security?
10. What is the RSA modulus?
11. How is Euler's phi function used in educational RSA key generation?
12. Why must e be relatively prime to $\phi(n)$?

13. How is the RSA private exponent calculated?
14. Why is textbook RSA unsafe?
15. What is OAEP?
16. Why should RSA not encrypt large files directly?
17. What does Diffie–Hellman establish?
18. Which Diffie–Hellman values are public?
19. Which Diffie–Hellman values remain private?
20. Why do both parties obtain the same shared value?
21. Why must Diffie–Hellman be authenticated?
22. What is a man-in-the-middle attack?
23. What is ephemeral Diffie–Hellman?
24. What is forward secrecy?
25. Why is a KDF used after Diffie–Hellman?
26. What is key separation?
27. What mathematical problem supports ElGamal?
28. Why does ElGamal produce two ciphertext values?
29. Why must ElGamal use fresh randomness?
30. What is elliptic curve cryptography?
31. What is scalar multiplication?
32. What is the elliptic curve discrete logarithm problem?
33. What is ECDH used for?
34. What is ECDSA used for?
35. Why can ECC use smaller keys than some traditional systems?
36. What is hybrid encryption?
37. Why is symmetric encryption used for bulk data?
38. What is a side-channel attack?
39. What is RSA blinding?
40. List five common public key implementation mistakes.

7. Cryptographic Hash Functions

Learning Objectives

By the end of this chapter, the reader should be able to:

- Define a cryptographic hash function and a message digest.
- Explain determinism, fixed-length output, efficiency, and the avalanche effect.
- Distinguish preimage resistance, second-preimage resistance, and collision resistance.
- Explain the birthday effect conceptually.
- Describe how hash functions process messages at a high level.
- Compare MD5, SHA-1, SHA-2, and SHA-3 conceptually.
- Distinguish hashing from encoding and encryption.
- Explain message authentication codes and HMAC.
- Explain the purpose of a key-derivation function and HKDF.
- Describe secure password storage using salts, work factors, and password-hashing algorithms.
- Compare PBKDF2, bcrypt, scrypt, and Argon2id conceptually.
- Identify common mistakes in hashing, MAC verification, and password storage.
- Perform a practical file-integrity and avalanche-effect experiment.

7.1 Introduction

Modern systems process large amounts of digital information. Files are downloaded, messages are transmitted, passwords are verified, software is installed, and electronic documents are signed. In many of these situations, a system needs a short value that represents the contents of much larger data.

A cryptographic hash function creates this short representation. It accepts input of almost any practical length and produces a fixed-length output called a hash value, message digest, or digital fingerprint.

Hash functions normally do not use a secret key. Anyone with the same input and algorithm can calculate the same digest. This makes hashing useful for integrity checking, but a plain hash does not prove who created the data.

7.2 What Is a Cryptographic Hash Function?

Cryptographic Hash Function

A cryptographic hash function maps input data of arbitrary practical length to a fixed-length digest. A secure cryptographic hash function is designed to provide preimage resistance, second-preimage resistance, and collision resistance.

The process can be represented as:

$$H(M) = d$$

where:

- H is the hash function.
- M is the input message.
- d is the resulting message digest.

For example, SHA-256 always produces a 256-bit digest, whether the input is one word or a large file.

Fixed-Length Output

Consider the inputs:

$$M_1 = \texttt{Hello}$$

and:

$$M_2 = \texttt{This is a much longer message used to demonstrate hashing.}$$

When SHA-256 processes either input, the result is exactly 256 bits. The digest length is determined by the algorithm, not by the input length.

7.3 Basic Characteristics

Deterministic Output

The same input processed by the same algorithm must always produce the same digest.

$$H(M) = d$$

If M does not change, d should not change.

Fixed-Length Output

Every hash algorithm has a defined output length. SHA-256 produces 256 bits, while SHA-512 produces 512 bits.

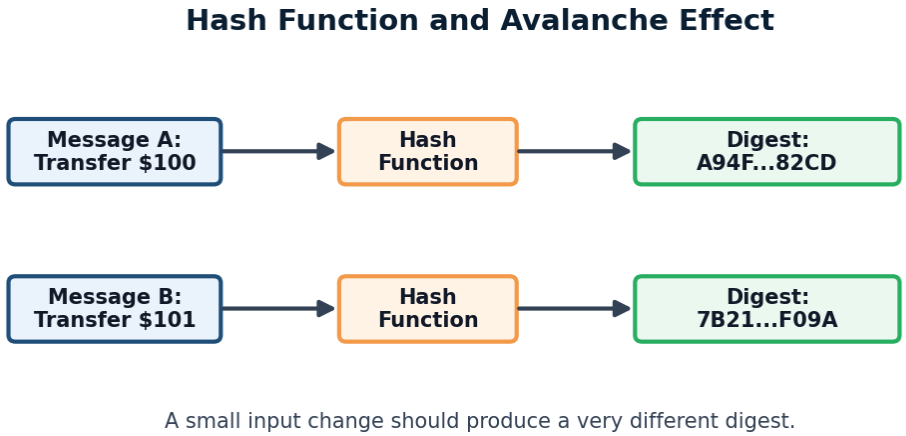


Figure 7.1: A small input change should produce a substantially different hash digest.

Efficient Computation

A normal user or system should be able to calculate a digest efficiently, even for a large file.

Input Sensitivity

A small input change should produce a substantially different digest. This is called the avalanche effect.

One-Way Design

Given a digest, finding a suitable input that produces it should be computationally infeasible. However, low-entropy inputs such as weak passwords may still be recovered by guessing possible values and comparing their digests.

7.4 How Hash Functions Process Messages

Hash families use different internal designs, but a simplified process is:

1. Accept the input message.
2. Add algorithm-defined padding.
3. Divide the padded message into fixed-size blocks.
4. Initialize an internal state.
5. Process each block using a compression or permutation function.
6. Produce the final digest from the resulting state.

A change in one part of the input should affect the final digest substantially.

R **Important Point.** SHA-2 uses an iterative compression-based design, while SHA-3 uses a sponge construction. Students do not need to implement either design manually.

7.5 Important Security Properties

Preimage Resistance

Given a digest d, it should be computationally infeasible to find an input M such that:

$$H(M) = d$$

Preimage Resistance

Preimage resistance means that, given a digest, finding an input that produces that digest should be computationally infeasible.

Second-Preimage Resistance

Given a specific input M_1, it should be computationally infeasible to find a different input M_2 such that:

$$H(M_1) = H(M_2)$$

where:

$$M_1 \neq M_2$$

Second-Preimage Resistance

Second-preimage resistance means that, given one specific input, finding another input with the same digest should be computationally infeasible.

Collision Resistance

A collision occurs when two different inputs produce the same digest:

$$H(M_1) = H(M_2), \qquad M_1 \neq M_2$$

Collision Resistance

Collision resistance means that finding any two different inputs with the same digest should be computationally infeasible.

Collisions must exist mathematically because the number of possible inputs is much larger than the number of fixed-length outputs. Security requires collisions to be impractical to find, not impossible to exist.

Property Comparison

Property	Attacker's Goal
Preimage resistance	Given a digest, find an input that produces it.
Second-preimage resistance	Given one input, find a different input with the same digest.
Collision resistance	Find any two different inputs with the same digest.

7.6 The Avalanche Effect

The avalanche effect means that a small change in the input should produce a substantially different output.

For example:

$$M_1 = \texttt{Security}$$

$$M_2 = \texttt{security}$$

The inputs differ only in the first letter, but their secure hash digests should appear unrelated.

Important Point. The avalanche effect is expected from a secure hash function, but it does not by itself prove that the algorithm is secure.

7.7 Digest Length and the Birthday Effect

An n-bit digest has:

$$2^n$$

possible values. A generic preimage search may require work related to 2^n, while a generic collision search may require work related to approximately:

$$2^{n/2}$$

because of the birthday effect.

For a 256-bit digest, generic collision resistance is therefore associated with approximately 128 bits of work, assuming no structural weakness exists.

7.8 Common Uses of Hash Functions

Cryptographic hashes support:

- File-integrity checking.
- Software-download verification.
- Digital signatures.
- Message authentication constructions.
- Password verification inside dedicated password-hashing schemes.
- Certificate and public-key fingerprints.
- Version control and content-addressed storage.
- Blockchain and distributed-ledger structures.
- Standardized key-derivation constructions.

File Integrity

A system may calculate and store a trusted digest for a file. Later, the file is hashed again. If the new digest differs from the authentic expected digest, the file has changed or the wrong file was obtained.

A digest difference does not reveal whether the change was accidental, authorized, or malicious.

Software Verification

A publisher may provide a digest for an installer. The user calculates the local digest and compares it with the expected value.

The expected digest must come from a trusted source. If an attacker replaces both the file and the displayed digest, a plain comparison may not detect the attack. A digital signature provides stronger publisher authentication.

Digital Signatures

Digital signature systems normally sign a digest representing the message rather than processing a large document directly. The complete signature process is explained in Chapter 8.

7.9 Historical and Modern Hash Families

MD5

MD5 produces a 128-bit digest. It is historically important but should not be used when collision resistance or modern cryptographic integrity is required.

SHA-1

SHA-1 produces a 160-bit digest. It is a legacy algorithm and should not be selected for new collision-sensitive applications such as digital signatures.

SHA-2

The SHA-2 family includes:

- SHA-224.
- SHA-256.
- SHA-384.
- SHA-512.
- SHA-512/224.
- SHA-512/256.

SHA-256 and SHA-512 are widely used in integrity checking, signatures, certificates, protocols, and message-authentication constructions.

SHA-3

SHA-3 is based on a different internal design from SHA-2. The family includes:

- SHA3-224.
- SHA3-256.
- SHA3-384.
- SHA3-512.

SHAKE128 and SHAKE256 are related extendable-output functions that can produce outputs of selected lengths.

The SHA-3 Sponge Idea

SHA-3 uses a sponge construction with two conceptual phases:

- **Absorbing:** message blocks are combined with the internal state.
- **Squeezing:** output bits are produced from the state.

The internal mathematics is handled by trusted implementations. Students only need to understand that SHA-3 uses a different construction from SHA-2.

Conceptual Comparison

Family	Digest Size	General Status
MD5	128 bits	Legacy and unsuitable for collision-sensitive security uses
SHA-1	160 bits	Legacy and unsuitable for new collision-sensitive uses
SHA-2	224–512 bits	Widely used modern family
SHA-3	224–512 bits	Modern family based on a different construction

7.10 Hashing, Encoding, and Encryption

These operations have different purposes.

- **Encoding** changes representation and is reversible without a secret. Examples include Base64 and hexadecimal.
- **Encryption** protects confidentiality and is reversible with the correct key.
- **Hashing** produces a fixed-length digest and is designed to support integrity and fingerprinting.

Operation	Main Purpose	Reversal	Secret Key
Encoding	Representation	Directly reversible	No
Encryption	Confidentiality	Reversible with key	Yes
Hashing	Integrity and fingerprinting	Designed to be one-way	Usually no

Base64 Is Not Encryption

Base64 changes binary data into a text representation. Anyone can reverse it without a secret key. It therefore provides neither confidentiality nor secure password storage.

7.11 Message Authentication Codes

A plain hash does not prove that its creator knew a secret. A message authentication code uses a shared secret key to support integrity and data-origin authentication.

Message Authentication Code

A message authentication code is a keyed cryptographic value used to verify message integrity and knowledge of a shared secret.

The sender calculates a MAC using the message and secret key. The receiver recalculates it using the same key. If the values match, the message was likely produced by a party that knew the key and was not modified.

A MAC does not provide public verification because both parties know the same secret.

7.12 HMAC

HMAC is a standardized MAC construction based on a cryptographic hash function and a secret key.

HMAC

HMAC combines a secret key with a cryptographic hash function to provide message integrity and data-origin authentication.

Conceptually:

$$\mathrm{HMAC}(K,M) = H\left((K' \oplus \mathrm{opad}) \,\|\, H\left((K' \oplus \mathrm{ipad}) \,\|\, M\right)\right)$$

where $\|$ represents concatenation.

Students should use a trusted HMAC library rather than implementing this formula manually.

HMAC API Scenario

An API client and server share a secret key. The client calculates an HMAC over the request path, timestamp, and body. The server recalculates the HMAC and also checks that the timestamp is recent.

This provides message integrity and evidence that the sender knew the shared key. The timestamp helps reduce replay risk.

7.13 Why Custom Keyed Hashes Are Dangerous

Applications should not invent constructions such as:

$$H(K \,\|\, M)$$

or:

$$H(M \,\|\, K)$$

Some hash constructions have properties, including length extension, that may make simple custom designs unsafe.

Important Point. A length-extension attack does not reverse a hash. It exploits how some iterative hash constructions continue processing. Use HMAC instead of inventing a keyed-hash formula.

7.14 Safe Verification of MAC Values

MACs and authentication values should be compared using a constant-time comparison function from a trusted library.

A normal string comparison may stop at the first different byte. Small timing differences can reveal information about the expected value.

Applications should:

- Use a library-provided constant-time comparison.
- Verify the expected tag length.
- Reject invalid tags before processing unauthenticated data.
- Avoid detailed error messages that reveal verification behavior.

7.15 Hash-Based Key Derivation

A key-derivation function converts input key material into one or more cryptographic keys.

HKDF is an HMAC-based KDF with two conceptual stages:

- **Extract:** processes input key material into a strong internal key.
- **Expand:** derives one or more output keys using context information.

Context may identify the protocol, session, communication direction, algorithm, or purpose of a derived key.

R **Important Point.** HKDF is not a password-hashing algorithm. Passwords have low entropy and require a dedicated password-hashing scheme.

7.16 Password Hashing

Passwords should not be stored in plaintext. If attackers steal a plaintext password database, the passwords are immediately exposed.

Passwords should also not be protected using only a fast general-purpose hash such as SHA-256. Fast hashes allow attackers to test large numbers of guesses quickly.

A password-storage system should use a purpose-built password-hashing or password-based key-derivation algorithm that is intentionally expensive to compute.

Secure Password-Storage Workflow

1. The user creates a password.
2. The system generates a unique random salt.
3. The system processes the password and salt using a password-hashing algorithm.
4. The system stores the algorithm name, parameters, salt, and password verifier.
5. During login, the system repeats the operation using the submitted password.
6. The new result is compared safely with the stored verifier.

R **Important Point.** The stored output is a password verifier. The system verifies a submitted password without recovering the original password.

7.17 Salts

Salt

A salt is a unique random value combined with a password before password hashing so that identical passwords do not automatically produce identical stored verifiers.

Salts provide these benefits:

- Equal passwords receive different stored values when salts differ.
- Precomputed rainbow tables become far less useful.
- Attackers must attack password records individually.

The salt is not normally secret and may be stored beside the password verifier.

7.18 Peppers

A pepper is an additional secret value used with password hashing. Unlike a salt, it should be stored separately from the password database, such as in protected application secrets storage or a hardware-backed key service.

A pepper provides defense in depth, but it does not replace a unique salt or a secure password-hashing algorithm.

If a pepper is compromised, rotation may require users to authenticate again or reset passwords because the original passwords are not stored.

7.19 Work Factors and Memory Cost

Password hashing deliberately increases the cost of each guess.

A work factor controls computation time. Memory-hard algorithms also require significant memory, making large parallel attacks more expensive.

The chosen parameters should be strong enough to slow attacks while keeping legitimate login verification acceptable. Parameters must be reviewed as hardware improves.

7.20 Password-Hashing Algorithms

PBKDF2

PBKDF2 repeatedly applies a pseudorandom function, commonly HMAC. It is widely supported and may be required in some standards-driven environments.

bcrypt

bcrypt is an adaptive password-hashing scheme with a configurable cost. It remains common in existing systems but has input-length and older-design limitations that implementations must understand.

scrypt

scrypt requires both computation and memory. Its memory cost makes large parallel attacks more expensive than fast general-purpose hashing.

Argon2id

Argon2id is a modern password-hashing option that combines memory cost and computation. It is generally preferred when supported and configured appropriately.

Algorithm	Main Cost	General Note
PBKDF2	Repeated computation	Broad support and standards compatibility
bcrypt	Configurable computation	Common established option
scrypt	Computation and memory	Designed to increase memory cost
Argon2id	Computation and memory	Modern preferred option when supported

7.21 Upgrading Password Verifiers

Password-hashing parameters should be reviewed as hardware improves. A system can upgrade an older password record after a successful login:

1. Verify the password using the stored algorithm and parameters.
2. Determine whether the record uses outdated settings.
3. Generate a new salt.
4. Rehash the password using the current algorithm and parameters.

5. Replace the old verifier.

This process is sometimes called opportunistic rehashing.

7.22 Rainbow Tables

A rainbow table contains precomputed information for many password candidates. It can accelerate attacks against unsalted password hashes.

Unique salts prevent one precomputed table from working efficiently across many users because each salt changes the password-hashing input.

7.23 Online and Offline Password Attacks

Online Attacks

An online attacker sends guesses to the real login service. Defenses include:

- Rate limiting.
- Multi-factor authentication.
- Monitoring and alerting.
- Secure account recovery.
- Detection of automated abuse.

Offline Attacks

An offline attack occurs after password verifiers are stolen. The attacker can test guesses without contacting the legitimate service.

Unique salts, strong password-hashing algorithms, sufficient memory and work factors, protected peppers, and strong user passwords are especially important against offline attacks.

7.24 Digest and Tag Truncation

Some protocols use shortened digests or authentication tags. Truncation reduces storage or transmission size but also reduces the security margin.

Applications should not shorten a digest or tag unless the relevant protocol or standard defines the permitted length and method.

7.25 Common Mistakes

- Storing plaintext passwords.
- Encrypting passwords when normal verification does not require recovery.
- Storing unsalted password hashes.
- Using SHA-256 or another fast general hash directly for password storage.
- Using MD5 or SHA-1 for new collision-sensitive security applications.
- Assuming hashing provides confidentiality.
- Treating Base64 as encryption.
- Trusting a downloaded file digest from an unauthenticated source.
- Creating a custom MAC instead of using HMAC.
- Comparing MACs or tags with unsafe string comparison.
- Reusing an HMAC key for unrelated purposes.
- Using HKDF as a password-hashing replacement.
- Failing to review password-hashing parameters over time.
- Truncating authentication values without protocol guidance.

7.26 Practical Activity

Hash and Avalanche Experiment

Group Size: 3 to 4 students
Total Marks: 15 marks
Complete the following tasks:

1. Create a short text file containing at least two sentences.
2. Compute the SHA-256 digest of the original file.
3. Record the digest exactly.
4. Change one character in the file.
5. Compute the SHA-256 digest again.
6. Compare the two digests.
7. Restore the original file and confirm that the original digest returns.
8. Explain determinism, integrity checking, and the avalanche effect.
9. Explain why a matching digest alone does not authenticate a publisher.

Required Submission

- Group-member names.
- Original and modified file contents.
- Both SHA-256 digests.
- A short analysis of the results.
- An explanation of why a digital signature provides stronger publisher authentication.

Marking Scheme

Assessment Item	Marks
Correct original-file hash	2
Correct modified-file hash	2
Clear comparison of both digests	2
Explanation of determinism	2
Explanation of the avalanche effect	2
Explanation of integrity checking	2
Explanation of trusted authentication	2
Teamwork and presentation	1

7.27 Chapter Summary

This chapter introduced cryptographic hash functions. A hash function processes input data and produces a fixed-length digest. Secure hash functions are designed to provide preimage resistance, second-preimage resistance, collision resistance, deterministic output, efficient computation, and strong input sensitivity.

The chapter compared MD5, SHA-1, SHA-2, and SHA-3. MD5 and SHA-1 are legacy algorithms unsuitable for new collision-sensitive applications. SHA-2 is widely used, while SHA-3 provides a modern family based on a sponge construction.

Hashing is different from encoding and encryption. Encoding changes representation, encryption provides confidentiality, and hashing supports integrity and fingerprinting.

A plain hash does not authenticate its creator. MACs and HMAC use a shared secret key to provide integrity and data-origin authentication. HKDF uses HMAC to derive separated cryptographic keys from input key material.

Passwords should be stored using dedicated password-hashing algorithms with unique salts and suitable cost parameters. PBKDF2, bcrypt, scrypt, and Argon2id are password-hashing options with different properties. Password verifiers should be upgraded as security requirements and hardware capabilities change.

7.28 Additional Exercises

1. Explain why a hash does not provide confidentiality.
2. Compare preimage and second-preimage resistance.
3. Explain why collision resistance matters for digital signatures.
4. Explain the birthday effect conceptually.
5. Describe the high-level stages used to process a message.
6. Compare SHA-2 and SHA-3 conceptually.
7. Explain why Base64 is not encryption.
8. Explain what HMAC adds to a plain hash.
9. Explain why custom keyed-hash formulas should be avoided.
10. Explain why MAC values should be compared in constant time.
11. Explain the Extract and Expand stages of HKDF.
12. Explain why HKDF is not suitable for password hashing.
13. Compare salts and peppers.
14. Explain why password-hashing algorithms should be slow.
15. Compare PBKDF2, bcrypt, scrypt, and Argon2id conceptually.
16. Explain opportunistic password rehashing.
17. Compare online and offline password attacks.
18. Explain why a published file digest must come from a trusted source.

7.29 Review Questions

1. What is a cryptographic hash function?
2. What is a message digest?
3. What does deterministic mean?
4. Why is hash output fixed in length?
5. What is preimage resistance?
6. What is second-preimage resistance?
7. What is collision resistance?
8. Why must hash collisions exist mathematically?
9. What is the avalanche effect?
10. What is the birthday effect?
11. How do hash functions process long messages conceptually?
12. Why are MD5 and SHA-1 unsuitable for new collision-sensitive applications?
13. Which algorithms belong to SHA-2?
14. Which algorithms belong to SHA-3?
15. What are SHAKE128 and SHAKE256?
16. What are the absorbing and squeezing phases?
17. How is hashing different from encryption?
18. How is encoding different from hashing?
19. What is a message authentication code?

20. What is HMAC?
21. Why does HMAC require a secret key?
22. Why should custom keyed-hash formulas be avoided?
23. Why should MACs be compared in constant time?
24. What is a key-derivation function?
25. What is HKDF?
26. Why is HKDF not a password-hashing algorithm?
27. Why should passwords not be stored in plaintext?
28. Why is fast SHA-256 alone unsuitable for password storage?
29. What is a password verifier?
30. What is a salt?
31. Why should each password record have a unique salt?
32. What is a pepper?
33. What is a work factor?
34. What is a memory-hard password-hashing algorithm?
35. What are PBKDF2, bcrypt, scrypt, and Argon2id?
36. What is opportunistic rehashing?
37. What is a rainbow table?
38. What is the difference between online and offline password attacks?
39. Why can digest or tag truncation reduce security?
40. List five common hash or password-storage mistakes.

8. Digital Signatures and Certificates

Learning Objectives

By the end of this chapter, the reader should be able to:

- Define a digital signature and explain its purpose.
- Distinguish a digital signature from encryption and a handwritten signature.
- Explain how hashing, private keys, and public keys support signing and verification.
- Describe RSA, DSA, ECDSA, and EdDSA signatures conceptually.
- Explain why private signing keys must be protected.
- Define a digital certificate and identify its main fields and extensions.
- Explain certificate signing requests and certificate fingerprints.
- Distinguish root, intermediate, and end-entity certificates.
- Explain public key infrastructure, certificate chains, trust stores, and trust anchors.
- Describe certificate and path validation.
- Explain certificate expiration, revocation, CRLs, OCSP, and OCSP stapling.
- Explain how certificates support HTTPS and software signing.
- Recognize common attacks and implementation mistakes.
- Inspect and analyze a website certificate using a browser.

8.1 Introduction

Digital communication creates an important security problem. When a message, document, software update, or electronic transaction is received, the receiver may need to determine who approved the data, whether the data was modified, and whether the public key belongs to the claimed person, organization, website, device, or service.

A digital signature provides evidence that the holder of a private key signed particular data and that the signed data has not changed since the signature was created.

A digital signature is not a scanned image of a handwritten signature. It is a cryptographic value

calculated using the signed data, a hash function, a signature algorithm, and a private signing key.

Digital Signature

A digital signature is a cryptographic value created using a private signing key and verified using the corresponding public key.

A digital certificate binds a public key to information about a subject according to a certificate authority's validation process and policy.

Digital signatures and certificates are used in secure websites, software updates, electronic documents, email security, financial transactions, cloud services, mobile applications, and government systems.

8.2 Why Digital Signatures Are Needed

Encryption mainly protects confidentiality. Encryption alone does not necessarily prove who approved a message or whether the message was changed.

Digital signatures support:

- **Integrity:** unauthorized changes to signed data can be detected.
- **Origin authentication:** the verifier obtains evidence that the corresponding private key created the signature.
- **Non-repudiation support:** the signature may contribute evidence that the private-key holder approved particular data.

Important Point. A valid signature proves that a particular private key was used. Identity validation, certificate policy, private-key protection, timestamps, audit records, and legal processes connect that key to a person or organization.

8.3 Digital and Handwritten Signatures

A handwritten signature is normally similar across documents. A digital signature depends on the exact signed data. If one character changes, the message digest changes and the original signature should fail verification.

Feature	Handwritten Signature	Digital Signature
Form	Written or drawn mark	Cryptographic bit string
Connection to content	Visually attached	Mathematically connected
Verification	Human or forensic comparison	Cryptographic algorithm
Change detection	Usually limited	Detects changes to signed data
Key requirement	No cryptographic key	Public and private key pair
Reuse	Similar mark may be reused	Depends on the exact data

8.4 Digital Signature Security Services

Integrity

During verification, the system processes the received data and checks whether the signature is valid for that exact data. A changed document should fail verification.

Origin Authentication

A valid signature provides evidence that the corresponding private key created the signature. The verifier must also establish that the public key belongs to the expected signer.

Non-Repudiation Support

A signature may support evidence that a signer approved data. Non-repudiation depends on secure private-key control, identity verification, timestamps, records, policies, and applicable law.

8.5 Components of a Signature System

A digital signature system normally requires:

- A message, file, document, or transaction.
- An approved cryptographic hash function.
- A signature algorithm and encoding scheme.
- A private signing key.
- A public verification key.
- A trusted method of connecting the public key to the signer.
- Secure key generation and private-key storage.
- A verification policy defining acceptable algorithms and certificates.

8.6 How Digital Signatures Work

The process has three stages: key-pair generation, signing, and verification.

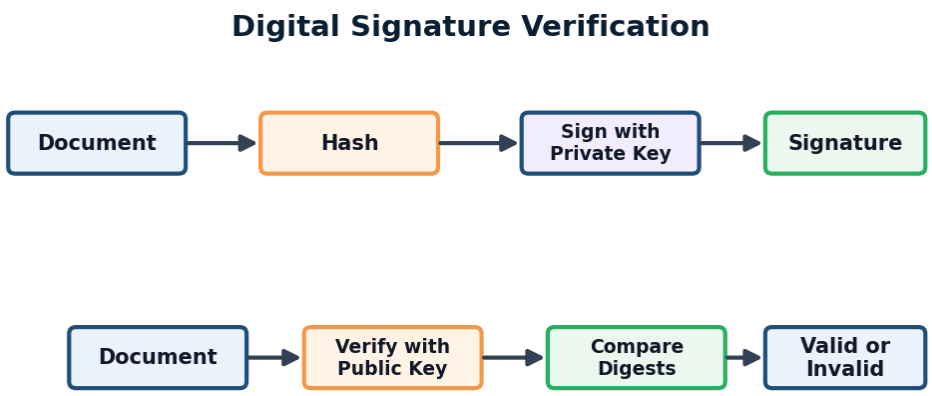


Figure 8.1: The signing and verification process used by a digital signature system.

Key-Pair Generation

The signer generates a public and private signing-key pair. The private key must remain protected, while the public key may be distributed to verifiers through an authenticated method.

Signing

A typical signing process is:

1. Prepare the message or file.
2. Calculate a cryptographic digest of the message.
3. Encode the digest according to the signature scheme.
4. Create the signature using the private signing key.
5. Send the message, signature, and required certificate information.

Conceptually:

$$\text{Message} \xrightarrow{\text{Hash Function}} \text{Digest}$$

$$\text{Digest} + \text{Private Signing Key} \xrightarrow{\text{Signature Algorithm}} \text{Digital Signature}$$

Verification

A verifier:

1. Receives the message and signature.
2. Obtains the signer's public key from a trusted source or certificate.
3. Uses the signature algorithm to verify the signature for the received message.
4. Validates the certificate path and intended key usage when certificates are used.
5. Accepts the signature only if every required check succeeds.

Changed Messages

Suppose the signed message is:

```
Transfer 1000 EGP
```

If it changes to:

```
Transfer 9000 EGP
```

the modified message produces a different digest and the original signature should fail verification.

R **Important Point.** A digital signature does not prevent modification. It enables unauthorized modification to be detected.

8.7 Worked Signature Scenario

Signing a Grade Report

An instructor signs the document:

```
Student ID 2025001:  Grade A
```

The system calculates a digest and creates a signature using the instructor's private signing key. The document, signature, and certificate are sent to the receiver.
The receiver verifies the signature using the public key in the certificate and validates the certificate path. Verification succeeds only if the signature is valid for the received document and all required certificate checks succeed.
If the grade changes from `A` to `B`, the signature should fail verification.

8.8 Hash Functions in Signatures

Signature systems normally hash the message before signing. Hashing provides a fixed-length representation and allows large files to be processed efficiently.

The hash function must be approved for the signature scheme and provide an appropriate security level. Chapter 7 explains preimage resistance, collision resistance, and hash families in detail.

8.9 RSA Signatures

RSA can create digital signatures. A simplified process is:

1. Hash the message.
2. Encode the digest using a secure RSA signature scheme.
3. Create the signature using the RSA private key.
4. Verify the signature using the RSA public key.

RSA-PSS

RSA-PSS is a modern randomized encoding scheme for RSA signatures. It combines the digest with structured formatting and a random salt before the private-key operation.

The RSA-PSS salt is part of the signature encoding. It is not the same as a password salt.

Important Point. RSA signing should not be described simply as encrypting with the private key. RSA encryption and RSA signatures use different encoding schemes, security goals, and verification procedures.

Applications should use trusted cryptographic libraries and should not implement custom RSA padding or signature encoding.

8.10 DSA, ECDSA, and EdDSA

DSA is based on the discrete logarithm problem. ECDSA uses elliptic curve groups and supports smaller keys than many traditional RSA deployments.

EdDSA uses Edwards-form elliptic curves. Common examples include Ed25519 and Ed448. EdDSA is designed for efficient signatures and, when implemented according to its standard, does not depend on generating a new random value for every signature.

Algorithm	Foundation	General Observation
RSA	Integer factorization	Widely supported; secure encoding is required
DSA	Discrete logarithms	Historically important signature standard
ECDSA	Elliptic curves	Strong security with comparatively small keys
EdDSA	Edwards curves	Efficient modern signature approach

R **Important Point.** DSA and ECDSA require a secure value for every signature. Reusing or predicting that value can expose the private key. Trusted libraries should manage this process.

8.11 Signatures and Encryption

Digital signatures and encryption have different purposes:

- Encryption protects confidentiality.
- Signatures support integrity and origin authentication.
- Encryption normally uses a key associated with the intended receiver.
- Signing uses the signer's private signing key.
- A message may be both signed and encrypted.

Signed and Encrypted Message

Alice may sign a message using Alice's private signing key and then encrypt the protected message for Bob. Bob decrypts the message and verifies Alice's signature using Alice's authentic public key.

8.12 Private Signing-Key Protection

If attackers obtain a private signing key, they may create fraudulent signatures that appear to come from the legitimate owner.

Controls include:

- Secure key generation.
- Protected cryptographic storage.
- Hardware security modules or trusted device key stores.
- Strong authentication before signing.
- Restricted access and administrative separation.
- Audit logging of important signing operations.
- Certificate revocation after compromise.
- Secure replacement and destruction of retired keys.

R **Important Point.** A strong signature algorithm cannot protect an exposed private key.

8.13 Trusted Timestamping

A trusted timestamp may provide evidence that particular data or a signature existed at a certain time.

A timestamping authority signs information containing a data digest and a time value. Long-term verification may also require the original certificate path, revocation information, timestamp certificate, and accepted algorithms.

R **Important Point.** A date written inside a document is not automatically a trusted timestamp. A trusted timestamp must be cryptographically created and verified through an accepted service.

8.14 Digital Certificates

A public key alone does not establish its owner's identity. An attacker can create a key pair and falsely claim that the public key belongs to a university, bank, or website.

Digital Certificate

A digital certificate is a signed electronic document that binds a public key to information about a subject, such as a domain, organization, person, device, or service.

A certificate commonly contains:

- Version.
- Serial number.
- Subject information.
- Issuer information.
- Validity start and end dates.
- Subject public key and algorithm.
- Signature algorithm.
- Certificate extensions.
- Issuer's digital signature.

Important Certificate Fields

Subject Information about the entity represented by the certificate.
Issuer The certificate authority that issued and signed the certificate.
Serial Number A value assigned by the issuer to distinguish the certificate.
Validity Period The time interval during which the certificate is intended to be valid.
Public Key The subject's public key and related algorithm information.
Signature The issuer's signature over the certificate data.

8.15 Certificate Signing Requests

Certificate Signing Request

A certificate signing request, or CSR, is a signed request containing a public key and requested certificate information. The requester signs the CSR using the corresponding private key to demonstrate possession of that key.

The certificate authority still validates the requested identity, domain control, or organization information according to its policy before issuing a certificate.

8.16 Certificate Extensions

X.509 version 3 certificates can contain extensions that restrict or explain certificate use.

Common extensions include:

- **Basic Constraints:** indicates whether the certificate belongs to a certificate authority.
- **Key Usage:** identifies permitted cryptographic operations.
- **Extended Key Usage:** identifies purposes such as server authentication, client authentication, email protection, code signing, or timestamping.
- **Subject Alternative Name:** lists domain names or other identities associated with the certificate.
- **CRL Distribution Points:** identifies locations for certificate revocation lists.
- **Authority Information Access:** may identify issuer information or an OCSP service.

Subject Alternative Name

For website certificates, the verifier checks the requested domain against the identities in the Subject Alternative Name, or SAN, extension.

For example:

- `portal.example.edu`
- `www.example.edu`
- `library.example.edu`

Key Usage and Extended Key Usage

Key Usage defines permitted low-level operations. Extended Key Usage identifies intended application purposes, such as TLS server authentication, client authentication, code signing, email protection, or timestamping.

Critical Extensions

An extension may be marked critical. If a verifier does not recognize or cannot correctly process a critical extension, the certificate must not be accepted.

A noncritical extension may provide useful information without requiring every verifier to understand it.

8.17 Certificate Fingerprints

A certificate fingerprint is a hash digest calculated over the encoded certificate. Fingerprints can help users compare certificates through a separate trusted channel.

A fingerprint does not create trust by itself. The expected fingerprint must be obtained authentically, and a modern approved hash algorithm should be used.

8.18 Public Key Infrastructure

Public key infrastructure, or PKI, is the combination of technologies, people, policies, procedures, hardware, software, and services used to manage certificates and public-key trust.

PKI may include:

- Certificate authorities.
- Registration and identity-validation processes.

- Certificate repositories.
- Trust stores.
- Certificate policies.
- Revocation services.
- Auditing and operational controls.
- Key-generation, renewal, replacement, and destruction procedures.

8.19 Certificate Authorities

A certificate authority, or CA, validates certificate requests according to policy and signs issued certificates.

Root Certificate Authority

A root CA is at the top of a certificate hierarchy. A root certificate is usually self-signed and configured as a trust anchor in an operating system, browser, application, or organization.

The self-signature does not create trust. The root is trusted because it was securely installed or configured as a trust anchor.

Root private keys require very strong protection and should be used as little as possible.

Intermediate Certificate Authority

An intermediate CA receives a certificate from a root or another intermediate CA. It signs certificates below it in the hierarchy.

Intermediate CAs reduce frequent use of the root key and allow issuance to be separated by purpose, organization, or policy.

End-Entity Certificate

An end-entity certificate belongs to a website, person, device, application, or organization. It normally appears at the beginning of a certificate path and is not authorized to issue other certificates.

8.20 Certificate Chains

A certificate chain connects an end-entity certificate to a trusted root:

$$\text{End-Entity Certificate} \rightarrow \text{Intermediate CA} \rightarrow \text{Root CA}$$

Each certificate is verified using the issuer's public key until the path reaches an accepted trust anchor.

University Portal Chain

A certificate for `portal.example.edu` may be signed by an intermediate CA. The intermediate certificate may be signed by a trusted root CA. The browser verifies the path and applies all required constraints before trusting the portal certificate.

8.21 Trust Stores and Trust Anchors

A trust store contains trusted certificates or public keys. Root certificates in the trust store act as trust anchors.

Trust Anchor

A trust anchor is a trusted public key or certificate used as the starting point for certificate-path validation.

A certificate is not trusted merely because its signature is mathematically valid. Its path must lead to an accepted trust anchor, and every required validation check must succeed.

8.22 Certificate and Path Validation

Certificate validation examines the end-entity certificate and verifies a certification path to an accepted trust anchor.

A verifier should check:

1. Certificate signatures are valid.
2. The current time is within the required validity periods.
3. The requested domain or identity matches an appropriate SAN entry.
4. The path leads to an accepted trust anchor.
5. Basic Constraints permit each CA certificate to act as a CA.
6. Key Usage and Extended Key Usage permit the intended operation.
7. Path-length and other certificate constraints are satisfied.
8. Recognized critical extensions are processed correctly.
9. Certificate status is acceptable when revocation checking is required.
10. The algorithms and key sizes satisfy policy.

Website Certificate Validation

When a browser connects to:

```
https://portal.example.edu
```

the browser checks the domain name, validity period, certificate path, signature algorithms, intended server-authentication purpose, and other required policy information.

8.23 Certificate Revocation

A certificate may need to become invalid before its planned expiration date because of private-key compromise, incorrect issuance, ownership change, affiliation change, service termination, or certificate replacement.

Certificate Revocation Lists

A certificate revocation list, or CRL, is a signed list published by a CA. It identifies revoked certificates, normally using their serial numbers.

Online Certificate Status Protocol

OCSP allows a verifier to request status information from an OCSP responder. A response may indicate good, revoked, or unknown status.

OCSP Stapling

With OCSP stapling, a server obtains a signed status response and sends it to the client during connection establishment. The client validates the response's signature and freshness according to protocol and policy.

Important Point. Expiration occurs at the planned end of a certificate's validity period. Revocation invalidates a certificate before that date. Revocation behavior depends on the application and policy.

8.24 Certificate Lifecycle

A typical lifecycle is:

1. Generate a key pair.
2. Protect the private key.
3. Create a certificate signing request.
4. Validate identity, domain control, or other required information.
5. Issue and sign the certificate.
6. Deploy the certificate and related private key securely.
7. Monitor use, validity, and expiration.
8. Renew or replace the certificate before expiration.
9. Revoke it after compromise or incorrect issuance.
10. Archive or destroy retired key material according to policy.

8.25 Self-Signed Certificates

A self-signed certificate is signed using its own private key. Such certificates may be suitable for testing or controlled private trust systems.

A self-signed certificate is not automatically trusted. The verifier must obtain and trust it through a separate secure process.

Important Point. Self-signed does not automatically mean malicious, but it does not provide public trust by itself.

8.26 Certificate Transparency

Certificate Transparency provides public logs of many publicly trusted website certificates. These logs help domain owners and security researchers detect unexpected or incorrectly issued certificates.

Certificate Transparency does not replace normal validation. The browser must still validate the domain name, certificate path, validity period, intended use, signatures, and other required information.

8.27 Certificates in HTTPS

HTTPS uses TLS to protect communication between a client and server. The server normally presents a certificate chain during connection establishment.

The certificate helps the client authenticate the server. TLS then establishes session keys that protect the confidentiality and integrity of the connection.

If certificate validation fails, the browser may display a warning because the connection may not be securely connected to the expected server.

8.28 Code and Software Signing

Software publishers sign applications, installers, drivers, and updates. Before installation, a system can verify:

- The software was signed by the expected publisher.
- The software has not changed since signing.
- The certificate path and signing purpose are acceptable.
- Any required trusted timestamp and certificate status are valid.

8.29 Email and Document Signing

Digital signatures can protect email messages and electronic documents. Verification should include the signature value, certificate path, key usage, signing time or trusted timestamp when required, and applicable policy.

8.30 Case Study: Signed Software Update

Suppose a university publishes an update for an examination application.

A secure process may be:

1. Build the update package.
2. Calculate a digest of the package.
3. Sign the package using a protected code-signing private key.
4. Publish the package and required signature information.
5. Verify the signature and certificate policy before installation.
6. Reject the update if verification fails.

Modified Update

If an attacker changes one file inside the signed update package, the digest changes and the original signature should fail verification.

If the code-signing key is compromised, the organization should stop using it, revoke the related certificate when applicable, investigate unauthorized signing, issue a replacement key and certificate, and provide clear update guidance.

8.31 Common Attacks and Mistakes

- Storing private keys in plaintext.
- Sharing private signing keys.
- Using outdated signature or hash algorithms.
- Describing RSA signing as private-key encryption.
- Implementing custom RSA signature encoding.
- Reusing insecure DSA or ECDSA per-signature values.
- Failing to validate the complete certificate path.
- Ignoring SAN name mismatches.
- Ignoring certificate warnings.
- Accepting expired or revoked certificates without policy justification.
- Ignoring Key Usage, Extended Key Usage, or Basic Constraints.
- Ignoring unknown critical extensions.
- Trusting arbitrary self-signed certificates.
- Failing to revoke a certificate after key compromise.

- Failing to renew certificates before expiration.
- Treating a valid signature as proof that the signer acted willingly.

Important Point. Certificate pinning is not a general replacement for normal validation. Incorrect pinning can cause outages during certificate rotation.

8.32 Practical Activity

Certificate Inspection and Trust Analysis

Group Size: 3 to 5 students
Total Marks: 15 marks
Open a secure website and inspect its certificate using the browser's certificate viewer.
Record:

1. Website domain.
2. Certificate subject.
3. Subject Alternative Name entries.
4. Issuer.
5. Validity start and end dates.
6. Public key algorithm.
7. Public key size or curve name.
8. Signature algorithm.
9. Intermediate CA.
10. Root CA or trust anchor.
11. Key Usage and Extended Key Usage.
12. Certificate fingerprint.

Then answer:

1. Does the SAN match the website domain?
2. Is the certificate currently valid?
3. How many certificates are in the path?
4. Which certificate or key is the trust anchor?
5. What happens if the website certificate expires?
6. What should happen if the server's private key is stolen?

Marking Scheme

Assessment Item	Marks
Correct identification of certificate fields	4
Correct certificate-chain explanation	3
Correct validity and domain-name analysis	2
Public-key and signature algorithm explanation	2
Compromise and expiration analysis	2
Teamwork and presentation	2

8.33 Chapter Summary

This chapter introduced digital signatures, certificates, and public key infrastructure. A signature is created using a private signing key and verified using the corresponding public key. Signatures support integrity and origin authentication and may support non-repudiation evidence when combined with identity verification, private-key protection, timestamps, records, policy, and legal processes.

Messages are normally hashed before signing. RSA signatures require a dedicated scheme such as RSA-PSS. DSA and ECDSA require secure per-signature values, while EdDSA uses an Edwards-curve design with standardized deterministic processing.

A digital certificate binds a public key to subject information. Certificate authorities issue certificates, and certificate paths connect end-entity certificates through intermediate CAs to accepted trust anchors.

Certificate validation checks signatures, names, validity periods, Basic Constraints, Key Usage, Extended Key Usage, critical extensions, trust paths, algorithm policy, and certificate status. Revocation may use CRLs or OCSP, while OCSP stapling allows a server to provide a signed status response.

The chapter also explained CSRs, fingerprints, trusted timestamping, Certificate Transparency, HTTPS certificates, software signing, self-signed certificates, certificate lifecycles, and common implementation mistakes.

8.34 Additional Exercises

1. Draw the signing process.
2. Draw the verification process.
3. Explain why messages are normally hashed before signing.
4. Compare encryption and digital signatures.
5. Explain why RSA signing is not simply private-key encryption.
6. Explain the purpose of RSA-PSS.
7. Explain why DSA and ECDSA per-signature values must be secure.
8. Compare RSA, DSA, ECDSA, and EdDSA conceptually.
9. Explain why private signing-key compromise is serious.
10. Define a certificate signing request.
11. Explain what a certificate fingerprint provides.
12. Compare root, intermediate, and end-entity certificates.
13. Design a certificate path for a university portal.
14. Explain the purpose of SAN, Key Usage, and Extended Key Usage.
15. Explain how critical extensions affect validation.
16. Compare certificate expiration and revocation.
17. Compare CRLs, OCSP, and OCSP stapling.
18. Explain why a self-signed certificate is not automatically trusted.
19. Explain Certificate Transparency conceptually.
20. Describe the response to a compromised code-signing key.

8.35 Review Questions

1. What is a digital signature?
2. How is it different from a handwritten signature?
3. Which key creates a signature?
4. Which key verifies a signature?
5. Why is a hash used before signing?

6. What happens if signed data changes?
7. Which security services do signatures support?
8. Why should non-repudiation be described carefully?
9. What is RSA-PSS?
10. Why is RSA signing not simply private-key encryption?
11. What are DSA, ECDSA, and EdDSA?
12. Why are DSA and ECDSA per-signature values sensitive?
13. Why must private signing keys remain protected?
14. What is trusted timestamping?
15. What is a digital certificate?
16. What is a certificate signing request?
17. What is a certificate fingerprint?
18. What information appears in a certificate?
19. What is Subject Alternative Name?
20. What is Key Usage?
21. What is Extended Key Usage?
22. What is a critical extension?
23. What is a certificate authority?
24. What is a root CA?
25. What is an intermediate CA?
26. What is an end-entity certificate?
27. What is a certificate chain or path?
28. What is a trust anchor?
29. Which checks form part of certificate validation?
30. What is certificate revocation?
31. What is the difference between CRL and OCSP?
32. What is OCSP stapling?
33. What is a self-signed certificate?
34. What is Certificate Transparency?
35. How do certificates support HTTPS?
36. How do signatures protect software updates?
37. List five certificate-validation mistakes.
38. What should happen after a private signing key is compromised?

9. Authentication and Access Control

Learning Objectives

By the end of this chapter, the reader should be able to:

- Distinguish identification, authentication, authorization, and accountability.
- Explain common authentication factors and password-based authentication.
- Explain multi-factor authentication and compare common MFA methods.
- Describe biometric authentication, biometric templates, and common biometric errors.
- Compare DAC, MAC, RBAC, and ABAC.
- Apply least privilege, separation of duties, and periodic access review.
- Explain account provisioning, role change, disabling, and deletion.
- Explain single sign-on and the role of an identity provider.
- Distinguish OAuth authorization from OpenID Connect authentication.
- Distinguish access tokens, ID tokens, refresh tokens, and scopes.
- Explain secure session management, logging, monitoring, and accountability.
- Design access controls for a university portal.

9.1 Introduction

Authentication and access control are essential parts of system security. Authentication answers the question, “Who are you?” Authorization answers, “What are you allowed to do?” A secure system verifies identity and then applies the correct permissions.

For example, a university portal first authenticates a student. It then authorizes the student to view personal grades and course materials but denies permission to edit grades or manage user accounts.

An authenticated user may still be denied access when the required permission is missing. Authentication and authorization are related, but they are not the same.

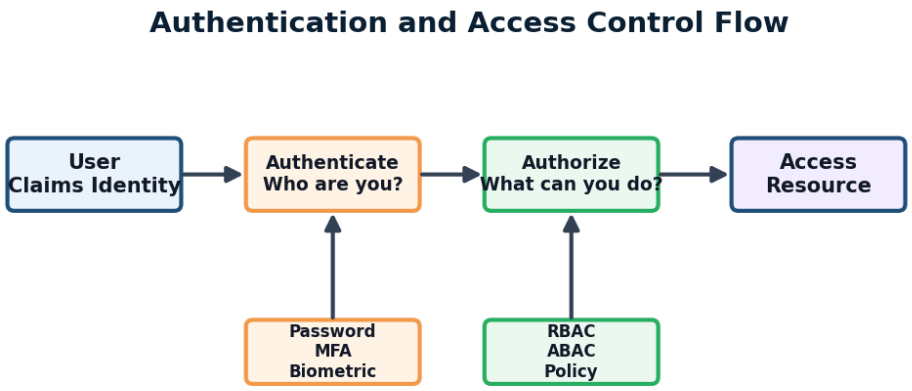


Figure 9.1: Authentication verifies identity, while authorization controls access.

9.2 Identification, Authentication, Authorization, and Accountability

Identification

Identification is the act of claiming an identity. A user may provide a username, email address, student ID, employee number, account number, or device identifier.

Identification

Identification is the process of claiming an identity to a system.

Entering `student123` tells the system which account is being requested. It does not prove that the person is the account owner.

Identifiers should be unique within their intended system and should avoid exposing unnecessary sensitive information.

Authentication

Authentication verifies a claimed identity.

Authentication

Authentication is the process of verifying the identity of a user, device, application, or service before granting access.

Authentication evidence may include a password, authenticator application, security key, smart card, certificate, or biometric sample.

Authorization

Authorization decides what an authenticated identity may access or perform.

Authorization

Authorization is the process of granting or denying access according to permissions, roles, attributes, and policy.

Common permissions include create, read, update, delete, approve, export, and administer.

Accountability

Accountability makes actions traceable.

Accountability

Accountability is the ability to trace an action or event to the responsible user, process, device, or service.

Accountability depends on unique accounts, reliable logs, accurate timestamps, protected audit records, and regular review.

The Complete Access Process

The four ideas work together:

1. Identification states which identity is claimed.
2. Authentication verifies the claim.
3. Authorization decides which actions are permitted.
4. Accountability records important actions and outcomes.

University Portal Example

A user enters a student ID, proves identity using a password and authenticator application, receives permission to view personal grades, and generates log records for the login and grade access.

9.3 Authentication Factors

Authentication factors are commonly grouped into:

- **Something you know:** password or PIN.
- **Something you have:** phone, smart card, authenticator, or security key.
- **Something you are:** fingerprint, face, iris, or another biometric characteristic.

Two passwords are not two-factor authentication because both belong to the knowledge category. MFA requires different factor types.

9.4 Password-Based Authentication

Passwords are familiar and inexpensive, but they can be guessed, reused, stolen, phished, or captured by malware.

Password-Based Authentication

Password-based authentication verifies identity using a secret password associated with an account.

Strong Password Practices

Good practices include:

- Use long, unique passwords or passphrases.
- Avoid common and previously compromised passwords.

- Do not reuse passwords across services.
- Use a password manager to generate and store unique passwords.
- Enable MFA, especially for sensitive accounts.

Length is important, but a long predictable phrase may still be weak. A password should not be based on easily discovered personal information.

Credential Stuffing

Credential Stuffing

Credential stuffing is an attack that uses credentials stolen from one service to attempt login to other services.

Unique passwords limit the damage caused by a breach at another service.

Phishing

Phishing attempts to trick users into revealing passwords, one-time codes, or other sensitive information through fake messages and login pages.

Warning signs may include unusual urgency, an unexpected request, a misleading domain, or a request to disclose a password or one-time code.

Secure Password Storage

Systems should not store plaintext passwords or fast unsalted hashes. They should store password verifiers created with a dedicated password-hashing algorithm, unique salts, and current cost parameters.

Chapter 7 explains salts, peppers, Argon2id, scrypt, bcrypt, PBKDF2, password verifiers, and parameter upgrading.

Online Guessing Defenses

Systems can reduce online guessing through:

- Rate limiting and progressive delays.
- Detection of automated attacks.
- MFA.
- Monitoring and alerting.
- Blocking known compromised passwords.
- Secure recovery procedures.

Account lockout must be designed carefully because attackers may deliberately lock legitimate users out.

9.5 Multi-Factor Authentication

Multi-Factor Authentication

Multi-factor authentication requires evidence from two or more different authentication-factor categories.

A password plus an authenticator application combines something the user knows with something the user has.

Why MFA Helps

If a password is stolen, an attacker may still be unable to authenticate without the additional factor. MFA is especially valuable for administrators, instructors, email accounts, financial systems, and accounts containing sensitive information.

Common MFA Methods

Method	Advantage	Important Limitation
SMS code	Broadly available	May be affected by phishing, interception, or SIM-swap attacks
Email code	Simple to deploy	Depends on the security of the email account
Authenticator code	Generated locally	Codes may still be phished
Push approval	Convenient	Vulnerable to approval fatigue and social engineering
Security key	Strong phishing resistance when correctly used	Requires supported hardware and recovery planning

MFA Fatigue

MFA fatigue occurs when an attacker repeatedly triggers approval requests and hopes the user approves one accidentally.

Defenses include limiting prompts, displaying request details, using number matching, warning users about unexpected requests, and preferring phishing-resistant authentication for high-risk accounts.

MFA Is Not Perfect

MFA does not prevent every attack. Malware, session theft, social engineering, insecure recovery, and approval mistakes may still lead to compromise. MFA should be combined with secure sessions, monitoring, access control, and user awareness.

9.6 Biometric Authentication

Biometric authentication uses physical or behavioral characteristics.

Biometric Authentication

Biometric authentication verifies identity using measured physical or behavioral characteristics.

Physical and Behavioral Biometrics

Physical examples include fingerprints, facial patterns, iris patterns, and hand geometry. Behavioral examples include voice, typing rhythm, touchscreen behavior, and movement patterns.

Biometric Templates

Biometric Template

A biometric template is a digital representation of selected biometric features used for comparison.

A system captures a new sample, creates comparable features, and measures similarity with the stored template. Templates are sensitive and must be protected.

False Acceptance and False Rejection

- A **false acceptance** occurs when an unauthorized person is accepted.
- A **false rejection** occurs when an authorized user is rejected.

The decision threshold affects the balance between these errors.

Biometric Risks

Biometrics may be affected by sensor errors, environmental conditions, spoofing, privacy concerns, bias, accessibility limitations, and difficult recovery after compromise.

Biometrics should normally have a secure fallback or recovery method. A biometric characteristic is not secret in the same way as a password and cannot be replaced easily.

9.7 Authorization Concepts

Authorization decisions may use:

- Users and groups.
- Roles and permissions.
- Access control lists.
- Resource ownership.
- Security labels.
- User, resource, action, and environmental attributes.
- Organizational policy.

Authorization must be enforced by the server or trusted system. Hiding a button in a user interface does not prevent an attacker from sending a direct request.

9.8 Access Control Models

Discretionary Access Control

Discretionary Access Control

DAC allows a resource owner to grant or remove access to that resource.

DAC is flexible and common in file-sharing systems, but users may accidentally share information too widely.

Mandatory Access Control

Mandatory Access Control

MAC uses centrally controlled policies, security labels, and clearance rules rather than owner-controlled sharing.

MAC is suitable for highly sensitive environments requiring consistent central control, but it is less flexible and may be complex to administer.

Role-Based Access Control

Role-Based Access Control

RBAC assigns permissions to roles and assigns users to those roles.

Roles may include student, teaching assistant, instructor, department administrator, and system administrator.

Role	View Grades	Edit Grades	Upload Material	Manage Users
Student	Own	No	No	No
Teaching Assistant	Assigned course	Limited	Yes	No
Instructor	Assigned course	Yes	Yes	No
Department Administrator	Department	Limited	Limited	Limited
System Administrator	As required	No	No	Yes

Poorly designed roles can provide excessive access or create role explosion.

Attribute-Based Access Control

Attribute-Based Access Control

ABAC makes decisions using attributes of the subject, resource, action, and environment.

An instructor might enter grades only when assigned to the course, during the approved submission period, using a managed device, and after strong authentication.

ABAC is flexible but requires accurate attributes, clear policy, careful testing, and conflict resolution.

Model Comparison

Model	Decision Basis	Typical Use
DAC	Resource owner	Personal and collaborative sharing
MAC	Labels and central policy	Highly controlled sensitive environments
RBAC	Organizational roles	Universities and enterprises
ABAC	Attributes and context	Fine-grained modern applications

9.9 Least Privilege and Separation of Duties

Least Privilege

Least privilege gives each user, process, application, and service only the access needed for assigned tasks.

Least privilege limits damage from compromise, misuse, and mistakes.

Privilege Creep

Privilege Creep

Privilege creep occurs when access rights accumulate and remain after they are no longer required.

Regular access reviews and prompt removal of old permissions reduce privilege creep.

Separation of Duties

Separation of duties divides sensitive processes among multiple people or roles. For example, one person may prepare a payment while another approves it.

This reduces the risk that one account can complete an entire high-impact process without oversight.

9.10 Account Lifecycle

A secure account lifecycle includes:

1. Request and identity verification.
2. Account creation and unique identifier assignment.
3. Role and permission provisioning.
4. Authentication-factor enrollment.
5. Periodic access review.
6. Permission changes when responsibilities change.
7. Temporary suspension when required.
8. Prompt disabling after departure or loss of eligibility.
9. Retention or deletion according to policy.

Dormant, shared, duplicate, and orphaned accounts create risk. Organizations should maintain clear ownership and regularly review privileged accounts.

9.11 Single Sign-On and Federation

Single Sign-On

Single sign-on allows a user to authenticate once and access multiple connected services without repeating the complete login process for each service.

A university may use one account for email, the student portal, library services, and cloud storage.

Identity Provider and Relying Party

An identity provider authenticates the user and communicates identity information to connected applications. A relying party trusts the identity provider's assertions according to an established federation relationship.

Benefits

SSO can reduce password use, centralize MFA, simplify account disabling, and provide consistent authentication policy.

Risks

Compromise of the central account may affect many services. Identity-provider failure can also interrupt access. Strong authentication, secure sessions, monitoring, recovery planning, and careful application registration are essential.

9.12 OAuth 2.0

OAuth 2.0 is primarily an authorization framework. It allows a client application to obtain limited access to a protected resource without receiving the user's password.

Access Token

An access token represents authorization to access a protected resource under defined conditions.

The client sends the access token to a resource server. Tokens must be protected because possession may allow access until expiry or revocation.

OAuth Roles

A simplified OAuth system includes:

- **Resource owner:** the entity that can authorize access.
- **Client:** the application requesting access.
- **Authorization server:** the service issuing tokens.
- **Resource server:** the API hosting protected resources.

Scopes

Scopes describe requested permissions, such as `read:profile` or `write:calendar`. Clients should request only necessary scopes.

Authorization Code Flow and PKCE

A common secure approach redirects the user to the authorization server. After approval, the client receives an authorization code and exchanges it for tokens.

Proof Key for Code Exchange, or PKCE, binds the authorization request to the client instance that started it. This helps protect intercepted authorization codes, especially in public clients such as mobile applications.

Refresh Tokens

A refresh token may allow a client to obtain a new access token without asking the user to repeat the complete authorization process. Refresh tokens are sensitive, should be narrowly issued, securely stored, rotated or replaced when required, and revoked after compromise.

9.13 OpenID Connect

OpenID Connect, or OIDC, adds an identity layer to OAuth 2.0.

OpenID Connect

OpenID Connect is an authentication protocol built on OAuth 2.0 that communicates information about an authenticated user.

ID Tokens

ID Token

An ID token is a signed token containing claims about an authentication event and the authenticated subject.

An ID token may include the issuer, subject identifier, intended audience, issue time, expiration time, and authentication information.

The client must validate the issuer, audience, signature, expiration, and other required values. An ID token is intended for the client and should not be used as a general API access token.

Access Token and ID Token Comparison

Feature	Access Token	ID Token
Main purpose	Authorize access to an API	Communicate authenticated identity information
Main recipient	Resource server	Client application
Question answered	What access is authorized?	Who authenticated and under what context?

R **Important Point.** OAuth is primarily for authorization. OpenID Connect adds authentication. Access tokens and ID tokens are not interchangeable.

9.14 Secure Token and Redirect Handling

Common protections include:

- Register exact redirect URIs.
- Use the authorization code flow and PKCE where appropriate.
- Validate issuer, audience, signature, expiration, state, and nonce as required.
- Protect access and refresh tokens from scripts, logs, URLs, and unauthorized storage.
- Request minimum scopes.
- Use short token lifetimes appropriate to risk.
- Revoke or rotate tokens after compromise.
- Use trusted protocol libraries rather than custom implementations.

9.15 Session Management

After authentication, many systems create a session. A session identifier represents the authenticated interaction and must be protected like a temporary credential.

Good practices include:

- Generate unpredictable session identifiers.
- Transmit them only over protected channels.
- Use appropriate secure cookie settings in web applications.
- Regenerate the session identifier after authentication or privilege change.
- Apply inactivity and maximum session timeouts.
- Provide logout and server-side session invalidation.
- Reauthenticate before high-risk actions.
- Detect suspicious session reuse or location changes.

Session theft may bypass the original login and MFA process, so authentication security must continue after login.

9.16 Logging, Monitoring, and Audit Trails

Audit Trail

An audit trail is a chronological record used to trace and investigate system activity.

Useful events include:

- Successful and failed authentication.
- MFA enrollment, change, and failure.
- Password and account-recovery events.
- Account creation, disabling, and deletion.
- Role and permission changes.
- Access to sensitive records.
- Privileged operations.
- Token issuance and revocation where appropriate.
- Authorization failures and suspicious activity.

Logs should be protected from unauthorized modification, synchronized to reliable time, retained according to policy, and reviewed. Logs should avoid storing passwords, secret tokens, or unnecessary sensitive data.

9.17 Common Attacks and Mistakes

- Treating identification as authentication.
- Checking authentication but not authorization.
- Enforcing permissions only in the user interface.
- Using weak, reused, or compromised passwords.
- Storing passwords incorrectly.
- Using easily phished MFA for high-risk accounts when stronger options are available.
- Allowing insecure account recovery to bypass MFA.
- Storing biometric templates without adequate protection.
- Giving broad roles or excessive scopes.
- Failing to remove old permissions and inactive accounts.
- Sharing administrator accounts.
- Confusing OAuth with authentication.
- Treating access tokens and ID tokens as interchangeable.
- Failing to validate token issuer, audience, signature, or expiration.
- Using broad or unvalidated redirect URIs.
- Storing tokens in logs or insecure client storage.
- Using long-lived sessions without reauthentication.
- Failing to monitor privileged activity.

9.18 Case Study: University Portal

A secure university portal may use:

1. Unique accounts for students, teaching assistants, instructors, and administrators.
2. Central SSO with strong MFA.

3. RBAC for broad responsibilities.
4. ABAC conditions for course assignment, enrollment, device state, and submission periods.
5. Server-side authorization for every protected request.
6. Separate privileged accounts for administration.
7. Short and protected sessions with reauthentication for sensitive actions.
8. Logging of grade changes, permission changes, login failures, and account recovery.
9. Periodic access review and prompt removal after role changes or graduation.

9.19 Practical Activity

University Portal Access-Control Design

Group Size: 3 to 5 students
Total Marks: 20 marks
Design authentication and access controls for a university portal.
Include these roles:

- Student.
- Teaching assistant.
- Instructor.
- Department administrator.
- System administrator.

Your design must include:

1. An identification and authentication process.
2. MFA requirements for each role.
3. A CRUD permission matrix.
4. RBAC role definitions.
5. At least three ABAC conditions.
6. Least-privilege and separation-of-duties controls.
7. Account lifecycle and access-review procedures.
8. SSO or OIDC usage where appropriate.
9. Required security logs and alerts.
10. At least five attacks and corresponding controls.

Marking Scheme

Assessment Item	Marks
Authentication and MFA design	4
Role and permission matrix	4
ABAC and least-privilege controls	3
Account lifecycle and access review	3
SSO, OAuth, or OIDC explanation	2
Logging and attack analysis	3
Organization and presentation	1

9.20 Chapter Summary

This chapter introduced identification, authentication, authorization, and accountability. Identification claims an identity, authentication verifies it, authorization controls permitted actions, and accountability makes important activity traceable.

Passwords remain common but require unique choices, secure storage, guessing defenses, phishing awareness, and MFA. MFA combines different factor types. Security keys can provide stronger phishing resistance than one-time codes when correctly deployed. Biometrics offer convenience but require protected templates, privacy controls, error management, and recovery methods.

DAC allows owner-managed access, MAC uses central labels and policy, RBAC assigns permissions through roles, and ABAC evaluates attributes and context. Least privilege, separation of duties, account lifecycle management, and periodic access reviews reduce excessive access.

SSO allows one authentication process to support multiple connected services. OAuth 2.0 provides delegated authorization, while OpenID Connect adds authentication. Access tokens authorize API access, ID tokens communicate authentication information to clients, and refresh tokens can obtain new access tokens.

Secure systems also protect sessions, validate tokens correctly, enforce authorization on every protected request, maintain reliable audit trails, monitor suspicious behavior, and remove unnecessary access promptly.

9.21 Additional Exercises

1. Give an example of identification, authentication, authorization, and accountability.
2. Explain why two passwords do not provide two-factor authentication.
3. Compare authenticator codes and security keys.
4. Explain MFA fatigue and two defenses against it.
5. Explain false acceptance and false rejection in biometrics.
6. Compare DAC, MAC, RBAC, and ABAC.
7. Design an RBAC matrix for an online course system.
8. Give three ABAC conditions for grade submission.
9. Explain least privilege and separation of duties.
10. Explain privilege creep and how access reviews reduce it.
11. Design an account lifecycle for teaching assistants.
12. Explain the benefits and risks of SSO.
13. Identify the main OAuth roles.
14. Explain scopes and minimum authorization.
15. Explain why PKCE is useful.
16. Compare access, ID, and refresh tokens.
17. List the checks required when validating an ID token.
18. Explain why secure session management remains important after MFA.
19. Design a logging plan for grade changes.
20. Explain why authorization must be enforced on the server.

9.22 Review Questions

1. What is identification?
2. What is authentication?
3. What is authorization?
4. What is accountability?

5. What are the three common authentication-factor categories?
6. Why are passwords vulnerable?
7. What is credential stuffing?
8. What is phishing?
9. What is multi-factor authentication?
10. Why are two knowledge factors not MFA?
11. What is MFA fatigue?
12. Which MFA methods can resist phishing more effectively?
13. What is biometric authentication?
14. What is a biometric template?
15. What are false acceptance and false rejection?
16. What is DAC?
17. What is MAC?
18. What is RBAC?
19. What is ABAC?
20. What is least privilege?
21. What is separation of duties?
22. What is privilege creep?
23. Which stages belong to the account lifecycle?
24. What is single sign-on?
25. What is an identity provider?
26. What is OAuth 2.0 mainly used for?
27. What is an access token?
28. What is a scope?
29. What is PKCE?
30. What is a refresh token?
31. What is OpenID Connect?
32. What is an ID token?
33. How is an ID token different from an access token?
34. Which ID token values should a client validate?
35. What is session management?
36. Why should session identifiers be regenerated after login?
37. What is an audit trail?
38. Which authentication and authorization events should be logged?
39. Why are shared administrator accounts harmful to accountability?
40. List five common authentication or access-control mistakes.